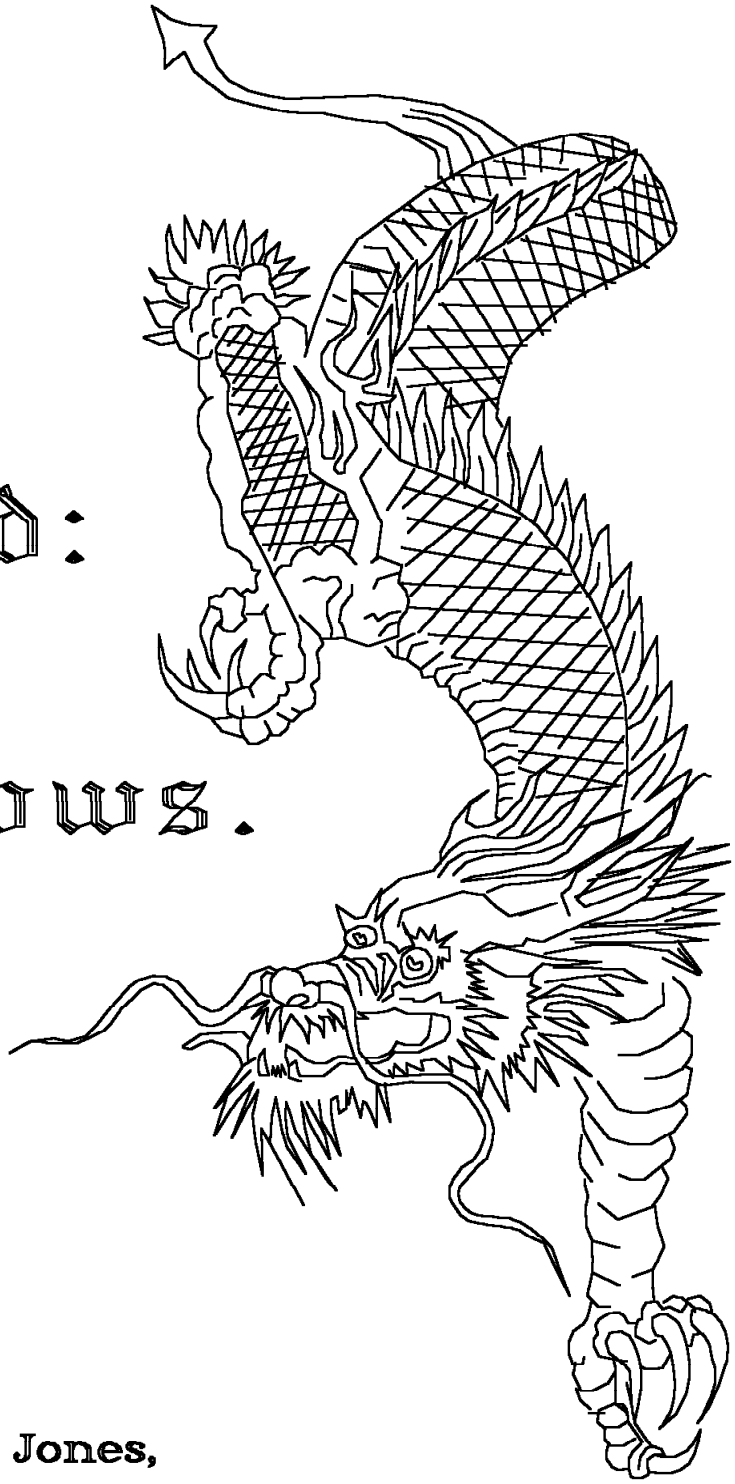
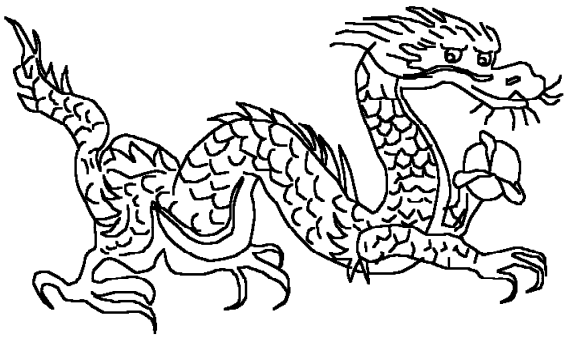
Neatwind: The Art of Windows.





Brendan Jones,
IBM Australia/New Zealand.

THE ART OF WINDOWS: AN INTRODUCTION TO NEATWIND.



Document Number NW-UG-6.0-0

13 March 1991

Brendan Jones.
BJONES at SYDVM1.

IBM Australia/New Zealand.
Coonara Avenue,
West Pennant Hills, 2125.
Australia.

Abstract

Neatwind is an easy to use, yet powerful dialogue manager that runs under the VM CMS operating system. It is easily called from programs written in the user's chosen language, such as REXX or APL.

This document provides the application programmer with an introduction to Neatwind. Prerequisites to this document are a working knowledge of REXX and CMS, and a reasonable level of programming skills.

© International Business Machines Corporation, 1990, 1991.

Table of Contents

NEATWIND. **1**

1.0 Simple Dialogue. **2**
1.1 Colours. 3
1.2 Style. 3
1.3 Positioning. 4
1.3.1 Using the whole screen on 32 line displays. 4
1.3.1.1 Drawing Windows. 4
1.3.1.2 Drawing FULLSCREEN panels. 4
1.4 Queries. 5

2.0 Icons. **6**
2.1 The PSLOAD Option. 6
2.2 Icons. 6
2.2.1 Filler Icons. 7
2.2.2 The Progress Bar. 8
2.3 Emphasised Text. 9
2.3.1 Multilingual characters. 9
2.3.2 Compatibility. 10

3.0 More windows. **11**
3.1 MENU windows. 11
3.1.1 Embedded Input Fields. 12
3.1.2 Defaults for MENU windows. 12
3.2 PICK windows. 12
3.2.1 Defaults for PICK windows. 13
3.2.1.1 Using the NOBLANK Option. 14
3.2.1.2 Using the DEFAULT Option. 14
3.2.1.3 Bit Strings. 14
3.3 The LOCK Option. 14
3.4 The Programmable Function Keys. 15
3.4.1 Reserved Function Keys. 15
3.4.2 Customised Function Keys. 15
3.4.2.1 The effect of styles on PF keys. 16

4.0 Vertical Scrolling. **17**
4.1 Tailoring your Scroll Options. 17
4.2 The MORE Icon. 17

5.0 Data Entry Screens. **19**
5.1 Field Validation. 19
5.2 Field specification 20
5.2.1 Checking other Types. 21
5.2.2 DATE and TIME checking. 21
5.2.2.1 Your own format. 22
5.2.2.2 Constraints. 22
5.2.2.3 Restrictions. 23
5.2.3 The EXEC Option. 23
5.2.4 Checking in the Application. 24

5.2.5 When to check? . . . 24
5.3 Initialising Values. . . . 25
5.4 Getting the results back. . . . 25
5.4.1 The OUTVARS option. . . . 25
5.4.2 Stems . . . 25
5.4.3 The STACK Option. . . . 26
5.4.4 The GLOBALV Option. . . . 26
5.4.5 The NOBLANK Option. . . . 26
5.4.6 The UPPERCASE Option. . . . 26
5.4.7 The STRIP Option. . . . 26
5.5 READONLY Fields. . . . 26
5.5.1 The Backslash Method. . . . 27
5.5.2 The PERMISSION Option. . . . 27
5.5.3 Security. . . . 28
5.5.4 The PROTECT option. . . . 28
5.6 The INVISIBLE option. . . . 28
5.7 Did they change anything? . . . 29
5.8 Getting the spacing right. . . . 29
5.8.1 Normal spacing. . . . 30
5.8.2 With the COMPRESS option. . . . 30
5.9 More colours . . . 30

6.0 Context Sensitive Help. . . . 32
6.1 Defining a HELP key. . . . 32
6.2 Help for what? . . . 32
6.3 The Question Mark. . . . 32
6.4 Contextual Help Example. . . . 33
6.5 Now, where was I? . . . 33
6.5.1 Format of NEAT$POSN. . . . 34

7.0 Action Bars. . . . 36
7.1 Using Action Bars in Neatwind. . . . 36
7.1.1 Pull-downs. . . . 37
7.1.2 What action did they select? . . . 37
7.1.3 Action Bar Example. . . . 38
7.2 Standard Actions. . . . 38
7.2.1 File. . . . 39
7.2.2 Edit. . . . 39
7.2.3 View. . . . 40
7.2.4 Options. . . . 40
7.2.5 Help. . . . 41
7.2.6 Standard Pull downs. . . . 41
7.2.7 Object Oriented Applications. . . . 41
7.2.7.1 The ASSIST Option. . . . 42
7.2.7.2 Defining your Objects. . . . 42
7.2.7.3 Example. . . . 43
7.2.7.4 What we haven't discussed. . . . 44

8.0 Integrated Help. . . . 45
8.1 Preparing the Help Documentation. . . . 45
8.1.1 Naming Conventions. . . . 45
8.1.2 Creating the Document. . . . 46
8.1.3 The NEATHELP Imbed. . . . 47
8.1.4 The NEATMAKE Tool. . . . 48
8.1.4.1 Other languages. . . . 48
8.2 Presenting the Help. . . . 49
8.2.1 The HELP Option. . . . 49
8.2.2 The Help Pull-down. . . . 50
8.2.3 Returning from Neatwind. . . . 51
8.3 Viewing the Help. . . . 51
8.4 Using NEATHELP Standalone. . . . 52

8.5 Trouble Shooting. 53

9.0 Dynamic Data Entry Windows. 54
9.1 The TEMPLATE Option. 54
9.2 The Add and Delete PF keys. 55
9.3 Implementing the Dynamic List. 55

10.0 Horizontal Scrolling. 56
10.1 The MARGINS Option. 56
10.2 The Left and Right PF keys. 56
10.3 Alignment. 57
10.3.1 Lines with Field Marks. 57
10.3.2 Lines without field marks. 58
10.4 Overlapping Margins. 58
10.5 Displaced Margins. 58
10.6 Presentation. 59
10.7 The PREFIX Option. 60
10.7.1 Use with Field Marks. 61
10.8 Restrictions. 61

11.0 SAA 90. 62
11.1 The PF3 key. 63

12.0 Goodies. 64
12.1 Titling your windows. 64
12.2 The ENTEROK Option. 64
12.3 The VANISH Option. 64
12.4 The CONFIRM Option. 65
12.5 The XCONFIRM Option. 65
12.6 The WAIT Option. 66
12.7 The BEEP Option. 66
12.8 The CTLCHAR Option. 66
12.9 Using your own fonts. 67

13.0 National Language Support. 68
13.1 Changing your terminal's tongue. 68
13.1.1 Codepages. 69
13.1.2 NLS Character Representation. 69
13.1.3 The NEATCODE file. 70
13.1.3.1 Simulating diacritic characters. 71
13.1.3.2 Do I need to create a NEATCODE file? 71
13.1.3.3 Creating the NEATCODE file. 71
13.1.4 Going it alone... 72
13.1.5 Field Specification Characters. 73
13.1.5.1 The NLS Escape Character. 73
13.2 Changing Neatwind's Tongue. 73
13.2.1 Creating the NEATNLS file. 74
13.2.1.1 Translating the Messages. 74
13.2.1.2 Message Width. 74
13.2.1.3 Translating the PF keys. 75
13.2.1.4 The Version Number. 76
13.2.1.5 Into action. 76
13.2.2 Action Bar Functions. 76
13.3 Changing your application's tongue. 77
13.3.1 Defining your Panels File. 77
13.3.2 Displaying the panel. 78
13.3.3 Substitutions. 78
13.4 Integration. 78
13.5 Non-Latin Language Support. 79
13.5.1 DBCS Languages. 79
13.5.2 Middle Eastern Languages. 80

13.6 Finding out more. 80

14.0 Making Neatwind faster. 81
14.1 The NUCXLOAD. 81
14.2 Calling compiled Neatwind. 81
14.3 The NUCXDROP. 82
14.4 Detecting the runtime library. 82

15.0 Return Codes. 83

16.0 Customising Neatwind. 84
16.1 The Control Panel. 84
16.1.1 The Control Panel Menu. 84
16.1.2 Screen Colours. 85
16.1.3 Warning Beep. 86
16.1.4 National Language Preferences. 86
16.1.5 Country. 87
16.2 Application Considerations. 87
16.2.1 Respecting the user's choices. 87
16.2.2 Changing preferences from within. 87
16.2.3 Disabling the Control Panel. 88

17.0 The Programmer's Toolkit. 89
17.1 The NEATQRY Program. 89
17.2 The NEATTRAN Program. 90
17.2.1 Dependent to Independent Translation. 90
17.2.2 Independent to Dependent Translation. 91

18.0 Think of the User. 92
18.1 CUA Design Guidelines. 93

19.0 Support. 95
19.1 Prerequisites. 95
19.2 The future of Neatwind. 95

Appendix A. National Language Codes. 97

Appendix B. Terminology of Codepages. 100
B.1 Syntactic Character Set. 100

Appendix C. Codepages 102

Appendix D. Country Identifiers. 103

Index 106

Neatwind.

Neatwind is a windowing utility that:

- has a simple interface
- is easy to learn and use
- requires only a few lines to call
- provides a wide range of dialogue windows
- automatically sizes the window
- allows you to select your own colours
- has a minimal interface, but there are options for more elaborate windowing...
- provides national language support
- is compatible across all IBM's 3270 architecture terminals.

  Neatwind shall *never* refuse to run on a terminal just because it lacks some feature. Neatwind always will automatically look for and use a substitute feature.

- advocates programmer and end-user ease of use and productivity.

Neatwind's biggest advantage is that it reduces your time spent coding windows and full-screen panels. To define a window you simply specify what you want it to look like, and what you want it to do. You let Neatwind worry about doing the rest. Once you specify the window to Neatwind your job is done.

This also eliminates all the errors you might code if you were recoding the dialogue handling yourself, and leaves you with more time to concentrate on the interesting and unique parts of your application. You can also use the complementary SQLCOMP package from VMTOOLS to eliminate the database handling too, drastically cutting your code size and increasing its quality simultaneously.

A simple window to display a message can literally be coded in 10 seconds. A more sophisticated one can be done in 10 minutes once you become familiar with Neatwind. As an example, a 1200 line program that would have taken 2-3 days to write I was able to do in 350 lines in 2 hours.

On with the tutorial...

1.0 Simple Dialogue.

Let's begin with a simple window which displays a short textual message a few lines long. There are two ways we can pass this message to Neatwind. The first is to queue the message on the program stack using the REXX `Queue` command.

```
'MAKEBUF'
 $stk = Queued()
 Queue "Know the enemy and know thyself,"
 Queue "  and in a hundred battles you will never be defeated."
 Queue "When you are ignorant of the enemy but know yourself,"
 Queue "  your chances of winning or losing are equal."
 Queue "If ignorant both of the enemy and of yourself,"
 Queue "  you are sure to be defeated in every battle."
 Queue " "
 Queue "                              'The Art of War', Sun Tzu."
'EXEC NEATWIND' Queued() - $stk '(MESSAGE'
'DROPBUF'
```

We simply queue each line to be displayed one by one. The way we have spaced the lines will be preserved by Neatwind when the window is displayed. The first argument to Neatwind is a number which tells it the number of message lines you have queued on the stack for it. Rather than hardcode this value, we have used the expression `Queued() - $stk`. This lets us insert and delete lines in the queued message without having to count lines and update any hardcoded constants.

The next argument to Neatwind is an opening bracket `(`. This signfies the start of the options section of the command. Immediately following this is the word `MESSAGE`. There are a wide range of windows available in Neatwind. `MESSAGE` is the simplest. It displays the message it is passed on the screen, and returns when the user presses the Enter or PF=END (function key 3) to continue.

Here is what the displayed window would look like:

```
  Know the enemy and know thyself,
    and in a hundred battles you will never be defeated.
  When you are ignorant of the enemy but know yourself,
    your chances of winning or losing are equal.
  If ignorant both of the enemy and of yourself,
    you are sure to be defeated in every battle.

                             'The Art of War', Sun Tzu.

  PFK: 3-END or Enter to continue.
```

Instead of using the stack to pass the message you can put it in a stemmed REXX variable. For example:

```
 fred.0 = 8
 fred.1 = "Know the enemy and know thyself,"
 fred.2 = "  and in a hundred battles you will never be defeated."
 fred.3 = "When you are ignorant of the enemy but know yourself,"
```

```
 fred.4 = "  your chances of winning or losing are equal."
 fred.5 = "If ignorant both of the enemy and of yourself,"
 fred.6 = "  you are sure to be defeated in every battle."
 fred.7 = " "
 fred.8 = "                              'The Art of War', Sun Tzu."
'EXEC NEATWIND (SHOWSTEM FRED. MESSAGE'
```

Each line to appear in the window is placed in one line of the stem. The first line appears in `fred1`, the second in `fred.2`, etc. The zeroth element of the stem holds the number of lines in the window. Since there are 8 lines in this case we set `fred.0 = 8` Because we are passing the number of lines through this element there is no need to pass this value as the first argument to Neatwind as we did with the stack example. The `SHOWSTEM` option passed to NEATWIND tells Neatwind to use the `FRED.` stemmed variable instead of the stack. Note that the variable specified with the SHOWSTEM option should be in upper case.

It really doesn't matter whether you use the program stack or a REXX stem as the medium to pass your window text. Use whatever you feel most comfortable with, or what best suits your current application. (Stems are good when the window text remains basically unchanged. The stack is good when you are displaying variable data, such as lists.)

1.1 Colours.

If you don't like Neatwind's default colours you may override them with your own choices. There are seven colours available; White, Red, Pink, Green, Yellow, Blue and Turquoise. When you specify these colours you may abbreviate them down to one letter (eg. R for Red.) If these are not supported on a user's terminal the hardware of that terminal will display alternates.

`FOREGROUND` is the colour of the windows border, and the text within it. For example, to select a blue window you would add the option `FOREGROUND=BLUE` to the Neatwind command line. You may abbreviate this right down to `FORE=B`.

When the window is overlaid this foreground colour sometimes washes out onto the text to the right of the window. For example, if you overlay a green window on a screen filled with blue text then some of that blue text will change to green at the fringes of the window. This doesn't change the functionality of of the window in any way, but if you do want a nicer "look" set the `BACKGROUND` option to the predominant colour of the text your are overlaying. In this example we would set `BACKGROUND=BLUE`. If you don't pick the correct colour, or you don't know what colour will be there don't worry about it.[1]

1.2 Style.

Neatwind supports three different styles of windows. The style used affects the default colours, the layout of the PF key line and a few things we shall discuss later. Simply select the style you prefer by specifying the relevant option as a Neatwind option.

- `SAA87`.

  This style abides by the initial SAA CUA standard released in 1987.

- `SAA90`.

  This style abides by the revised SAA CUA standard released in December 1989 and gained usage during 1990. This revision has some reasonably drastic changes, so we had to add it as a separate style rather than upgrading the previous one. We'll discuss these differences in more detail later.

---

[1] The `BACKGROUND` option has no effect on (Asian Language) DBCS terminals. On these, the background will always be green.

- `I/S`.

  This is based on the old I/S screen standard which was predominant before the current CUA dialogue guidelines were released. There are still many older applications in use which use this style. The CUA reference says it is optimised for ease of use by experienced users, rather than being easy to learn by unexperienced ones. The I/S style may be a better choice for the latter audience, as it tends to be better at prompting the user.

We shall use the `I/S` standard within this document, though most people will probably decide to use the `SAA87` or `SAA90` option instead.

1.3 Positioning.

The `ROW` and `COLUMN` options specify where you want the upper-left hand corner of the window positoned. If you specify the `COLUMN` too far to the right Neatwind will automatically push the window back onto the screen. Same applies to the `ROW` option. For example, `ROW=4 COLUMN=20` would position the window to start at the 20th column on the 4th row.

Neatwind isn't restricted to just windows. You may ask it to draw a full-screen panel by specifying the `FULLSCREEN` option. In this case the `ROW` and `COLUMN` options are ignored.

1.3.1 Using the whole screen on 32 line displays.

In order to understand how to use the whole screen you must understand how 32 line displays operate.

There are two popular display sizes, 24 and 32 lines. The most common are the 24 line displays, which are always in 24 line mode. However, the 32 line displays can be in either 32 or 24 line mode. When a 32 line display is in 24 line mode we can change any of the 24 lines displayed, but if we write to one of the 8 not displayed (such as the 25th) we get an I/O error.

1.3.1.1 Drawing Windows.

When Neatwind draws windows it always restricts itself to the first 24 lines of the display. This prevents the above error, as it will never try changing (eg. the 25th) line of a 32 line display in 24 line mode.

However, if you know the display is displaying the maximum number of rows possible you can tell Neatwind to go ahead and use the whole screen by specifying the `USEALL` option.

If you are drawing a window on a `FULLSCREEN` panel or just called `VMFCLEAR` to force a screen clear (which switches the display to its maximum screen size) then you may specify the `USEALL` option to make maximum use of the display.

The `NOUSEALL` option tells Neatwind it may only use the first 24 lines of the display. This is the default for windows.

1.3.1.2 Drawing FULLSCREEN panels.

On the other hand when you call Neatwind asking it to draw a `FULLSCREEN` panel, it assumes the display is already in its maximum mode. That is, if called on a 32 line display it assumes that display is in 32 line mode.

To make sure the screen is in maximum mode you should call the CMS command `VMCLEAR` just before you call Neatwind. Of course if you forget to do this and you call Neatwind on a 32 line display in 24 line mode no great harm will be done. Neatwind will try and update the screen, realise what has happened, force its own screen clear, and then update the screen in full 32 line mode. However this take a few seconds longer and causes a blink, so you're better off doing the `VMCLEAR` yourself.

We could have got Neatwind to clear the screen itself everytime it is called in FULLSCREEN mode, but this would slow it down. One of the advantages with the approach we have taken is you'll note that inside the screen Neatwind has very good response time.

USEALL is the default setting for FULLSCREEN panels. You can force FULLSCREEN panels to be drawn in 24 line mode by specifying the NOUSEALL option.

1.4 Queries.

Frequently you will want to ask the user a simple question. For example, “Do you really want to delete ... ?” Here is an example of such a window.

```
 jill.0 = 2
 jill.1 = "Do you really want to delete"
 jill.2 = "this chapter of the proposal?"
'EXEC NEATWIND (SHOWSTEM JILL. QUERY'
 select
   when rc = 1 then Say 'They said Yes'
   when rc = 0 | rc = -10 then Say 'They said No'
   otherwise Say 'NEATWIND Error. Unexpected return code =' rc
 end
```

This will display the following window:

```
  Do you really want to delete
  this chapter from the proposal?

  PFK: 12-CANCEL or answer Y or N: _
```

For such a window specified with the QUERY option there are three possible return codes. The return code is 1 if they answer yes, 0 if they answer no, and -10 if they press the PF=Cancel key. If they do press the Cancel key you should take the path of least damage. In the above case this would be the same as “No” (that is, “No, I don't want to delete ....”)

You may specify default answers (the most likely one they will want to type) by specifying the DEFAULT option. For example, DEFAULT=Y.

2.0 Icons.

Neatwind has some sophisticated features such as icons and emphasised text. If you're lucky enough to have a terminal with PSS capability Neatwind will present special graphical versions of these. If you're not, Neatwind will automatically look for other character sets on your terminal to do the same. This chapter discusses exploiting these features.

2.1 The PSLOAD Option.

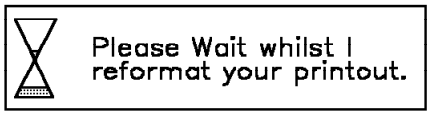
If you intend to use icons or the bold or italic font then the very first call to Neatwind in your application should specify the option `PSLOAD`. This should be done regardless of whether that first call uses the advanced features or not.

You could in theory specify this option on every call to Neatwind in your application, but its is slightly more efficient and cleaner just to do it on the first.

The `PSLOAD` option downloads the graphical icons and fonts to the user's terminal. There is no harm specifying this option for a user whose terminal does not support Programmable Symbol Sets. Neatwind will recognise this and know not to try and download the font.

2.2 Icons.

Icons can be used to make a message clearer to the user. For example, if you want to specify a delay you would display an hour glass icon along with a message telling the user to wait for a minute. Neatwind lets you incorporate icons into your window very easily. Lets's look at a simple window with an hourglass icon.



Here is the code fragment that produces this window:

```
'MAKEBUF'
 Queue '&WAIT1.Please wait whilst I'
 Queue '&WAIT2.reformat your printout...'
'EXEC NEATWIND 2 (MESSAGE NOWAIT PSLOAD'
'DROPBUF'
```

Although we have specified `PSLOAD` here, if it was specified on an earlier window displayed in the application it would not be necessary.

The `NOWAIT` option returns immediately from the window without waiting for the user to press anything. This is ideal for these “Please Wait” options.

Because the hourglass icon is so large it is split into two icons; &WAIT1. and &WAIT2. We position these one under another in the above example. Note that the embedded icon name is prefixed by an ampersand, and terminated by a period (or “full stop.”)

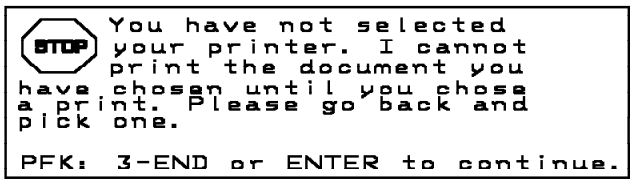
Neatwind examines the user's terminal, and automatically inserts the appropriate icon when the window is displayed. This could be a graphical icon if the user's terminal has programmable symbol set (PSS) support. If the user is running a text terminal equivalent icons will be pieced together from whatever characters are available.

Here is a list of all the available icons:

```
INFO        An information symbol ("i" in a blue circle.)
WARN        A warning sign (black exclamation mark in a yellow circle.)
ASK         Ask a question (a question mark in a green circle)
WAIT1       An hourglass (to indicate a "Please Wait.")
WAIT2         Indicates a short delay.
STOP1       A stop sign (to say "You can't do this because....")
STOP2
STOP3
ERROR1      Used to indicate a serious error (the jolly roger)
ERROR2
IBM         The striped IBM logo.
COPYR       The international copyright symbol (c)
BULLET      A round, circular bullet (used to flag a point)
SMILE       A smiling face
PROGRESS1   A progress bar.
PROGRESS2     Indicates a long delay.
```

2.2.1 Filler Icons.

Sometimes the icon may only take up the first few lines of your window, and the message may continue for several more lines. This causes the text to misalign. Here is an example of a misaligned window:

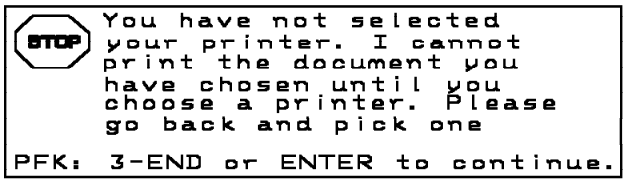


If you're happy with this, fine. If you want the text to line up through the window you should use a filler icon. There is a filler icon corresponding to each of the icons listed above. Remove any numeric (eg. 1) at the end of the icon name, and append an “F.” eg. `STOP1` becomes `STOPF.` `WARN` becomes `WARNF.` These filler icons are automatically replaced with the correct number of blanks to line the text up. You could hardwire the blanks directly, but these would only line up correctly on PSS or non-PSS terminals, but not both. Filler icons are device independent.

Here is code that uses the filler icons:

```
'MAKEBUF'
 Queue "&STOP1.You have not selected"
 Queue "&STOP2.your printer.  I cannot"
 Queue "&STOP3.print the document you"
 Queue "&STOPF.have chosen until you"
 Queue "&STOPF.choose a printer. Please"
 Queue "&STOPF.go back and pick one."
'EXEC NEATWIND (MESSAGE'
'DROPBUF'
```

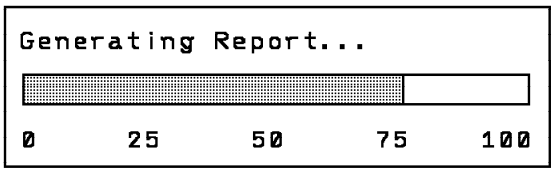
Here is the window it produces:



2.2.2 The Progress Bar.

CUA stipulates that for short delays you display the hour-glass, and for longer ones you display the progress bar. This is, quite simply, a horizontal bar with a scale numbered from 0 to 100%. As your application works through its task it slowly fills up to 100%, at which point the task is completed and control returns to the user.

Here is an example of a progress bar:



To produce this window simply embed the &PROGRESS1. and &PROGRESS2. icons in your window, one under the other. Specify the percentage of work done so far using the `PERCENT` option. For example, `PERCENT=75` in the above window. Specify the `NOWAIT` option so they user doesn't have to hit Enter every time this window is updated.

Here is a code fragment that would produce the above window:

```
total = 50
do i = 1 to total
  pct = i / total * 100      /* how much have we done so far? */
 'MAKEBUF'
  Queue 'Generating Report...'
  Queue ' '
  Queue '&PROGRESS1.'
  Queue '&PROGRESS2.'
 'EXEC NEATWIND 4 (MESSAGE SAA87 NOWAIT PERCENT='pct
 'DROPBUF'
  Call generate_line i       /* do work here */
end i
```

As your program steps through its task call Neatwind to update the progress bar. The resolution of the progress bar is in 5% units, but there is no problem calling it at any point between these. (eg. `PERCENT=68.4` is perfectly valid).

Unfortunately, there is no way to provide the user with the ability to “stop” the application, aborting the task it is performing. They'll just have to sit it out.

2.3 *Emphasised Text.*

Neatwind gives you many ways of emphasising text within the window. You may do this by using either highlighting, an emphasised bold or italic font, reverse video or different colours.

To switch to an emphasised text mode imbed the percentage sign `%` in your window text, and immediately follow it by one of the following characters; An “H” for highlighting. An “E” the an emphasised bold font. An “I” for an italic font. Or finally, a “V” for reverse “flared” video. To switch back to the default window colour (which you should also do), follow the percentage sign by an “F” meaning foreground. These letters may be specified in either upper or lower case.

If you don't like the default colours (picked depending on your style option), you may change them using the `HILITE, BOLD, ITALIC, FLARE` or `FOREGROUND` options.

Here is an example:

```
'MAKEBUF'
 Queue "  This is%Ha hilighted phrase.%F"
 Queue "  This is a %Ebold%F word, but these are not."
 Queue "  Maybe you want to speak in%Iitalics?%F"
 Queue "%RThis whole line is in reverse video.%F"
'EXEC NEATWIND 3 (MESSAGE'
'DROPBUF'
```

If you like, you may set the colours directly in the window. A percentage symbol followed by an uppercase `R, W, P, G, Y, B` or `T` will switch to that corresponding colour (Red, White, etc.) Their lower case equivalents do the same, but do so in bold font. If your terminal doesn't support bold font then the text to the right of your escape will be highlighted instead.

Note that the escape sequences take up two characters. By default, they will also take up two characters in the displayed window. It therefore is a good idea to squeeze the highlight text right up against them. For example; `This is a%eBold%fword` instead of, `This is a %e Bold %f word.` However if you specify the `COMPRESS` option, the escape sequence will only take one space on the displayed window.

You're better off using the “H I F E V” escape characters because Neatwind sets them to the appropriate colours according to your style (for example, “CUA 1987 standard.”) If you hardcode all your colours you'll make it more work to convert to later revisions of the CUA standards.

Although “*%*” is the default escape character, you may define your own using the `ESCAPE` option. For example, `ESCAPE=~` would be used to change it to a tilde. You might do this if you have percentage signs in the window text you want to be displayed literally.

2.3.1 Multilingual characters.

Multilingual characters such as “ö” are not available in the bold or italic fonts. If you attempt to type these directly from the keyboard garbage will appear in their place.

However, if you use NLS Escape Sequences as documented in “NLS Character Representation.” on page 69, Neatwind will approximate the multilingual characters as best it can. For example, “attaché” would be approximated as “attache.”

A word of warning though; in some languages dropping accents and the like can significantly change the meaning of the word. For example the French word “cote” which means *quote* or *bid.* However, “côte” means *coast*, “coté” means *measured* and “côté” means *side.* Accented characters also sound different from their less florid counterparts. If you have such a word in which the accent must be shown then we suggest you use colour, rather than the bold or italic fonts to emphasise it.

2.3.2 Compatibility.

Before Neatwind was converted for multi-country operation it used control characters only available on a U.S. keyboard. For backward compatibility these characters are still supported, but their use is not recommended.

These characters are "~" to switch to flared reverse video, "}" for the bold emphasised font, or "{" or "¢" for highlighted text mode. "!" switches to foreground mode. These characters are however not available on all terminals, and on some (German) for example, they have different character codes. That means even if you can find that character on one of these keyboards it won't necessarily work!

Because of this use of these characters is strongly discouraged. To write a truly portable application we suggest you use the "H I F E V" control characters described above.

3.0 More windows.

Let's look at slightly more sophisticated windows.

3.1 MENU windows.

The first will be a simple menu. It will display a list of choices and let the user pick one.

For example;

```
'MAKEBUF'
 Queue " 1. Place an Order."
 Queue " 2. Lookup an Order."
 Queue " 3. Add a Customer."
 Queue " 4. Delete a Customer."
 Queue " P. Print off address labels."
'EXEC NEATWIND (MENU=1234P'
 $rc = rc
'DROPBUF'
```

will display the following window;

```
  1. Place an Order.
  2. Lookup an Order.
  3. Add a  customer.
  4. Delete a customer.
  P. Print off address labels.

  PFK: 12-CANCEL or your selection: _
```

The user can enter any character specified in the menu option. In our case we specified `MENU=1234P`, so the user can enter 1, 2, 3, 4 or P as a response. Neatwind will not let them enter any other character.

The return code will be set to the position of the corresponding character in the menu option. That is, rc=1 for 1, rc=2 for 2, rc=3 for 3, rc=4 for 4 and rc=5 for P.

Note that it is up to you how you format menu in the window text. In fact, if you omit an option from the menu but leave it in the `MENU=1234P` option, Neatwind will still accept it. What you display in the window text is entirely up to you.

If the user presses PF=CANCEL instead of filling in the window then Neatwind will give a return code rc=-10. When the user does this they are saying “I changed my mind and don't want to answer this window.” Usually pressing this key should take them back to the main menu of your application.

3.1.1 Embedded Input Fields.

On `MENU` windows Neatwind will place the input field on the PF key line. CUA says that it should appear to the left of the first option in the window text. Thus:

```
     _  1. Place an Order.
        2. Lookup an Order.
        3. Add a  customer.
        4. Delete a customer.
        P. Print off address labels.

  F3=Exit F12=Cancel Enter
```

If you want your window to look like this you should place an at-sign character "@"[2] where you want the input field to be. Thus:

```
'MAKEBUF'
 Queue "@1. Place an Order."
 Queue "     2. Lookup an Order."
 Queue "     3. Add a Customer."
 Queue "     4. Delete a Customer."
 Queue "     P. Print off address labels."
'EXEC NEATWIND (MENU=1234P'
 $rc = rc
'DROPBUF'
```

Note the adjusted spacing to cater before the two spaces that Neatwind has to insert before and after each input field. If you want to know more about getting the spacing right see "Getting the spacing right." on page 29.

3.1.2 Defaults for MENU windows.

You can specify the default choice with the `DEFAULT` option. For example, `DEFAULT=P` in the above example would cause `P` to appear in the menu input field. The user could then press Enter straight away to take the default choice. Alternately, they may change it to something else before pressing enter.

By anticipating the choice you expect the user to take, it is you can save them time. Of course there are many screens where you cannot predict the user's likely choice, and thus displaying one would only confuse them. On these screens (such as an application menu), you should not provide any default.

3.2 PICK windows.

These windows are usually used for filling in options. For example;

```
 w.0 = 8
 w.1 = "&ASK.What would you like to eat?"
 w.2 = " "
 w.3 = "   Roast Beef  @"
 w.4 = "   Chicken     @"
 w.5 = "   Sushi       @"
 w.6 = " "
 w.7 = "You cannot have all three meals."
```

---

[2] If you are using a non-English terminal then this character may be different. See "Field Specification Characters." on page 73.

```
 w.8 = "Place an X beside your selection"
'EXEC NEATWIND (SHOWSTEM W. PICK=1-2 OUTVARS (BEEF CHICKEN FISH)'
 if beef    then say 'The Beef is good...'
 if chicken then say 'The Chicken is fine...'
 if fish    then say 'The Sushi tastes like raw fish!'
```

will display this window:

```
   (?) What would you like to eat?

   Roast Beef  _
   Chicken     _
   Sushi       _

  PFK: 12-CANCEL or select and ENTER
```

First, note the `PICK=1-2` option. That says the user can pick one or two of the listed options, but not both. `PICK=1` would only let them pick one option. `PICK=-3` would let the pick up to 3. `PICK=ANY` (the default) lets them pick any number of options.

The `OUTVARS` option says to set each REXX variable to 1 (meaning true) if it is selected, or 0 (meaning false) if it is not. The `BEEF, CHICKEN` and `FISH` variables are set in the same way. The variable and the name in the window text don't have to be the same. We could have gone `OUTVARS (FOOD1 FOOD2 FOOD3)` had we wanted to, but `OUTVARS (BEEF CHICKEN FISH)` just happens to be signficantly easier to understand.

The at signs "@" we have embedded in the window become input fields in the displayed window. This allows you to place them wherever you like within the window.

The user then edits the list, sticking an "X" beside options they want to select, and a blank beside those they don't. If you are using SAA, then a slash "/" is the valid selector character, though Neatwind will accept both in either mode.

If they fill in the window *correctly,* and leave (either by pressing Enter or PF3) then their selections will be passed back via the `OUTVARS` variables. If however they press PF=CANCEL Neatwind will end giving a return code of rc = -10, and the REXX variables will not be passed back.

For example, suppose the user fills in the window like this:

```
   (?) What would you like to eat?

   Roast Beef  X
   Chicken     _
   Sushi       X

  PFK: 12-CANCEL or select and ENTER
```

If they pressed enter, then the REXX variables `beef` and `fish` would both be set to 1 (meaning true). Since chicken was not selected, the variable `chicken` would be set to 0 (meaning false.) Simple, huh?

3.2.1 Defaults for PICK windows.

Suppose most people order the Chicken and Beef, prefering to avoid the Sushi. We can make this selection the default, so that when the window appears Chicken and Beef are already selected. If this is what the user wanted, they can press Enter straight away without having to change anything.

3.2.1.1 Using the NOBLANK Option.

We can set the defaults by initialising the variables specified in the OUTVARS option to 1 if you want it selected as a default, and 0 if you do not. For example,

```
 beef    = 1     /* true  */
 chicken = 1     /* true  */
 fish    = 0     /* false */
'EXEC NEATWIND (SHOWSTEM W. PICK=1-2 OUTVARS (BEEF CHICKEN FISH) NOBLANK'
```

See the NOBLANK option we have added? This tells Neatwind to look at the current values of the variables to determine which ones are selected by default. If you don't specify NOBLANK then these variables are all set to blank, which means by default none of them will start off selected. This would happen even if you had initialised their values beforehand.

3.2.1.2 Using the DEFAULT Option.

Another way to set defaults is to use the DEFAULT option. For example, DEFAULT=110 says that the first and second entries are selected, and the third is not. This is because while the first and second chatacters are set to 1, the third is set to 0. We call this string of 1s and 0s a bit string. If you use this option you don't have to initialise the variables as we did in the above example.

There is no difference whether you choose to set your defaults by using the DEFAULT option, or by presetting the variables and specifying the NOBLANK option. Use whichever you happen to feel more comfortable with.

3.2.1.3 Bit Strings.

By the way, if you prefer to have a bit string returned by OUTVARS rather than individual variables (the same way as the DEFAULT option), then simply specify the OUTVARS option with a single variable. For example, If we had specified OUTVARS (FOOD) for our window and the user had selected only chicken and beef, then the variable FOOD would be set to the corresponding bit string. In this case, 110.

If you forget the OUTVARS option when you draw a PICK window the selections will be returned via the program stack. They will be in bit string format.

3.3 The LOCK Option.

When Neatwind overlays a window on a screen it leaves whatever is underneath the way it found it. This means if the user cursors out of the window they can type in input fields on other parts of the screen, even though those input fields are not attached to anything. It also means if you don't specify COLUMN=1 when they hit the tab key they may go to one of the inoperative input fields.

Most users will understand they should only type in the window and not type in old input fields not in the active window. Similarly, If you specify COLUMN=1 then they won't have to tab through inactive input fields.

If however, this still worries your users you can specify the LOCK option. This tells Neatwind to disable all the old input fields on the screen, so there is no chance of the user becoming confused and filling in the wrong one. The disadvantage of using this option is it takes a couple of seconds for Neatwind to read what is already on the screen and modify it. It is slightly faster not to use the LOCK option, and rely on the user's common sense.

3.4 The Programmable Function Keys.

You may optionally use spare function keys for your own application, providing Neatwind doesn't need them.

3.4.1 Reserved Function Keys.

Neatwind uses some standard function keys.

1. Function Key 3.

   This key is the END or Exit key, depending on what style you are using. In the `I/S` and `SAA87` styles the user presses this key to indicate "I have finished with this window and want to continue with the application." This gives a rc=0. In the `SAA90` standard it means "I want to cancel right out of the applications." This gives a rc=9999.

2. Function Key 7.

   This is the BWD or Bkwd key. When a window grows too large this key is automatically placed there to let the user scroll back through the window text. Neatwind handles the scrolling transparently to your application.

3. Function Key 8.

   This is the FWD or Fwd key. It is the converse of the BWD or Bkwd key.

4. Function Key 10.

   This is the Action Bar key. Its is only used when you specify the `ACTION` option. This is discussed later on.

5. Function Key 12.

   This is the CANCEL or Cancel key. The user presses this when they want to "cancel out of this window, but no further." By default, this gives a return code rc=-10. If you specify the `HIRETURN` option, then this will give a rc=10000 (ten thousand) instead. REXX has the nasty habit of printing traces whenever it encounters a negative return code, so the `HIRETURN` option is your best choice. We've left the rc=-10 as the default for backward compatibility with Neatwind applications written before we discovered this. If you are using this old return code then be sure to put a `Trace Off` at the start of your REXX programs to silence the trace, and all will be ok.

All the other function keys are available for you to use yourself. Additionally, you may use PF keys 7, 8 and 10 if you know that Neatwind won't be using these for your window.

Neatwind maps PF keys 13-24 onto PF keys 1-12. You cannot define a separate function for eg. PF key 13 as opposed to PF key 1. This is to cater for people who have terminals only with 12 PF keys, or people with the 3279 keypad who seem to commonly use the PF13-24 keypad instead of the awkward to get at PF1-12 keys.

3.4.2 Customised Function Keys.

You may customise your own function keys by using the `PF` option. For example, the option `PF1=HELP` would define PF1 as a customised function key. In the window it would display `1-HELP` or `F1=Help` on the PF key status line. If the user presses this key then Neatwind would terminate with a return code rc=1 (That is, the same as its function key number).

By default, when a user hits a PF key Neatwind won't let them leave unless all input fields on the screen are filled out correctly.

Obviously, you wouldn't always want this. For example, the user should be able to press HELP at any time. If they knew how to fill in the input fields correctly, they probably wouldn't need it!

To avoid this checking specify the NOCHECK option on the end of the PF key definition. Thus PF1=HELP=NOCHECK.

You may choose any name you like for your PF keys. It may be in mixed case, but cannot include spaces.

Customised PF keys may not be defined on MENU or QUERY windows, as these already use the return code to pass back their selections. Customised PF keys may be used on all other types of windows. Here is an example:

```
f.0 = 3
f.1 = ' '
f.2 = 'Press any of the PF keys below to'
f.3 = 'test them.    Press PF3 to leave.'
mymsg = ''   /* Use this as a message */
do until mymsg = ''
 'EXEC NEATWIND (MESSAGE SHOWSTEM F. MSG ('mymsg')' ,
                 'PF1=AAA PF2=BBB PF4=CCC PF5=DDD PF6=EEE'
  if rc = 0  /* If PF3=Exit then leave - it always gives rc = 0 */
  then leave
  else mymsg = 'PF key' rc 'was pressed.'
end
```

Note the MSG () option. This is used to display our own one-line message on the first line of the window. We set a variable to the message we want to display and call Neatwind again to display it. We quit when PF3 is pressed, which gives a rc=0 by default. If we defined our own PF3 key using a PF3=XXX option then it would give rc=3.

3.4.2.1 The effect of styles on PF keys.

Neatwind knows some PF keys share the same function, even though they have different names in different styles. Neatwind is very forgiving in the use of these.

For example, the END PF key (in the I/S style), and the Exit PF key (in the SAA styles are equivalent). Thus, if for example you define the PF3=END option on a window which has the SAA90 option (for example), Neatwind will automatically convert that into PF3=Exit when it is displayed, as that is the SAA equivalent.

Other recognised synonyms are FWD and Fwd, and BWD and Bkwd.

This makes it easier for you to convert your application between styles without having to waste time changing something so trivial and obvious.

4.0 Vertical Scrolling.

If you give Neatwind too many lines to fit on the display at once it will automatically make the window scrollable. The PF 7 and 8 keys will appear, and the user shall be able to scroll through the window text using these.

Your application does not have to do any of the scrolling itself. Neatwind handles it all by itself, transparent to your calling program. Your program does not have to even know that the window is even being scrolled.

4.1 Tailoring your Scroll Options.

Neatwind decides on the size of the displayed part of the scrollable window by using the `SCROLLSIZE` option. This defaults to 10. You may set it smaller or larger. Eg. `SCROLLSIZE=15`. If you want Neatwind to use as much of the screen as possible, but remain in a window specify `SCROLLSIZE=MAX`. Alternately, you may decide just to make it a `FULLSCREEN` panel instead.

You may reserve lines to be permanently shown at the top and bottom of your window text using the `KEEPTOP` and `KEEPBOTT` options. These default to `KEEPTOP=0 KEEPBOTT=0`. The lines these options reserve are deducted from `SCROLLSIZE`.

By default, windows are only made scrollable if Neatwind realises they are too big. If you want the window to be made scrollable anyway then specify the `FORESCROLL` option.

4.2 The MORE Icon.

According to the CUA standard, whenever scrollable data is displayed a “More” indicator should be displayed above that data. It should display an up and/or down arrow to show in which directions the user can scroll through the data. If these arrows are not available then a “+” and “-” should be displayed instead.

If the window is horizontally scrollable (as discussed in “Horizontal Scrolling.” on page 56) then a left and right arrow (or “<” and “>”) will also appear.

Optionally, underneath the scrollable area can be another indicator which says “More...” if there is more data underneath, and “Bottom” if the user is at the bottom.

Neatwind implements these using a special &MORE. icon. If you place a &MORE. icon in a `KEEPTOP` area then it take on the first form. (eg. “More: +-”) If you place it in the `KEEPBOTT` area it takes on the second (ie. “More...” or “Bottom”) You cannot embed the &MORE. icon in the scrollable data area.

Here is an example:

```
'MAKEBUF'
 Queue 'Please scroll through this list and pick'
 Queue 'the branch of your choice.'
 Queue ' '
 Queue '  Branch  Manager'
 Queue '  ------  ------------------------------ &MORE.'

 do i = 1 to 40
   Queue  '  'format(i, 6)  || ,
          '  'word('Jones Doug Smith Nancy Mulray', random(1, 5))
 end i

 Queue '                                          &MORE.'
'EXEC NEATWIND (MESSAGE KEEPTOP=5 KEEPBOTT=1 SCROLLSIZE=15' ,
               'SAA87 PSLOAD=NEATBOLD'
'DROPBUF'
```

This is what the window would look like:

```
   Please scroll through this list and pick
   the branch of your choice.

   Branch  Manager
   ------  ------------------------------ More: +
        1  Jones
        2  Smith
        3  Jones
        4  Nancy
        5  Smith
        6  Nancy
        7  Doug
        8  Mulray
                                          More...

  F3=Exit F7=Bkwd F8=Fwd Enter
```

The back and forward arrows will only appear if you have a PSS terminal, *and* `PSLOAD=NEATBOLD`, either when you do the window or at some point on an earlier window in the application. This is a good case for doing the `PSLOAD=NEATBOLD` on the first window or panel in your application displayed using NEATWIND.

5.0 Data Entry Screens.

Data Entry screens are perhaps the most powerful feature within Neatwind. These allow you to throw together highly flexible screens very quickly. When Neatwind displays this screen it will also enforce a wide range of checks on the data.

To do a data entry screen you simply specify the `DATAENTRY` option when you call Neatwind. You pass the window text to Neatwind in the same way you pass other windows, except you group together at-sign characters "@" to denote input fields.[3]

For example:

```
'MAKEBUF'
 Queue 'Please enter the following employee'
 Queue 'information:'
 Queue ' '
 Queue '  Employee Number:@@@@@@'
 Queue '  Employee Name:  @@@@@@@@@@ @@@@@@@@@@@@'
 Queue '  Sex (M or F):   @'
 Queue '  Job Type:       @@@@@ (Mgr, Clerk or Sec)'
 Queue '  Age (years):    @@@ (optional)'
'EXEC NEATWIND 7 (DATAENTRY SAA87'                ,
     'OUTVARS (EMPNR EMPNAME SEX JOBTITLE AGE)'   ,
     'TYPE.1=WHOLE LENGTH.1=6'      ,
     'LENGTH.2,3=MANDATORY'         ,
     'TYPE.4=(CHAR MF)'             ,
     'TYPE.5=(LIST MGR CLERK SEC)' ,
     'TYPE.6=WHOLE LEN.6=OPTIONAL'
 $rc = rc
'DROPBUF'
 select
   when $rc = 0 then Say 'Employee #'empnr 'is now on file.'
   when $rc = -10 then Say 'User cancelled out of window.'
 end
```

Look at the first two lines of options. The first says that this is a `DATAENTRY` screen, which is to be displayed in the `SAA87` style. The second line denotes the REXX variables that the values entered in the input fields are to be returned in. In this example, the first field goes in EMPNR, the second in EMPNAME, and so on.

5.1 Field Validation.

In the above example we enforce field validation. We don't have to do this, but since we'd do the checking in our application if we didn't get Neatwind to do it we might as well.

There are two options for enforcing field validation checks. Let's look at the ones in our above example.

---

[3] If you are using a non-English terminal this character may be different. See "Field Specification Characters." on page 73.

The line `TYPE.1=WHOLE LENGTH.1=6` says that the first input field must be a whole number, and that it must be exactly six characters long.

The line `LENGTH.2,3=MANDATORY` says that these fields must be filled in. (The default is for the field to be optional, unless it has a TYPE definition). The comma says this condition applies to fields 2 and 3. You may include any number of fields this way for both the LENGTH and TYPE option. Because we have not specified a TYPE option for fields 2 and 3 the user is allowed to type in anything, so long as it is not blank![4]

The line `TYPE.4=(CHAR MF)` says that the fourth field must have either an M or F in it. Upper or lower case are considered equivalent when enforcing these checks.

The line `TYPE.5=(LIST MGR CLERK SEC)` says that one of the words in the list must by typed into the fifth field. Again, upper and lower case, or a mixture thereof are accepted.

The line `TYPE.6=WHOLE LEN.6=OPTIONAL` say that the sixth field (age) is a whole number. Whenever a TYPE field is specified then that field reverts to mandatory. Because we want this field to be optional we override it by saying `LEN.6=OPTIONAL`.

5.2 Field specification

As you have seen, you can specify commas to indicate a number of fields have the same attributes. For example, `TYPE.1,5,6,8=WHOLE`.

If you have many fields with the same attributes, you may simply omit the field number and set them all in one go. If there are any exceptions you may set them after you do the global assignment. For example, `TYPE.=WHOLE TYPE.1=(LIST MGR CLERK SEC) TYPE.2=(CHAR MF)` In this case all fields expect the 1st and 2nd will be whole numbers.

If you want to specify a range of values you may specify a starting field, an increment and a finishing field (up to or equal to). You may use as many multiple ranges as you like when you define your window.

For example:

```
'MAKEBUF'
 Queue "Teacher's Name: @@@@@@@@@@@@@@@@@@"
 Queue "Teacher's Age:  @@@"
 Queue " "
 Queue "Student Age     Sex"
 do i = 1 to 10
   Queue "        @@@     @"
 end i
 Queue "Year class started: @@@@"
'EXEC NEATWIND (DATAENTRY' ,
     'LEN.1=MAND TYPE.2=WHOLE' ,
     'TYPE.3+2<22=WHOLE TYPE.4+2<22=(CHAR MF)' ,
     'TYPE.23=WHOLE LEN.23=MAND'
 $rc = rc
'DROPBUF'
```

The option `TYPE.3+2<22=WHOLE` says start at the 3rd field, go up in steps of 2 up to and including the 22nd field, and set each of these fields to be enforced as `WHOLE` numbers. You can include other columns or ranges by using the comma in this option. These rules apply equally to the `LENGTH` option (which may be abbreviated as `LEN`).

---

[4] `MANDATORY` translates to "length must be 1 or more characters."

5.2.1 Checking other Types.

We have seen the LIST, CHAR and WHOLE types. There is also a NUMERIC type which enforces the field to be a number, but not necessarily whole (ie. it can have a fractional part after the decimal point).

The `ALPHANUMERIC` (which may be abbreviated as `ALPHA`) type accepts letters (both upper and lower case) and digits. It does not accept spaces which means that only a single word may be entered in these fields. The `STRING` type on the other hand does accept spaces, thus allowing more than one word to be entered in these fields.

Here is an example:

```
 s.0 =  4
 s.1 = 'Family name:  @@@@@@@@@@@@@@@'
 s.2 = 'First name:   @@@@@@@@@@@@@@@'
 s.3 = 'Phone number: @@@-@@@@'
 s.4 = 'Job title:    @@@@@@@@@@@@@@@'
'EXEC NEATWIND (DATAENTRY SHOWSTEM S.' ,
               'TYPE.1,2=ALPHA TYPE.3,4=NUM TYPE.5=STRING'
```

Note that the `ALPHANUMERIC` and `STRING` types only accept the English alphabet. If you are working in a foreign language then we recommend you use the `CHAR` option to do the checking. By including the English alphabet and your own local characters you can do an equivalent check.

There is also a subrange option. eg. `TYPE.1=0..200` This says that field can be any number between 0 and 200. You may include negative numbers in this range. The numbers may be fractional (ie. real) if you specify a decimal point in one of the limits. eg. `TYPE.1=0..200.0` In both of these cases you may omit one limit of the subrange. For example, `TYPE.1=0.0..` says the field must be zero or greater. `TYPE.1=..200` says the field must be an integer up to or equal to 200.

There is an equivalent of this subrange supported for the `CHAR` type. For example, `TYPE.1=(CHAR <A-Z><0-9>)` makes sure that all the characters in the nominated field are either letters or digits. (Because Neatwind ignores case you don't have specify the lower case letters separately).

You can also include other characters in this notation. For example, we can include the underscore, dot and blank in the definition. By including the blank we allow multiple words in the field. Thus: `TYPE.1=(CHAR <A-Z><0-9>._ )`

Note the space between the underscore and the closing bracket. That adds space to the list, even though it isn't all that obvious. To make it more obvious we can use the `<BLANK>` token designed especially for this situation. Thus: `TYPE.1=(CHAR <A-Z><0-9>._<BLANK>)`

5.2.2 DATE and TIME checking.

Neatwind includes a very powerful date and time checking facility. You can check against all Gregorian forms of time or date, or even define your own. You can also specify a constraint, such as the entered date/time being before or after now, or some other date/time.

Here is a list of the standard date/time formats.

ISO	yyyy-mm-dd (default for DATE)
JIS	yyyy-mm-dd
JULIAN	yyddd
USA	mm/dd/yyyy
EUROPEAN	dd/mm/yyyy
IBMEUROPEAN	dd.mm.yyyy
TIME	hh:mm:ss (default for TIME)
24HOUR	hh:mm:ss
12HOUR	hh:mm:ss xm

To define the first field as a date we say `TYPE.1=(DATE)`. To define that field as a European format date we would say `TYPE.1=(DATE EUR)` or `TYPE.1=(DATE EUROPEAN)`. All of the above formats may be abbreviated down to two letters.

To define a field as a time we would simply say `TYPE.2=(TIME)`.

5.2.2.1 Your own format.

You can enforce your own date/time format by passing it as an option instead of one of the above standard formats. Say we want a two-digit month followed by a slash and two-digit year. We would simply say `TYPE.2=(DATE MM/YY)`.

You can build your own format from the following tokens.

YYYY Year between 1900 and 2100.

YY Two-digit year.

MM Two-digit month of year.

DD Two-digit day of month.

HH Two-digit hour of day (12 or 24 hour format).

XM Meridian (AM or PM)

Morning or Afternoon respectively. This notation is only used in the English-speaking countries. All others use 24 hour mode. If XM is specified HH reverts to 12 hour mode, otherwise it remains in 24 hour mode. Meridian appears as "xM" in usage messages. eg. "HH:MM:SS xM format expected."

NN Two-digit minute of hour.

When you define the field note we use NN instead of MM to avoid ambiguity with the month. This will be automatically converted into MM when displayed in usage messages for the user, as they can safely resolve the ambiguity. eg. "HH:MM:SS format expected."

SS Two-digit second of minute.

DDD Three-digit day of year. This is used for Julian date format (YYDDD).

Any characters embedded are treated literally. For example, if you specify a slash in your format the user must type a slash at that position in their input. A substitute character will not be accepted. If you want to include a space in your format specify it with an underscore. For example, `TYPE.2=(TIME HH:NN:SS_XM)`. This will be translated into a space.

The `TIME` and `DATE` options are fully interchangeable. The only difference is the default format. Other than that they are identical.

5.2.2.2 Constraints.

You may specify a constraint that the date/time the user types in is before or after some other date/time. That second date/time must be in the same format as the user date/time.

For example, `TYPE.2=(DATE USA BEFORE 10/31/91)` says the date the user types in must be in USA format, and before (but not on) the 31st of October, 1991. `TYPE.2=(TIME ONAFTER 12:00:00)` says that a 24 hour time entered must be on or after 12 midday.

Here are the four constraint keywords:

BEFORE	Before, but not on some time/date.
ONBEFORE	Before or on some time/date.
AFTER	After, but not on some time/date.
ONAFTER	After or on some time/date.

As described above, follow this keyword by the time/date in the same format as the input field. You may specify `NOW` or `TODAY` instead of that time/date to indicate right now. For example, `TYPE.2=(DATE EUR AFTER TODAY)`

You may also use the `PAST` or `FUTURE` keywords instead which translate to on or before today, and on or after today respectively. For example, `TYPE.3=(DATE PAST)` or `TYPE.3=(DATE EUR FUTURE)`.

5.2.2.3 Restrictions.

The date/time the user types in must be spaced identically to the one you have defined. For example, 1/1/90 would not be considered a valid date. The user would have to enter 01/01/90 instead. If we didn't enforce this it would be very difficult to process date formats such as `YYYYMMDD`.

Anything Neatwind doesn't recognise as a `DATE` or `TIME` option is assumed to be a user-defined date/time format. Be warned however, if you misspell an option that option will be interpreted as a user-defined date/time! If a user gets an error message that says something like "<misspelled option> format expected," then this is what has happened.

Occasionally you will run into a field that cannot be checked using one of our predefined type options. For example, there is no standard checking format for part numbers, machine feature fields, car registration numbers of serial or scanner codes with check digits. There are two ways we can check these non-standard types.

5.2.3 The EXEC Option.

We can write a function as an external REXX EXEC that returns 1 if its argument is valid, and 0 if it is not. You can ask Neatwind to use this external function to validate input fields by specifying the `EXEC` option.

For example:

```
'MAKEBUF'
 Queue 'Enter the order number: @@@@@@'
 Queue '          car numplate: @@@@@@@@ (registration number)'
'EXEC NEATWIND (DATAENTRY TYPE.1=WHOLE TYPE.2=(EXEC CHKREGO)'
 $rc = rc
'DROPBUF'
```

Here is a simple example of the CHKREGO EXEC which checks that it is passed a valid vehicle registration number:

```
/* CHKREGO EXEC
 * Returns 1 iff passed a valid rego number in the format
 * of three letters, a dash, and three numbers.
 * Eg. OFZ-739.  Personalised number plates are not
 * accepted in this example.
 */
Parse Var aaa +1 dash +1 nnn +3 .

Return datatype(aaa, 'UPPER') & dash = '-' & ,
       datatype(nnn, 'WHOLE') & nnn >= 0
```

By using the `EXEC` option you may easily apply checks to non-standard data types. You could even, for example, have a function to check a customer number that does an SQL query on a customer database to check its validity. So long as the EXEC returns 1 if the argument is valid, and 0 if it is not you will have no problem.

5.2.4 Checking in the Application.

Some checking operates on more than one field at once. For example, in order to check that a list has no duplicates you must look at every field in the list. This sort of checking can't be done using the TYPE or LENGTH options. Instead we do the checking at the application level.

For example:

```
quit   = 0   /* true if they PF=END out */
cancel = 0   /* true if they cancel out */
mymsg = ''   /* our error message */
do until quit | cancel
 'MAKEBUF'
  Queue 'Enter Registration Plates of'
  Queue 'cars in the company carpool.'
  Queue '----------------------------'
  do i = 1 to 20
    Queue '@@@@@@@'
  end i
 'EXEC NEATWIND (DATAENTRY KEEPTOP=3 MSG('mymsg')' ,
                'OUTVARS (CARLIST.) NOBLANK' ,
                 TYPE.=(EXEC CHKREGO) LEN.=OPTIONAL'
  $rc = rc
 'DROPBUF'
  if $rc = 0
  then do
    do i = 1 to 20
      do j = 1 to 20
        if carlist.i ¬= ''         & ,
           carlist.j ¬= ''         & ,
           carlist.i ¬= carlist.j
        then do
          mymsg = 'Error: Lines' i 'and' j 'are duplicates.'
          leave i
        end
      end j
    end i
    if mymsg = ''  /* if no error message */
    then quit = 1  /* true */
  end
  else cancel = 1  /* true */
end
```

Note how we use the TYPE.=(EXEC CHKREGO) option to validate the number plates as they are entered? Once all the number plates are validated by Neatwind then we check for duplicates within the application. The KEEPTOP=3 option is used to keep the top three lines with the prompt message displayed on the screen at all times, even when the user scrolls through the list.

The OUTVARS (CARLIST.) option says to return the fields in the REXX variables CARLIST.1, CARLIST.2, CARLIST.3 and so on. The total number of fields used is held in CARLIST.0. If CARLIST.0 = 20, then the fields go all the way up to CARLIST.20.

If you want to highlight specific fields as errors when you return control to Neatwind (say those lines that are duplicated) use the ERRFIELDS option. For example, ERRFIELDS (6 8) would highlight the 6th and 8th field as errors. The cursor is always positioned on the first field given (the 6th in this case). When the user edits these fields, or leaves the error highlighting will be removed.

Note the NOBLANK option. If we didn't specify NOBLANK then Neatwind would blank the input fields when we call it a second time to report for example, a duplicate error. The default is BLANK. This is discussed in more detail later in this chapter.

5.2.5 When to check?

You can tell Neatwind when to check the input fields by specifying the CHECKTYPE option.

- CHECKTYPE=FINAL

Check that all the input fields are valid only when the user tries to leave the window.

- `CHECKTYPE=IMMEDIATE`

  Check immediately all the fields in the scrollable area of the window immediately. Values in the `KEEPTOP` and `KEEPBOTT` zones are ignored. This allows the user to scroll through a list of input fields before filling in one in the KEEPTOP zone, for example.

- `CHECKTYPE=ALL`

  Check immediately *all* the entire displayed area (scrollarea, `KEEPTOP` and `KEEPBOTT` zones) immediately.

  Neatwind is reasonably efficient in the way it goes about its checking. It won't recheck a field (valid or invalid) until the user actually changes it. The fact that Neatwind does this is transparent to the user, apart from the improved response time.

5.3 Initialising Values.

By default, Neatwind will use the `BLANK` option, which clears all input fields when the window is first called.

If you want Neatwind to display the values as they currently are, specify the `NOBLANK` option. You should use this when you want to preload default values into the input field (eg. a saved list of car registration numbers), or you are calling Neatwind more than once (ie. looping back).

Neatwind will get the initial values from the same variables that you have specified in the `OUTVARS` option. For example:

```
 empnr   = 10
 empname = 'Dale Cooper'
 f.0 = 2
 f.1 = 'Employee Nr:   @@'
 f.2 = 'Employee Name: @@@@@@@@@@@@@@@@@@@@'
'EXEC NEATWIND (DATAENTRY SHOWSTEM F. NOBLANK OUTVARS (EMPNR EMPNAME)'
```

5.4 Getting the results back.

5.4.1 The OUTVARS option.

We've already seen how the `OUTVARS` option which places the results in REXX variables in the program calling Neatwind. This can place field values in specific variables, or stems, or a mixture of both.

For example, `OUTVARS (TEACHNAME TEACNNR SEX. AGE. YEAR)` Would place the first two fields in `TEACHNAME` and `TEACHNR` respectively. The very last field would be placed in the variable `YEAR.` All those fields in between would be placed in the `SEX` and `AGE` stem in that order. (ie. SEX.1, AGE.1, SEX.2, AGE.2, SEX.3, AGE.3 and so on).

5.4.2 Stems

The zero element of each Stem contains the total number of elements in that stem. For example, if `SEX.0 = 20` then there are 20 such subscripted fields. Same applies to the `AGE.0` variable.

For `OUTVARS` these results are passed back if the user leaves the window using PF=END, PF=Exit, Enter, or any other PF key which you defined yourself without the NOCHECK option. The only exception to this rule is the PF=Cancel or PF=CANCEL key. By default it will not

pass back the results, though you can ask it to by specifying the NOCHECK option. (It will still not check the values of the field when you cancel in either case).

5.4.3 The STACK Option.

Alternately you may use the program stack by specifying the `STACK` option. Each field is stacked one per line. Retrieve these values using the REXX Pull instruction. The first value pulled will be the first field. The second value will be the second field, and so on.

5.4.4 The GLOBALV Option.

Finally, you may choose to use CMS Global Variables by specifying the `GLOBALV` option. You can extract the field values from the NEATWIND Global Variable group using the GLOBALV command. For example, `GLOBALV SELECT NEATWIND GET VAL.1 VAL.2` would put the value for the first field in the variable `VAL.1`, and the second in `VAL.2`.

5.4.5 The NOBLANK Option.

If you specify `NOBLANK` then Neatwind will take the initial field values from the same place that you specify should contain the edited values.

If you are programming in REXX or EXEC2 then the `OUTVARS` option is recommended. If you are programming in some other language then `STACK` or `GLOBALV` may suit you better.

If you use the `GLOBALV` or `STACK` options then Neatwind will always pass the values of the field back when you hit a PF key, even if it is PF=Cancel. This is just in case you wanted to know the values when they did.

5.4.6 The UPPERCASE Option.

Sometimes you will want to convert the values the user enters into uppercase. Filenames are a good example of this.

Rather than asking the user to enter the values in uppercase (which can be very cubersome for them), you may specify the `UPPERCASE` option. This will automatically convert all lower case letters in all fields into their upper case equivalents. This conversion takes effect when the values are passed back to your program.

5.4.7 The STRIP Option.

By default, fields are padded with blank spaces to bring them up to their required length. For example, a ten character field will always return a value ten characters long. If the user enters less than ten characters the difference will be padded by spaces.

You can ask Neatwind to supress this padding (both leading and trailing spaces) by specifying the `STRIP` option. This takes effect when the values are passed back to your program.

5.5 READONLY Fields.

Sometimes you want to display a field, but not let the user change it. For example, anyone may be able to view their employee payroll record and change their address, but only the payroll department can change your other details (such as salary). Your manager can view your payroll record, but not change any of its details directly.

You could provide two completely different programs to handle this, but this seems pointless writing two sets of nearly identical code.

Neatwind provides an easy way to do this by defining certain data entry fields to be *read only*. There are two ways to do this:

5.5.1 The Backslash Method.

You can simply define the fields as read only in the window text. To do this, define them using the backslash "\"[5] instead of the normal at-sign "@." Here is an example:

```
'MAKEBUF'
 Queue "Customer Name:   \\\\\\\\\\\\\\\\\\\\"
 Queue "Customer Number: \\\\\\"
 Queue " "
 Queue "Qty  Part Number"
 Queue "@@@  @@@@@@@@"
 Queue "@@@  @@@@@@@@"
 Queue "@@@  @@@@@@@@"
 Queue "@@@  @@@@@@@@"
'EXEC NEATWIND (DATAENTRY READONLY'
 $rc = rc
'DROPBUF'
```

The backward slashes say the first and second field are read-only. You must specify the `READONLY` option. If you don't the backslashes will be left alone, and not considered to be anything special.

`READONLY` automatically selects the `NOBLANK` option.

5.5.2 The PERMISSION Option.

An alternative to the backslash method is to use the `PERMISSION` option. This is useful if you have panels with very mixed readonly and read/write fields, or you have so many different types of access it is impractical to define a different backslash panel for each one.

The `PERMISSION` option is very similar to the `TYPE` and `LENGTH` options. There are two types of permissions, `READ` and `WRITE` (the latter meaning read *and* write). You assign these permissions to a list of columns. Here is an example for the payroll situation discussed earlier:

```
 /* What can they look at? */
 select
   when find(payroll_dept, Userid()) > 0 then
     perms = 'PERM.=WRITE'  /* payroll can change anything */
   when find(managers.this_userid, Userid()) > 0 then
     perms = 'PERM.=READ'   /* manager can only view */
   when this_userid = Userid()  then
     perms = 'PERM.=READ PERM.2,3+1<5=WRITE'
   otherwise do
     Say 'Hey!! Why are you looking at' this_userid'?'
     Exit 999
   end
 end

'MAKEBUF'
 Queue 'Employee Signon: \\\\\\\\'
 Queue 'Employee Name:   @@@@@@@@@@@@@@@@@@@@'
 Queue 'Address:         @@@@@@@@@@@@@@@@@@@@@@@@@@@@'
 Queue '                 @@@@@@@@@@@@@@@@@@@@@@@@@@@@'
 Queue '                 @@@@@@@@@@@@@@@@@@@@@@@@@@@@'
 Queue 'Salary ($):      @@@@@@@@@@@@@@@'
```

---

[5] If you are using a non-English terminal this character may be different See "Field Specification Characters." on page 73.

```
 'EXEC NEATWIND (DATAENTRY' perms
  $rc = rc
 'DROPBUF'
```

The PERM option (which can be also written as PERMISSION) specifies what fields are read only, and which are read/write. Since the manager and payroll have global permissions we need only specify this in one place. Since the employee has splotchy permissions we define a global READ, then go back and give WRITE permission on the exceptions. Note we use field specification ranges, as discussed in “Field specification” on page 20.

Note how we defined the first field as backslashes? This makes it read-only regardless of the PERMISSION option's settings. We do this as an example; for instance, if the userid was a key and should only be changed using the action bar's File pull-down (eg. save as, open).

5.5.3 Security.

Needless to say, don't give everyone write permission to your database and files, and rely on Neatwind to stop them from changing these values. Always implement your security system through SQL views, or server machines if you are changing files.

The read-only fields in Neatwind only let the user know what they should change and what they shouldn't. You should always have the correct security on the actual data itself.

5.5.4 The PROTECT option.

Read-only fields are coloured differently from normal input fields, to show that they are different. By default, these fields will be the same colour as the FOREGROUND.

If you don't like this scheme you may change the read-only fields colours by setting the PROTECT attribute to your desired colour. For example, PROTECT=BLUE would make all protected (read-only) fields appear dark blue.

5.6 The INVISIBLE option.

You may make certain input fields invisible. This is useful for things such as passwords, where you don't want what the user types to be visible to anyone who may be looking over their shoulder.

Simply list the numbers of the fields you want listed with the INVISIBLE option. For example, INVISIBLE (1 10 12) would make the 1st, 10th and 12th field invisible.

Alternatively, if you specify INVISIBLE by itself, you may define the invisible input fields using the not-sign “¬,”[6] instead of the normal at-sign “@.” Thus:

```
'MAKEBUF'
 Queue 'Userid:  @@@@@@@@'
 Queue 'Password:¬¬¬¬¬¬¬¬'
'EXEC NEATWIND (DATAENTRY INVISIBLE'
'DROPBUF'
```

[6] If you are using a non-English terminal this character may be different See “Field Specification Characters.” on page 73.

5.7 Did they change anything?

Sometimes you want to know if a user changed anything on a window or panel. For example, it is common not to resave an object if the user only looked at it and didn't actually change anything.

Neatwind sets the REXX variable `NEAT$CHANGED` to `1` if and only if the user changed any of the field values on the screen. This will be `0` if the user did not change anything. This applies to `DATAENTRY` and `PICK windows`. If Neatwind abends with an error then this variable will not be set or changed. Thus *in this case* when you do your checking `if neat$changed = 1` or `if neat$changed = $.true` would be safer than saying just `if neat$changed`.

Since you may have to loop back through the window you should check the `neat$changed` value each time you return. Here is an example of such code:

```
f.0 = 2
f.1 = "Name: @@@@@@@@@@@@@@"
f.2 = "Age:  @@@"

save_object = $.false          /* $.true = 1, $.false = 0 */
do forever
  'EXEC NEATWIND (SHOWSTEM F. DATAENTRY PF1=Help'

   /* Was the screen changed this iteration? */
   if neat$changed = $.true   /* check against 1 in case not set */
   then save_object = $.true  /* yes, so must save object        */

   select
     when rc = 0 then do       /* user exist peacefully      */
       if save_object          /* if we need to save object */
       then call save_routine  /* then call save routine    */
       leave                   /* and exit the loop         */
     end

     when rc = -10 then do     /* User Cancels out          */
       save_object = $.false   /* Don't bother saving it    */
       leave
     end

     when rc = 1 then          /* User press PF=Help key    */
       call help_routine

     otherwise do
       Say 'Neatwind Error. rc='rc
       Exit -999
     end
   end
 end
```

5.8 Getting the spacing right.

Neatwind tries to squeeze every available space out of the screen, but has some difficulty coping with the 3270 architecture. Consequently, the spacing you have in your window when you define an input field won't be exactly what appears on the displayed window.

If this doesn't worry you (or you take account of this when you define your windows), that is fine. If not, the following list of rules will give you an idea of the extraneous space Neatwind will place around input fields. Knowing this, you can omit spaces before and after the fields to get everything to line up nicely.

5.8.1 Normal spacing.

These rules apply when you call Neatwind without the `COMPRESS` option. When you design a window this way, the spacing of the window as defined in the program will closely resemble that displayed on the screen. There are however a few exceptions:

- Neatwind inserts two spaces before each input field.
- Neatwind inserts two spaces after each input field, unless you follow it immediately by a two character escape sequence (eg. "%H")
- If input fields are adjacent in your definition (separated by one or two blanks), then only two spaces will separate them on the displayed window. This overrides both the above rules (though you may still follow the second field by an escape sequence).

So, in order to get your columns to line up in a tabular `DATAENTRY` window delete two spaces before each starting input field, and leave exactly two spaces between adjacent input fields.

If you don't do this your window will still work fine in all other ways. There will just be a slight misalignment eg. between the column headings and the data fields. That is all.

Colour changes take two characters in the window definition, one for the escape character "%", and another for the following control character eg. "H." These take two spaces on the displayed window.

Note that this mode is the default.

5.8.2 With the COMPRESS option.

When you display a window using the `COMPRESS` option, the following rules apply. The two character escape sequence (eg. "%H") now only takes one character on the displayed window. This means you can change colour, for example, in only one space. Similarly, only one space is needed to separate input fields. There is a disadvantage to using this option. Namely, the window define in your program will not resemble the one displayed on the screen all that closely. This is because the two-character escape sequence collapsing to one tends to distort the spacing.

- Neatwind inserts one space before each input field.
- Neatwind inserts one space after each input field, unless you follow it immediately by a two character escape sequence (eg. "%H")
- If input fields are adjacent in your definition (separated by one or two blanks), then only one space will separate them on the displayed window. This overrides both the above rules (though you may still follow the second field by an escape sequence).

Note that the `COMPRESS` option is not a default. You must enable it explicitly.

5.9 More colours

You can change the colour of the input fields using the `INPUT` option, or the colour of erroneous input fields (those holding an invalid value) with the `ERRINPUT` option. For example:

```
 grades. = 'E'
 f.0 = 5
 f.1 = " Laura Palmer's School Report"
 f.2 = " "
 f.3 = " English  @"
 f.4 = " Math     @"
 f.5 = " Science  @"
'EXEC NEATWIND (TYPE.=(CHAR <A-F>) INPUT=TURQ ERRINPUT=BLUE' ,
               'OUTVARS (GRADES.)'
```

The colour you specify for `ERRINPUT` is also the colour of the window's message line.

You can also change the colour of the icons or action bar (discussed later) using the `ICONCOLOUR` and `ACTCOLOUR` options.[7]

[7] Neatwind also accepts the American spelling "color." For example, `ACTCOLOR` is a valid alternative to `ACTCOLOUR`.

6.0 Context Sensitive Help.

Neatwind is capable of providing your users with a context sensitive help facility. This allows the user to point at a field of interest using the cursor, and on pressing the HELP key have help text specific to that field displayed. If the cursor is not near any input field at the time the HELP key is pressed then a general help is displayed.

Context Sensitive Help only operate with `DATAENTRY` and `PICK` screens.

6.1 Defining a HELP key.

To define a context sensitive help key you must specify a HELP modifier when you define your help key. For example, `PF1=Help=HELP`. The first `Help` is the name of the PF key. We have called it "Help" because this conforms to SAA. If you are not writing an SAA application you may give it another name instead, but help is usually the most obvious. For example, `PF9=AssistMe=HELP` is equally valid, but less obvious to the user.

The second `HELP` after the second equal sign is the modifier keyword (the same sort of thing as the `NOCHECK` option discussed earlier).

When the user presses this PF key Neatwind returns control to the calling application, with the appropriate PF key return code. (If you are following SAA, PF1 is always the help key, so the return code will be rc=1).

The difference between defining `PF1=Help=HELP` instead of just `PF1=Help` is that extra information is passed back from Neatwind.

6.2 Help for what?

If you have specified `OUTVARS`, then the REXX variable `HELP$NUMBER` contains the number of the field they asked for help on. This is blank if they weren't near an input field when they pressed the HELP key. `HELP$MAXFIELDS` contains the total number of fields in that window. If you specified the `OUTVARS` option then `HELP$NAME` will also contain the name of the variable you were asking for information on.

If you specified `GLOBALV` or `STACK` then the results will be returned through that medium.

6.3 The Question Mark.

Quite a few pre-SAA applications let the user get help on a field by entering a "?" in that field. When you have defined a `HELP` key as above, this will also be accepted by Neatwind. Both have the same effect, though for the question mark method Neatwind will clear the field where the "?" was entered.

6.4 Contextual Help Example.

Here is a simple example of a field using contextual help. We shall just say “help text for xyz” here, but in a real application you'd do something more useful, such as a special HELP routine which extracts the help text from a file and displays it in a window.

```
do forever
 'MAKEBUF'
  Queue "Teacher's Name: @@@@@@@@@@@@@@@@@@"
  Queue "Teacher's Age:  @@@"
  Queue " "
  Queue "Student Age     Sex"
  do i = 1 to 10
    Queue "        @@@     @"
  end i
  Queue "Year class started: @@@@"
 'EXEC NEATWIND (DATAENTRY SAA87' ,
      'LEN.1=MAND TYPE.2=WHOLE LENGTH.3+1<22=OPTIONAL' ,
      'TYPE.3+2<22=WHOLE TYPE.4+2<22=(CHAR MF)' ,
      'TYPE.23=WHOLE LEN.23=MAND SCROLLSIZE=MAX' ,
      'OUTVARS (TNAME TAGE SAGE. SSEX. YEAR)'    ,
      'PF1=Help=HELP'       /* We've added a help key */
  $rc = rc
 'DROPBUF'
  select
    when $rc = 1 then do
      select
      when help$name = 'TNAME' then what = 'Teacher Name Help Text.'
      when help$name = 'TAGE'  then what = 'Teacher Age Help Text.'
      when help$name = 'SAGE.' then what = 'Student Age Help Text.'
      when help$name = 'SSEX.' then what = 'Student Sex Help Text.'
      when help$name = 'YEAR'  then what = 'Class Start Year Help Text.'
      otherwise what = 'General Help.'
      end
      helptext.0 = 1
      helptext.1 = what
     'EXEC NEATWIND (MESSAGE SAA87 SHOWSTEM HELPTEXT.'
    end
    when $rc = 0   then leave  /* Ok; End the window peacefully */
    when $rc = -10 then leave  /* cancel */
    otherwise do; Say 'NEATWIND Error. rc='$rc; leave; end
  end
end
```

When you do write your help routines do remember to give them the opportunity to access the General Help text from the field-specific help screen. You shouldn't force them to move the cursor to some off-window position just to get the general help!

6.5 Now, where was I?

When our example returns to the Neatwind data entry screen after displaying the help window it has lost its place. Not remembering where it was (the user may have scrolled forward several screens), Neatwind gives up and resets the window to the first line of the window text, and homes the cursor. Needless to say, this is pretty off-putting for the poor user who has to scroll through the data to get back to where he or she was. There is however, a nice and easy way around this.

When Neatwind leaves a window it sets a REXX variable `NEAT$POSN` in the calling program. This records where the user was in the window when they left it (eg. by pressing PF=Help). This variable is set automatically when you return from `DATAENTRY` or `PICK` windows. You can ask it be returned from other window types by specifying the `REVEALPOSN` option.

To reposition the cursor on your next call to the dataentry window, simply include that variables contents in the option line. When Neatwind sees this in the option line it will know to start the window on that line, and where to position the cursor.

Here is how we would incorporate this into the above example:

```
neat$posn = ''         /* Make sure it starts off blank */
do forever
 'MAKEBUF'
  Queue "Teacher's Name: @@@@@@@@@@@@@@@@@@"
  .
  .
  .
 'EXEC NEATWIND (DATAENTRY SAA87' neat$posn ,
      'LEN.1=MAND TYPE.2=WHOLE LENGTH.3+1<22=OPTIONAL' ,
  .
  .
```

We always start by setting it to a blank so it doesn't cause trouble on the first call, and there after include it in each call to the options line.

Be warned! If you call another Neatwind `DATAENTRY` or `PICK` window within the same loop it will also set the `NEAT$POSN` variable. If that is the case you should save the original value in another variable, and restore it when you have finished with that second window. If you code your other window in a separate procedure then REXX's scoping rules will fix the problem, and saving and restoring `NEAT$POSN` will not be necessary.

`ERRFIELDS` always takes precedence over any embedded `NEAT$POSN` in the options line.

6.5.1 Format of NEAT$POSN.

To use NEAT$POSN in the above way, there is no need for you to actually know what it holds. where the cursor was when the user left the window, then your might read this chapter.

NEAT$POSN consists of five slash delimited fields, in the form “rrccc/s/z/l/c/v/w.” These are as follows:

1. rrccc
   The screen row and column number the cursor was positioned on.

2. s
   An internal reference to where the scrollable area within the window started. Do not alter this reference.

3. z
   This is the zone the user had positioned the cursor within. There are xxxxx zones.

   MSG. Top message or title line, action bar or action bar rule.
   TOP. In a KEEPTOP area for a scrollable window.
   BOTTOM. In A KEEPBOTT area for a scrollable window.
   SCROLLAREA: In the centre window text or data entry area.
   NULL. In the scroll area, but past the end of the window text/
   OUTSIDE. Not within the window.

4. l
   For MSG, TOP, BOTTOM and SCROLLAREA zones this contains the number of the line within that zone.

5. c
   The column within the window the user has positioned the cursor upon. The left most part of the window text is column 1.

6. v
   This is the margin view they are currently looking at. The leftmost view is number 0. This is only active when the `MARGINS` option has been specified.

7. w
   This is the line that the SCROLLAREA zone starts at.

You may extract these fields to work out where within the window the user was pointing, but should not alter them in any way. The result of doing so will be undefined. Remember that NEAT$POSN is only set in `DATAENTRY` and `PICK` windows.

7.0 Action Bars.

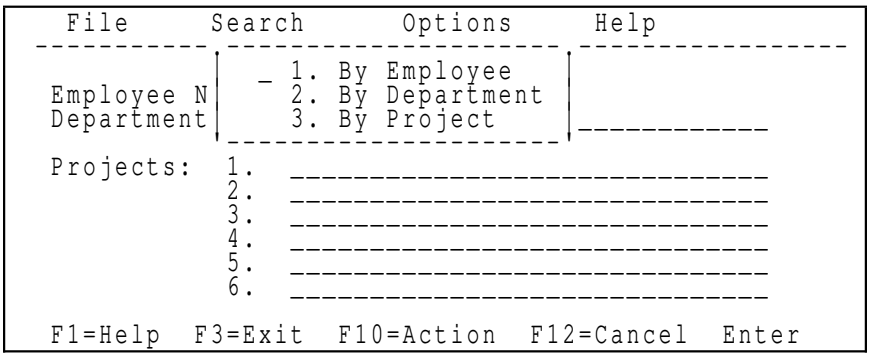
The CUA standard moved application specific functions (eg. Search, Print, etc) from the PF keys and onto the action bar.

The action bar lists these functions as keywords on a line at the top of the panel above the title. To access such a function the user press the PF=Action key to move their cursor from the data area of the window onto the action bar line. (They can just move the cursor directly there themselves if they like; it doesn't really matter on non-programmable terminals).

The user then moves the cursor on top of the function keyword of their choice (eg. Search) and press Enter. Optionally, they may be shown a Pull-down menu with other choices on it (eg. Search by Employee, Search by Department). The application then performs the operation, perhaps throwing up another window to ask for more details (eg. search arguments).

The user may quit out of a pull-down window by pressing PF12=Cancel. In Neatwind we also let them do this just by leaving the pull-down input field blank.

Here is an example of a window with an Action Bar.



7.1 Using Action Bars in Neatwind.

You can specify the action bars on windows in Neatwind using the `ACTION` option. Here is an example:

```
'EXEC NEATWIND (SAA ACTION (File Search Options Help)
```

The keywords in the brackets after the `ACTION` item are displayed in the action bar.

By default, Neatwind won't let a user select an action bar function unless any data entry fields on the screen are valid. Obviously, you would not want this on functions such as “Help” and “Options,” and perhaps even “Search.” You may ask Neatwind to avoid checking on these actions by specifying the `NOCHECK` option. Thus:

```
'EXEC NEATWIND (SAA ACTION (File Search=NOCHECK Options=NOCHECK Help)
```

7.1.1 Pull-downs.

Now, let's suppose the “File” action has three options displayed in its pull-down: “Save” “Save As” and “Quit.” We can also define these in Neatwind:

```
'EXEC NEATWIND (SAA ACTION (File (1. Save@2. Save As@3. Quit=NOCHECK)
                            Search=NOCHECK Options=NOCHECK Help))
```

The pull-down's options are listed in brackets after the appropriate function keyword, separated by at-signs “@.”[8] The number at the start of the pull-down option is followed by a period. That number is what they must type in the pull-down to select that option.

Note the `NOCHECK` option after the “Quit” option. This avoids checking when the user quits. There is no `NOCHECK` option on either “Save” or “Save As,” so Neatwind will enforce field checking when they select those. When `NOCHECK` is not specified in a pull-down, Neatwind will inherit the checking option from the action keyword (“File” in this case, which has defaulted to `CHECK`). `NOCHECK` may be abbreviated just down to just `N`.

7.1.2 What action did they select?

When the user selects an action bar option the results are returned in four variables. If `OUTVARS` was specified they are set as REXX variables in the calling application. If `STACK` or `GLOBALV`, they are passed back either on the stack or in the GLOBALV NEATWIND group respectively.

Neatwind signifies that an Action Bar item has been selected by giving a return code of rc=9998.

Here are the four variables:

1. ACTION$TYPE.

   This is the action function keyword, always in upper case. For example, this would be “FILE” if the “File” action was selected.

2. ACTION$OPT.

   This is the number of the option shown if the user selected from a pull-down following the action. If you did not specify a pull-down for this action then this variable will be set to an empty string (ie. blank.)

3. ACTION$OPTN

   This is the name of the pull-down option whose number is given in ACTION$OPT. For example, if we are using the standard file pulldown and The “New” option was selected then ACTION$OPT would be “1”, and ACTION$OPTN “NEW.” This is always given in upper case.

4. ACTION$ROW, ACTION$COL.

   If you wish to do your own pull-down then here are the row and column on which the top-left-hand-corner of your window should be placed. If you are displaying your own pull-down via Neatwind pass these as the `ROW` and `COLUMN` options.

Neatwind only does one-level pull-downs. If the user wants multi-level or different type of pull-down (say, for options), then don't specify a pull-down for that action, and Neatwind will return control to your program as soon as the user selects it.

---

[8] On non-English keyboards this character may be different. It will be the same as your input field specification character. See “The CTLCHAR Option.” on page 66.

7.1.3 Action Bar Example.

Here is an code fragment showing how the above Neatwind call might be processed by the calling application.

```
do forever
 'EXEC NEATWIND (ACTION (File (1. Save@2. Save As@3. Quit=NOCHECK)' ,
                         'Search=NOCHECK Options=NOCHECK Help)'      ,
                         'DATAENTRY OUTVARS (EMPNR DEPNR PROJECTS.)' ,
                         'TYPE.1,2=WHOLE SAA'
  select
    when $rc = 9998 then select   /* rc=9998 means action bar */
      when action$type = 'HELP' then Say 'General Help Text'
      when action$type = 'FILE' then
        select
          when action$opt = 1 then Say 'Save employee record here.'
          when action$opt = 2 then Say 'Ask for filename, THEN save'
          when action$opt = 3 then leave /* quit */
        end
      when action$type = 'SEARCH' then Say 'call search routine here'
      when action$type = 'OPTIONS' then do
        pdown.0 = 4
        pdown.1 = '@ full-time employees'
        pdown.2 = '@ part-time employees'
        pdown.3 = '@ retirees'
        pdown.4 = '@ vacation students'
       'EXEC NEATWIND (PICK=ANY FORE=WHITE SQUEEZE' ,
                      'COL='action$col 'ROW='action$row
        if rc = 0
        then Pull fulltime +1 partime +1 retired +1 students +1 .
      end
    end
    when $rc = -10 then do; say 'Cancelled out'; leave; end
    when $rc = 0   then do; say 'Ending window'; leave; end
    otherwise do; Say 'NEATWIND Error. rc='rc
  end
end
```

Note the `SQUEEZE` option on the “Options” window. This eliminates the top-title line, and bottom two-lines (including the PF key legend) to make it look like a pull-down window.

You'll note the code for our window is growing as we add these features, but if you look closely you'll note there is not a lot of fat there anyway. A fully-functional specification of the window would take up roughly the same amount of space.

When you can code the workings of an object (eg. window) in the same number of lines it'd take to define it then you have an ideal situation. By that stage any errors you introduce should be functional (forgetting to say you wanted something) rather than coding bugs in trying to implement the same. This is because you are saying *what* you want. Leave the effort of working out *how* to get it to Neatwind.

7.2 Standard Actions.

If you aren't writing your applications with CUA, or you are already familiar with its standards then you may omit this section.

If you want to abide by the CUA standard, it isn't enough to just use action bars. There should be some standard action keywords you provide. We'll briefly discuss these to save you having to rush out and get a copy of the CUA Guidelines.

CUA isn't just about changing your screen colours and PF keys. It's about changing the way your applications interact with the user.

Your applications should manipulate “objects.” Think of an object as being defined by the data that you're displaying in your window or panel. For example, you might be manipulating

“invoice” objects. You can create new objects, read in old ones, save an invoice, and maybe even give it a new invoice number.

Applications are modeless. A user shouldn't have to pick a certain action or option in order to change something on the screen which is displayed in front of them.

The following descriptions are a summary from the “Common User Access Basic Interface Design Guide,” SC26-4583. Refer there for more detail, or clarifications.

Here are the standard CUA action keywords. You are perfectly free to add your own actions applicable to your application.

7.2.1 File.

This action is used to manipulate or create new objects. This action has several options. Not all of these may be relevant to your application.

1. New.

   Create a new object.

2. Open.

   Reads a stored object in. This prompts the user with a window to ask them for the objects name or lookup key. Alternately, you could list all available choices and let the user select one.

3. Save.

   Save the current object.

4. Save As.

   Saves the existing object in a new one without changing the old one. The user is prompted for the name of the new object, if that field is not already on the screen.

5. Print.

   Print the current object.

6. Exit.

   Exit right out of the application. If you don't provide a “File” action then the “Exit” option should be provided at the base of your first pull-down.

7.2.2 Edit.

Think of the way your users organise their work on their desk. They organise objects as pieces of paper, and may cut and paste data between them.

CUA doesn't demand you provide a sophisticated clipboard function on NPTs as is available on PS/2s running OS/2. You could however use something simpler, and just as wonderful to someone who hasn't previously had the ability. For example, your cut and paste manipulations could operate on files. When the user selects these ask them for a CMS filename, and store the data there.

Here are the standard options to Edit. You may not decide to use all of them. Some such as “Undo” or “Mark” may be too expensive computationally.

1. Undo.

   Undo the last executed user action. If you're manipulating text you should at least let the users undo paste, cut, clear and typing changes operations. If you're manipulating objects (eg. employees, invoices) then select an undo appropriate to that object.

2. Mark.

Mark a portion of the object you are viewing for an edit operation. Unless you are manipulating plain text you might omit this option, and work on the whole object by default.

3. UnMark.

   Deselects the marked area.

4. Cut.

   Copy the marked area into the clipboard (a CMS flatfile would be fine), and delete the marked object.

5. Copy.

   Copy the marked area into the clipboard, but leave the marked object intact (ie. not like Cut.)

6. Paste.

   Take whatever is in the clipboard (a CMS flatfile would do for mainframes), and display it as the current object, inserting it at the current location if you are manipulating plain text.

7. Clear (Optional).

   Clears the marked object. If a scrollable portion is cleared the space is not compressed.

8. Delete (Optional).

   Deletes the current object. If a scrollable portion is cleared the deleted space is compressed.

7.2.3 View.

There are different ways the user may want to look at the object, without actually changing it. For example, if they are editing a scrollable list of employees they may wish to view the table using the employee surname instead of their employee number. They may want to only view, say managers, and omit first level employees. In a diary application you may let the user view their diary by day, week or month.

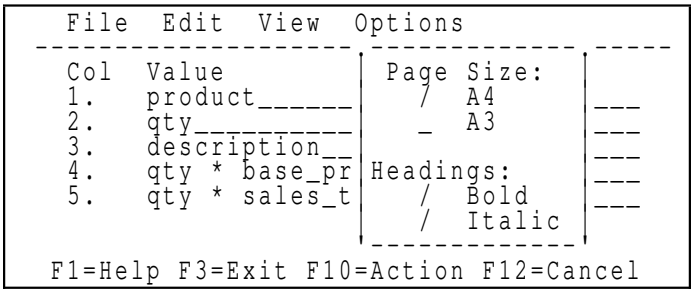
Obviously, the options you present would depend on your application, so there is no “standard” pull-down for the “View” action. Tailor your own to suit the application.

7.2.4 Options.

Options again is dependant on your application. In a report generator options may be whether or not to box the columns, whether to number the pages, etc. These options would be listed here.

The pull-down for this would be a pick list, so you may use the `PICK` option. Use the `SQUEEZE` option to make your window look like a pull-down.

For example:



7.2.5 Help.

With CUA although we supply a HELP PF key, we also supply Help as an action bar function. It has five options on its pull-down.

1. Help for Help.

   This displays help text telling the user how to get help and how to use the help facilities. It'd be best to implement this as a program you can call from all your applications.

2. Extended Help.

   This displays help text giving the user general help about the panel. For example, what it is, what it does, and how to use it. Don't go to excessive detail explaining standard SAA functions.

3. Keys Help.

   This displays help text telling the user what the PF keys and Enter do.

4. Help Index.

   Shows the table of contents of your help file for that application.

5. About.

   This displays a window giving your application name, any logo, author, support information and copyright notice.

This only tells you what help the user has asked for. The next step is to actually display the help. There are two ways you can do this. One is to write your own code to select and display the appropriate help screens. The other is to specify the `HELP` option to do it for you. For more information on this see “Integrated Help.” on page 45.

7.2.6 Standard Pull downs.

As File, Edit and Help all have standard pull downs Neatwind provides a quick way to define them. Simply use the `STD` option when you define the action bar.

For example:

```
'ACTION (File=STD Edit=STD Search=NOCHECK Options=NOCHECK Help=STD)'
```

The standard options for File are 1. New, 2. Open, 3. Save, 4. Save As, 5. Print and 6. Exit. The same for Edit are 1. Undo, 2. Cut, 3. Copy, 4. Paste, 5. Clear and 6. Delete. Those for Help are 1. Help for help, 2. Extended Help, 3. Keys Help, 4. Help index and 5. About.

Note that we left the Mark and UnMark options out of the Edit function? This is because we believe they'd be most commonly used in text editor type windows, which would be better done with Xedit's inbuilt screen handler. If you do however wish to define these options then specify them manually in the Neatwind command line above.

If you specify the `STD` option on a function keyword with no standard pull-downs an error will result.

7.2.7 Object Oriented Applications.

We have already discussed how the *File* pulldown is used to mainpulate an object type. Let's take an office inventory system as an example, used to display and mainpulate a PC object type. For each PC we may record our own unique asset number, part number, serial number and entrusted owner.

The *New* option under the File would create a new PC *object.* The *Open* option would load an already existing PC object, and the *Save* option would save an edited object back into the database. *Save As* would let the user save the current PC object under another asset number. Although not part of CUA, Neatwind recognises the need to be logically consistent and thus also supports a *Delete* option under the *File* pulldown. This allows the user to remove an object no longer needed.

The problem is that supporting the options under the *File* menu demands a fair bit of application code. For example, if the *Open* option is selected you must make sure they save any changes they have made to the object they are currently viewing. Next you must ask for the name of the object to open, making sure it exists. If you are performing a *New* or *Save As* you must make sure the new object will not overwrite an old one with a different name.

7.2.7.1 The ASSIST Option.

The `ASSIST` option removes this burden from the application. Instead, Neatwind will ask the user if they want to save any changes, ask for the name of the new object (if necessary), and confirm if the new object will overwrite an old one.

In order for Neatwind to carry out this dialogue you must pass it a description and list of the objects being mainpulated. This information is assembled in a REXX stem, whose name is passed with the `ASSIST` option.

7.2.7.2 Defining your Objects.

Here is an example for the PC object type we discussed above.

```
object.title    = 'PC'
object.key      = 'Asset Number'
object.akey     = 'an Asset Number'  /* for correct grammar */
object.length   = 4
object.type     = 'WHOLE'

object.currname = ''
object.changed  = 0   /* false */

object.0 = 2
object.1 = '0030'
object.2 = '0038'
```

`object.title` is the name of the object. This should be singular, rather than plural. `object.key` the name of the key that uniquely identifies it. For correct grammer, you should also provide `object.akey`. This refers to a specific instance of the key. For example, if the `key` is "asset number" then `akey` would be "an asset number."

`object.length` and `object.type` are the maximum length and datatype of the key. The latter is the same as the datatype specifications used with the `TYPE` option.

`object.currname` is the key of the currently opened object. If we are starting from scratch this will be blank. If we have opened an object (say asset number 0030), then it would hold that value. `object.changed` is a boolean; 1 for true or 0 for false. Your application should check the `NEAT$CHANGED` REXX variable, and set `object.changed` to true if a change occurs.

Finally, `object.0` contains the number of currently existing objects. `object.1` contains the key of the first object. After the key (from the 5th character onwards in this case), we may optionally provide a more detailed description. For example, "Narelle's PS/2."

The case of the object names is preserved, but ignored when checking for objects that already exist or will be overwritten.

When we call Neatwind we pass it the `ASSIST` option, followed by the name of the stem you have used. This is "`OBJECT.`" in our case, Note that the stem name *must* end with a period. You must also use the `ACTION` option to define the *File* pulldown. It does not matter whether you using the standard `STD` pulldown, or define your own.

The *File* pulldown you use however *must* have a *Save As* option defined. The user will need to use this to save an object that has not yet been named.

7.2.7.3 Example.

Here is a code fragment to drive our inventory display. While it is a long example, consider how much longer it would be if we had to code our own dialogue windows to support the *File* pulldowns.

```
$.true = 1
$.false = 0

quit = $.false
do until quit
 'MAKEBUF'
  Queue "Part Number:    @@@@@@@"
  Queue "Serial Number:  @@@@@@@@@@@@"
  Queue "PC Assigned to: @@@@@@@@@@@@@@@@@@@"
 'EXEC NEATWIND 3 (DATAENTRY OUTVARS (PARTNR SERIALNR OWNER) ' ,
      'ASSIST OBJECT. ACTION (File=STD Help=STD) NOBLANK' ,
      'SAA90 TITLE (Viewing PC Number' object.currname')'
  $rc = rc
 'DROPBUF'

  /* Remember if the object is changed */
  if neat$changed = $.true
  then object.changed = $.true

  select
    /* Action Bar */
    when $rc = 9998 then
      select
        when action$type = 'FILE' then
          select
            when action$optn = 'NEW' | action$optn = 'OPEN' then do
              if object.savechanges
              then call save_pc object.currname
              object.currname = object.newname
              object.changed  = 0  /* false */

              if action$optn = 'NEW'
              then do; partnr = ''; serialnr = ''; owner = '';
              else call load_pc object.currname   /* Open */
            end
            when action$optn = 'SAVE' then do
              call save_pc
            when action$optn = 'SAVE AS' then do
              call save_pc object.newname
            when action$optn = 'PRINT' then
              call print object.currname, partnr, serialnr, owner
            when action$optn = 'EXIT' then do
              if object.savechanges
              then call save_pc object.currname
              quit = 1  /* true */
            end
          end  /* File */
        when action$type = 'HELP then
          call help
      end /* Action Bar */

      /* SAA90 style Enter - saves changes and ends window */
      when $rc = 0 then do
        if object.changed
        then call save_pc object.currname
        quit = 1  /* true */
      end

      /* SAA90 style Exit - terminates, confirming if necessary */
      when $rc = 9998 | $rc = -10 then do
        if object.changed then do
```

```
            q.0 = 1
            q.1 = 'Save Changes?'
           'EXEC NEATWIND 1 (SAA90 QUERY SHOWSTEM Q.'
            if rc = 1 then call save_pc object.currname
          end
          quit = 1 /* true */
        end

        otherwise Nop
      end
 end
```

If the New, Open or Exit options are selected we check the `object.savechanges` REXX variable. This will be set to true iff there were changes made to the object, and the user wishes to save them before continuing.

The New, Open, Save As and Delete options will set the `object.newname` REXX variable to hold the key of the object to create, load save as or delete respectively.

Let's take the New option as an example. The first thing we do is see if we have to save the old object. If we do we call the `save_pc` procedure. We clear the data fields on the screen, and set the `object.changed` variable to `false` as we are starting afresh. Finally we move the new name held in the `object.newname` variable into `object.currname`, reflecting the new name of the current object.

Note that the `save_pc` procedure would have to update the `object` stem if a new object is being added (by increasing `object.0` by one, and inserting the key in the new slot). Similarly, if we are using the *Delete* keyword we would have to remove the deleted entry from the stem.

Using the `ASSIST` option will decrease code size and increase quality when you are programming the CUA style interface.

7.2.7.4 What we haven't discussed.

We only addressed CUA's object oriented application interface. We have not addressed true object oriented interfaces. These have many additional and more intuitive functions, such as the ability to erase an object by dragging it to a trash can icon.

Similarly we have not discussed object oriented programming, which provides a far easier way to program in these sort of environments then traditional coding techniques.

8.0 Integrated Help.

Well written documentation can subtantially increase the ease of use of your application, increasing user satisfaction, and reducing your time spent on support. Sadly this isn't realised by a lot of application developers, who look at documentation as as an unproductive and uncreative waste of time.

Not so! These developers should realise that good documentation will increase the rate at which users look at a new application. It also increases the likelihood of them continuing to use it, rather than dumping it in the “too dificult” basket. Aside from surveys, one of the best ways to monitor user satisifaction is to actually monitor application usage. I suggest you do this; the results can be a real eye opener.

That said, the second most common reason for there being poor documentation is that until now there has been no standard method for creating online help, or for presenting it within the application. This meant the developer often had to come up with their own help system, often on an application by application basis. The result of this was inconsistency, and a lot of wasted effort by developers having to program the help functions individually into each screen.

Neatwind's Integrated Help Facility addresses all of these concerns. It allows help to be created in a GML or BookMaster SCRIPT file, so it is easy to write an easy to maintain. Secondly, a single option instructs Neatwind to handle display of the help text. You don't even have to define the Help PF key. Neatwind will take care of that (among other things) for you. This will reduce the size and complexity of your application code. Finally, Neatwind uses the SAA CUA standard for display of help text. This is consistent across CUA compliant applications, and you will also find it quite easy to use. Personally, I find it easier to use than the hardcopy manuals.

8.1 Preparing the Help Documentation.

This section instructs you on how to create and maintain your help documentation, and how to prepare it for use by Neatwind.

8.1.1 Naming Conventions.

First give your help document a name up to five characters long. If your application is small to medium in size then you will probably name the help document after the application. If your application is larger you may choose to carve it into functions and have a help document for each function.

Let's produce a help document for a sample Payroll application. Since we are restricted to having a name no longer than five characters we shall simply call it `PAY`.

It doesn't actually matter what you call the help document providing it is consistent, and doesn't clash with the names of any other help documents. You will use the name you give your help document to tell Neatwind where to look for the help.

Next we must identify the following for each window or fullscreen panel within the application.

1. Panel Identifier.

This is usually a code no more than 8 characters long uniquely identifying the window or panel within the application (though in Neatwind this code may be as long or as short as you like). We shall assume three screens, `PAYROLL01`, `PAYROLL02` and `PAYROLL03`.

2. Fields on that screen.

   For `DATAENTRY` and `PICK` windows identify each field on the screen. Let's assume that panel `PAYROLL01` has three fields; employee name, employee number and salary. If the window displays these using the `OUTVARS` option then we would use the specified REXX variable names to identify the fields (which will hopefully be meaningful).[9] If we are using the `GLOBALV` or `STACK` options then we identify them by the order within which they are displayed within the window. If they are displayed in the above order then employee name would be 1, employee number 2 and salary would be 3.

3. Programmable Function Keys.

   What Programmable Function Keys are active on the screen and what do they do?

Now let's create that help document...

8.1.2 Creating the Document.

You can create the help document in SCRIPT GML or BookMaster format. If you are familar with neither of these products then we suggest you do a course. Alternately read the introductory GML guide, the "Generalised Markup Language Starter Set User's Guide," SH20-9186.

The file you edit should have a filename consisting of your help document name (`PAY` in our above example), catentated with the three-character national language code you are using. For example, `ENG` is the code for U.K. English.[10] The filetype of the document must be `SCRIPT`. Thus the Payroll Help File for U.K. English would be `PAYENG SCRIPT`.

Now, for each panel we identified in the application add the following chapters to your help document. It is important that the chapter names correspond to the conventions listed below.

1. Extended Help.

   Chapter Name: *panelid: comments*

   This describes the contents of the panel, and describes what tasks may be performed from the panel. If the panel is a `MENU` or `QUERY` window or panel you should also describe the sole input field. Extended Help doesn't define input field level help for other types of windows. We do that in a separate chapter:

2. Contextual Help.

   Chapter Name: *panelid fieldname: comments*

   For `DATAENTRY` and `PICK` windows we describe the individual field level help here. It describes what the field is, what its valid values are, where is comes from and what is represents. The *fieldname* used is either the REXX Variable name if used with an `OUTVARS` option, or the field number within the window otherwise. Other types of windows don't have contextual help.

The following chapters can either be defined on a panel by panel basis or once for the entire application. For example, if the PF keys are uniform across the entire application we might just have

---

[9] If you are using `OUTVARS` to return a single bit string from a `PICK` window (as described in "Bit Strings." on page 14), then the field name will be the number of the field within the screen, rather than the variable name. In this case the latter would be useless, as the same variable name would apply to all fields.

[10] A full list of NLS language codes is given in "Appendix A. National Language Codes." on page 97. National Language Support is discussed in "National Language Support." on page 68.

a single chapter called “Keys Help.” If however they change then we would have three chapters; “PAYROLL01 Keys Help,” “PAYROLL02 Keys Help” and “PAYROLL03 Keys Help.”

3. Keys Help.

   Chapter Name: *panelid KEYS HELP:comments*
   Chapter Name: *KEYS HELP:comments*

   This chapter lists the PF keys active on that panel and what they do. List the SAA CUA standard keys and provide a brief description of what they do, but there is no need to go into detail as user's should already have a good idea of what these keys do.

4. Tutorial (optional).

   Chapter Name: *panelid TUTORIAL:comments*
   Chapter Name: *TUTORIAL:comments*
   Chapter Name: *TUTORIAL*

   This teaches the user how to use the panel, and what they can do with it. It could include examples and exercises (though under Neatwind these cannot current be interactive. It is recommended you at least provide at least examples, as people tend to learn faster by looking at a few examples rather than reading several pages of abstract definitions.

5. Reference Phrase Help (optional).

   Chapter Name: *panelid REFERENCE PHRASE HELP:comments*
   Chapter Name: *REFERENCE PHRASE HELP:comments*
   Chapter Name: *REFERENCE PHRASE HELP*

   ;This chapter is a glossary of words and phrases which may need explanation for the less experienced user. For example, an application which manages an SQL/DS machine might define “DBSPACE” or “REORGANISE A DBSPACE” in the glossary.

   Neatwind differs slightly from CUA here. CUA advocates that this is displayed in two-level form; One that lists all the words and phrases in the glossary, each entry being selected for a more detailed explanation. Neatwind keeps a flat glossary for simplicity, though this may be revised in the future.

The following chapter is defined only once.

6. About.

   Chapter Name: *ABOUT*

   This is a short chapter, no more than 20 lines long. This displays ownership and copyright information about the application.

Your help document will eventually be displayed using Neatwind, so we recommend you avoid use of special Neatwind characters and symbols, such as the amphersand `&` or the escape character, `%`.

Comments in the chapter name are optional, but we recommend you use them as they are easily put to good use. For example, if panel `PAYROLL01` was for Employee Registration, then an appropriate chapter name would be “PAYROLL01: Employee Registration.”

When you write the help text try and be clear and concise. Write in plain language. When someone is looking for help the last thing they want to do is wade through a plethora of literary artistries. Write for a ten year old. Also use plenty of white space too. That is, use paragraphs to separate points. This makes the text much easier to read.

8.1.3 The NEATHELP Imbed.

When you are preparing your document for Neatwind you should use the SCRIPT `.im` command to imbed the file `NEATHELP SCRIPT` into your document. Further, you should also remove

any front matter (such as the title page, abstract, table of contents, etc.) If you include these Neatwind will get confused and point to the wrong pages when you ask for help. The first tag in your SCRIPT document should be a `:h0.` or `:h1.` starting the first chapter.

You may however want to print your document out on a printer whilst you are working on it. If you structure your document as follows you will be able to do this and include front matter, without corrupting your help file when you create it later.

```
.* If generating help then skip front matter
.if 3820A eq 3270
.th .im neathelp
.th .go skipfr
:gdoc.
:frontm.
:titlep.
:abstract.
:preface.
:toc.
:body.
...skipfr
:h1.PAYROLL01: Register Employee.
 .
 .
```

This SCRIPT code simply skips over the front matter if the file is being scripted for a screen (and thus the help files are being made), rather than a printer.

Of course, If you don't want to include any front matter and don't intend to print the file you can simply go

```
.im neathelp
```

instead.

8.1.4 The NEATMAKE Tool.

Once you are happy with your help document it is time to convert it for use with Neatwind. This is acheived by using the `NEATMAKE` tool. Suppose we have entered our help text into the file `PAYENG SCRIPT A` in GML format. We convert it by entering the CMS command:

```
NEATMAKE PAY
```

This produces two files; `PAYENG NEATHELP` which is the formatted help text, and `PAYENG NEATINDX` which is the index. Keep both of these files with your application. If you want to change the help text, simply edit the `SCRIPT` file and run `NEATMAKE` again. Then replace the `NEATHELP` and `NEATINDX` files with your application with their latest versions.

If your help text is in Bookmaster format then issue the CMS command

```
NEATMAKE PAY (BOOKMASTER
```

instead. Note though that to use this format you must be linked to the BookMaster disk on your node. This varies from installation to installation, so if you are unsure consult your local Help Desk or Information Centre.

8.1.4.1 Other languages.

By default `NEATMAKE` looks for and proceses the U.K. English Help text. You can ask it to process a different languages help file by specifying the `NLS` option.

For example assume we have written three help files for our Payroll application, in U.K. English, Portuguese and French. These would be named `PAYENG SCRIPT`, `PAYPTB SCRIPT` and `PAYFRA SCRIPT` respectively. To process all of these files we would have to enter the following CMS commands, one after the other:

```
NEATMAKE PAY
NEATMAKE PAY (NLS PTB
NEATMAKE PAY (NLS FRA
```

We did not have to specify `NLS ENG` on the first call as `ENG` (meaning U.K. English) is the default. Note that you must create one help file for each language you intend to support. Translation is *not* performed automatically. For further information on National Language Support refer to "National Language Support." on page 68.

8.2 Presenting the Help.

We've now discussed preparing your help document for Neatwind. Now we'll discuss the `HELP` option used to display it. This provides a very painless, easy way to integrate the help files you've produced into your application.

8.2.1 The HELP Option.

Let's take a simple example for panel `PAYROLL01` of our payroll application.

```
 p.0 = 5
 p.1 = 'Please enter the Employee''s Details.'
 p.2 = ''
 p.3 = 'Employee Name:   @@@@@@@@@@@@@@@@'
 p.4 = 'Employee Number: @@@@@@'
 p.5 = 'Salary:          @@@@@@@@@@@@@@@@@@@@@@@@'
'EXEC NEATWIND' ,
     '(DATAENTRY OUTVARS (NAME SERIAL SALARY) SAA' ,
      'TITLE (PAYROLL01  Employee Registration)' ,
      'SHOWSTEM P. HELP (PAY PAYROLL01 *)'
```

Look at the option `HELP (PAY PAYROLL01 *)`. The `PAY` says to use the files `PAYnnn NEATHELP` and `PAYnnn NEATINDX` produced by `NEATMAKE`, where `nnn` is the National Language Code. Since the default is `ENG` (U.K. English) and another hasn't been specified, Neatwind will use the files `PAYENG NEATHELP` and `PAYENG NEATINDX`.

The second parameter, `PAYROLL01` is the panel identifier. The third parameter, the asterix `*` is optional. When specified it tells Neatwind that contextual (field level) help is available for this panel.

If you create some example help files and run the above code fragment the resulting window will look like this:

```
PAYROLL01   Employee Registration
Please enter the Employee's Details.

Employee Name:   ________________
Employee Number: ______
Salary:          ______________________

F1=Help F3=Exit F12=Cancel
```

Did you notice that the PF key `F1=Help` was automatically defined? If you move the cursor onto one of the fields (say Employee Name) and press `F1=Help` the help text will appear for that field. If the cursor is not on a field when you press the help key you will be shown the extended help for the panel.

Let's look at another example.

```
 m.0 = 2
 m.1 = '   1. Register an Employee.
 m.2 = '   2. Delete an Employee.'
'EXEC NEATWIND (MENU SAA' ,
      'TITLE (PAYROLL02  Payroll Main Menu)' ,
      'SHOWSTEM M. HELP (PAY PAYROLL02)'
```

Because this is not a `DATAENTRY` or `PICK` window, there is no contextual help available. Thus, we don't specify the asterix `*` in the `HELP` option.

8.2.2 The Help Pull-down.

The `HELP` option will also handle the Help pulldown of any action bar you define. For example;

```
 d.0 = 5
 d.1 = ''
 d.2 = 'Employee Number: @@@@@@'
 d.3 = ''
 d.4 = 'Enter the serial number of'
 d.5 = 'the employee being deleted.'
'EXEC NEATWIND' ,
     '(DATAENTRY OUTVARS (SERIAL)' ,
      'TILE (PAYROLL03 Delete an Employee)'
      'ACTION (File (Exit=NOCHECK) Help=STD)' ,
      'SHOWSTEM D. HELP (PAY PAYROLL03 *)'
```

This is how the window would appear with the Help pulldown active:

```
File  Help
------┌----------------┐-------------
PAYROL|1. Help for Help|mployee
      |2. Extended Help|
Employ|3. Keys Help    |
      |4. Help Index   |
Enter |5. About        |r of
the em'----------------'ed.

F1=Help F3=Exit F10=Action F12=Cancel
```

Again the PF key `F1=Help` is automatically defined, but now there is also an action bar with a File and Help pulldown. Neatwind will process any options the user takes off the Help pull-down. Note though that although the PF key was automatically defined, the Help pull-down was not. You must manually define it with the `ACTION` option.

You don't have to use the `Help=STD` pull-down. You can use any Help pulldown option defined in the “Common User Access Basic Interface Design Guide,” SC26-4538. Here is a list of those accepted Help pulldown options:

1. Help for Help.
2. Extended Help.
3. Keys Help.
4. Help Index.
5. About.
6. Tutorial.
7. Reference Phrase Help.

8.2.3 Returning from Neatwind.

The `HELP` option instructs Neatwind to handle the help internally. Thus there is no need for your application code to check for `F1=Help` being pressed (which would normally give a return code rc=1), as Neatwind now handles this case by itself.

However, there may be cases where you want Neatwind to return with some cases and not others. You can specify this by entering a *help level* keyword between the `HELP` option and its parameters in brackets. For example, `HELP PFKEY (PAY PAYROLL02)` tells Neatwind to handle `F1=Help` internally, but to return to the application if the Help pulldown is used.

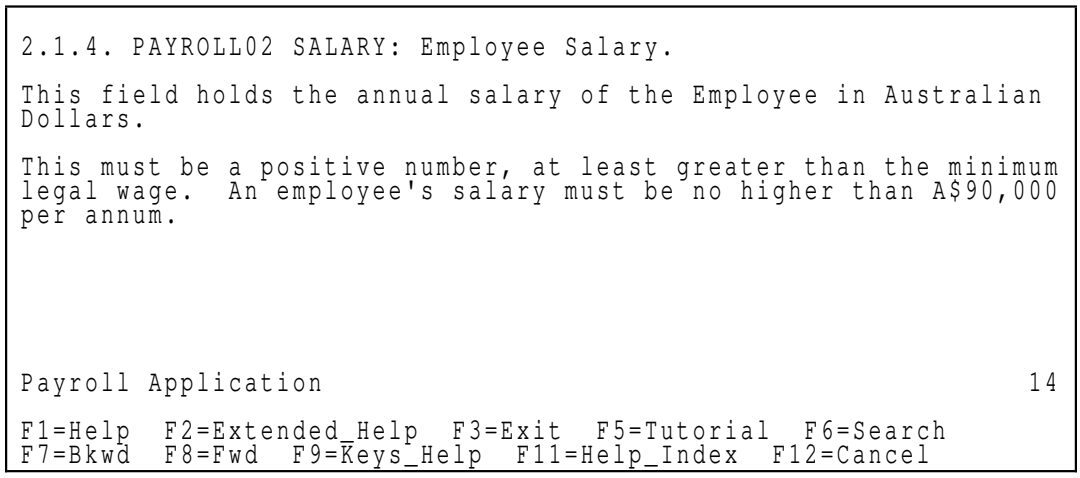
Alternately, `HELP ACTION (PAY PAYROLL02)` says handle `F1=Help` being pressed, and any Help pulldown option being selected except "About." We allow this as the About window displayed from a help screen isn't visually very inspiring. You may want to create your own window using different colours and pretty formatting.

Finally, `HELP ABOUT (PAY PAYROLL02)` tells Neatwind to handle both `F1=Help` and all Help pull-down options, including "About." This setting is the default. Thus we could have said `HELP (PAY PAYROLL02)` instead, which would have exactly the same effect.

All other PF keys and action bar pulldowns return normally. They are not effected by any of these settings.

8.3 Viewing the Help.

Help is displayed in a fullscreen panel complying to the SAA CUA 1990 Standard. This is an example of what your users would see if they requested help from an application using Neatwind's integrated help.



Here is a description of the valid PF keys on this screen and what they do. Note that the `Tutorial` and `Keys Help` PF keys only appear if the appropriate chapters are in the help document.

PF key	Type	Description
1	*Help*	Displays help for viewing the help screen.
2	*Extended Help*	This displays the general help about the screen you are currently viewing. Suppose you are viewing contextual help about a single field on a screen. Pressing this key would show you help describing the screen in general.
3	*Exit*	Leave the help screen and return to the application.
5	*Tutorial*	This takes you to a tutorial screen, displaying exercises or examples. The tutorial might be specific to the screen you were using, or a general one for the whole application.
6	*Search*	This lets you either search for a certain chapter, or a certain page. A window will appear to prompt you for the search argument.
7	*Bkwd*	Scroll backwards through the help document.
8	*Fwd*	Scroll forwards through the help document.
9	*Keys Help*	Display a list of PF keys active on the screen you were viewing, and descriptions of what they do. This may either be specific to that screen, or a general list applicable to the whole application.
11	*Help Index*	This displays the table of contents. You may move the cursor beside a chapter of interest, type a slash (the SAA selector character) and hit Enter. This will take you to that chapter within the document.
12	*Cancel*	This leaves the help screen and returns to the application.

8.4 Using NEATHELP Standalone.

You now know how to use the Integrated Help Function. For nearly all applications this will be sufficient. This following section discusses how to interface into integrated help directly. This won't be applicable to most users, so you may choose to skip this section.

There may be occasion where you want to call help directly. For example, to go directly to a tutorial if the user chooses to do so off the main menu.

To invoke help directly from your application call the `NEATHELP` program. Pass it the help filename, panel identifier (and field name if applicable) as parameters. In the options specify the type of help you want displayed; `HELP FOR HELP, EXTENDED HELP, KEYS HELP, HELP INDEX, CONTEXTUAL HELP, ABOUT, TUTORIAL` or `REFERENCE PHRASE HELP.`

For example,

```
'EXEC NEATHELP PAY PAYROLL02 SALARY (CONTEXTUAL HELP'
```

would display the contextual (field level) help for the salary field on panel `PAYROLL02.` If you omit the help type from the options `NEATHELP` will display the chapter whose title is passed as a parameter. For example, if your help text contains a chapter “The National Wage Case” you could invoke it via the call:

```
'EXEC NEATHELP PAY The National Wage Case'
```

Case is actually ignored, so

```
'EXEC NEATHELP PAY THE NATIONAL WAGE CASE'
```

is equally valid. Chapter names must be unique to the first 32 characters.

You can also pass NEATHELP the NLS and CODEPAGE options in an indentical format that of the Neatwind program. This allows you to use different language help files. ENG (U.K. English) is the default National Language. See “Changing Neatwind's Tongue.” on page 73 for details.

By default, NEATMAKE discards anything in a chapter name after a colon as comments. These are discarded. If for some reason you want to keep the text after the colon simply specify the NOCOLON_MODE option when you call NEATMAKE. In practice, it would be rarely, if ever, that you would have a need to do this.

8.5 Trouble Shooting.

If you find the wrong help screens are being displayed when you select a help function, then you may have specified the file or panel identifier incorrectly to the HELP option.

If context sensitive help is not being shown on a DATAENTRY or PICK window, then you either didn't include the field level chapters in your help text, forgot to pass the asterix * to the HELP option, or you're not using the correct field names.

If the wrong pages appear then make sure the NEATHELP and NEATINDX files are synchronised. That is, make sure they have the same date and time (within a few seconds) when you look at them with FILELIST. If they are synchronised make sure you did not include any front matter in the generated version. You can spot this by looking at the front of the NEATHELP file.

If you get an error saying that Neatwind cannot find the help then make sure you have included both the NEATHELP and NEATINDX files with your application disk.

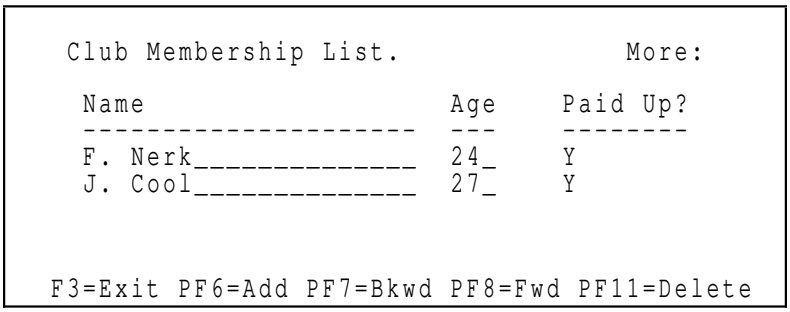
If Neatwind says it cannot find any topic then make sure you have not lost the NEATINDX file.

If all else fails use NEATMAKE to remake the help files, and place them with your application.

9.0 Dynamic Data Entry Windows.

Occasionally you may need to write an application screen that lets the user dynamically add or delete records from a list. Traditionally these are quite complicated windows to program. Neatwind fortunately, provides a much easier way to implement these.

Here is an example of such a window:



When the user presses PF6=Add a blank record is inserted in the window. When they press PF11=Delete the cursor is on top of is deleted. Only records in the scrollable area of the window are deleted. This stops people for example, deleting the title or PF key line.

9.1 The TEMPLATE Option.

First we must define a template of the line we shall insert if they press the PF6=Add key. We do this using the TEMPLATE Option. For the above window the definition is quite simply:

```
TEMPLATE (@@@@@@@@@@@@@@@@@@@@  @@@ @)'
```

This is the same as the line we would define in the window text. This one consists of three input fields; 20, 3 and 1 character long respectively. You might have to adjust the spacing (as you would when you define a normal DATAENTRY window), as there is a limit to how closely Neatwind can place adjacent input fields.

We can give the inserted field TYPE and LENGTH attributes by using negative numbers to indicate those of the inserted field (as opposed to those already displayed in the window). In the above case we might say:

```
LEN.-1=MANDATORY TYPE.-2=WHOLE TYPE.-3=(CHAR MF)
```

-1 means the first field in the template, -2 the second and so on.

9.2 The Add and Delete PF keys.

No matter what you decide to call these PF keys, they have two meanings; one is `INSERT` and the other is `DELETE`. You specify these PF key modifiers when you define the PF key, as you would the other PF key modifiers `NOCHECK` and `HELP`. For example:

```
PF6=Add=INSERT PF11=Delete=DELETE
```

You do not have to use these PF key numbers, and you do not have to call them `Add` and `Delete`. The following is equally valid:

```
PF2=InsStudent=INSERT PF4=WipeStudent=DELETE
```

9.3 Implementing the Dynamic List.

Here is the implementation for the above screen:

```
 name.1 = 'F. Nerk';  age.1 = 28;  paidup.1 = 'Y'
 name.2 = 'J. Cool';  age.2 = 54;  paidup.2 = 'Y'
'MAKEBUF'
 Queue "  Club Membership List &MORE."
 Queue " "
 Queue "  Name                Age     Paid Up?"
 Queue "  ---------------     ---     --------"
 Queue "@@@@@@@@@@@@@@@@ @@@ @"
 Queue "@@@@@@@@@@@@@@@@ @@@ @"
'EXEC NEATWIND (OUTVARS (NAME. AGE. PAIDUP.)' ,
     'DATAENTRY NOBLANK CHECKTYPE=IMM SAA' ,
     'LEN.-1,1+3=MANDATORY TYPE.-2,2+3=WHOLE TYPE.-3,3+3=(CHAR YN)'
     'PF6=Add=INSERT PF11=Delete=DELETE' ,
     'TEMPLATE (@@@@@@@@@@@@@@@@ @@@ @) KEEPTOP=4' ,
 $rc = rc
'DROPBUF'
```

We start off by displaying two members who have already joined. We initialise the REXX variables with their data and display this using the `NOBLANK` option. We pre-insert two of the templates in the window text for them.

We specify the type and length attributes for the template fields (using negatives), and for the fields already displayed in the window. For the latter we specify (eg. “1+3”) to say apply this type or length to the first field, and then go up in increments of three until you run out.

We define the PF keys we are using for additions and deletions, specify the `TEMPLATE`, and that's it!

When Neatwind returns control to the calling program the REXX variables nominated in `OUTVARS` will contain the edited field values. The zero element of each of these stems will contain the total number of subscripts. For example, if the user quits with 5 records on the screen then in this case NAME.0, AGE.0 and PAIDUP.0 will all be set to “5.”

10.0 Horizontal Scrolling.

Neatwind can display windows up to 72 characters wide. Full screen panels may use the full physical screen width, typically 80 characters.

In spite of this you may still have occasion where you want to display lines to long for either of these formats. For example, a window listing an employee database may display one employee record per line. Each line may list an employee's name, phone number, job title, manager, department and location. Let's assume the width of these fields are 40, 20, 30, 40, 10 and 30 characters respectively. If we allow two spaces between each field (one space usually isn't sufficient to break the visual continuity), each line would measure 180 characters.

CUA advocates this situation be handling by splitting up the line into views; each view shows only one segment of the line. To see the other segments of the line the user presses a `left` or `right` programmable function key. For example, the first view might display the employee name and phone number. The next could display their job title and manager. The last could display their department and location.

10.1 The MARGINS Option.

Let's assume the personel application formatted the personel database as described above; with the fields in that order with two spaces separating each one.

The first view (name and phone) would occupy columns 1 to 62. The second (job title and manager) would use columns 65 to 136. The final third view (department and location) would occupy columns 139 through to 180. Note how we have not included columns 63, 64, 137 or 138? These are the two space field separators we talked about. There is no need to display these when the columns they separate lie on a view boundary. We have left the blank separators at columns 41, 42, 95, 96, 149 and 150 as they separate fields in the middle of a view.

To ask Neatwind to enable horizontal scrolling against those three views we specify the `MARGINS` option.

```
'EXEC NEATWIND (SAA87 MESSAGE MARGINS (1-62 65-136 139-180)'
```

We pass with this option the columns defining the three views we wish to display. Two new PF keys, `left` and `right` will also automatically appear. The user will be able to scroll horizontally using these keys.

Windows may be both vertically and horizontally scrollable at the same time.

10.2 The Left and Right PF keys.

CUA stipulates that PF keys 19 and 20 are used as the left and right keys. Unfortunately this doesn't sit well with a large number of terminals. Some only have 12 PF keys (numbered 1 to 12). Others have the full 24, but PF keys 1 to 12 are only accessible with the ALT key. On these terminals most people tend to use PF keys 13 to 24 which are easier to access via the keypad.

Neatwind thus automatically selects PF keys 9 and 11 to play the role of the left and right keys. You may override this selection by manually defining your own PF keys for left and right. You are however still restricted to using only PF keys 1 through 12. For example,

```
'EXEC NEATWIND (SAA87 MESSAGE MARGINS (1-62 65-136 139-180)' ,
               'PF4=LEFT PF5=right'
```

Note that the case used to define the keys is not signficant. They must however be defined in English. If you are using another national language (specified via the `NLS` option), they will appear translated on the displayed window.

10.3 Alignment.

You may set up margins on any type of window. The columns you specify as margins are applied against the window definition you pass via the stack, `PANEL` or `SHOWSTEM` option. This may be slightly different from what actually is displayed on the screen.

Take the `DATAENTRY` window for example. Spaces may be inserted before or after the displayed input fields in order to meet the rules of the 3270 architecture. If you are using the `COMPRESS` option then two character escape sequences appearing in the window definition will only take one space on screen.

10.3.1 Lines with Field Marks.

To make things easier you have the option of using “field marks” within selected lines. These will slice those lines into the views, overriding the column specifications of the `MARGINS` option.

The field mark character is accessible off your keyboard via the “field mark” key. Your terminal knows it as control character #5, so you may also hardcode it with the REXX expression `x2c('05')`. Since this may not be available off all keyboards, or may be lost when you are exchanging files between a PS/2 and a host you may redefine this character as discussed in “The CTLCHAR Option.” on page 66.

Simply embed the field mark character in the line you are passing to Neatwind. Use it to divide which parts of the line you want to go into which view. The line segment before the first field mark goes into the first view. That before the second goes into the second view, and so on. Thus a line using field marks to delimit three views would be in the following form:[11]

```
1st view <FieldMark> 2nd view <FieldMark> 3rd view
```

For example,

```
'MAKEBUF'
 Queue 'Welcome to the Fawlty Towers Reservation System.'
 Queue ' '
 Queue 'Please provide your%iSurname:*@@@@@@@@@@@@'
 Queue 'your%ifirstname:*@@@@@@@@@@@@'
 Queue 'your age: @@@@@*your sex: @ (M or F)'
 Queue 'We hope you have an enjoyable stay.'
'EXEC NEATWIND (DATAENTRY MARGINS (1-29 30-60) CTLCHARS (   *)'
'DROPBUF'
```

[11] If you are using the `PREFIX` option (discussed later), then this format is slightly different. Refer to “Use with Field Marks.” on page 61.

Here we have redefined the field mark to an asterix using the CTLCHARS option. Lines that don't include the field mark are divided strictly along the margin columns. For example, the first line would be split into two views; “Welcome to the Fawlty Towers” and “Reservation System.”

Lines that include field marks are split along them. They are *not* split according to the column ranges specified in the MARGINS option. For example, the third line would be split into two views; your%ifirstname: and the input field @@@@@@@@@@ If you look at the example you will see there is a field mark character between these two line segments.

10.3.2 Lines without field marks.

Lines without field marks are cut along the margins. You should avoid margins that cut through input fields.

For example, if a 10 character input field is cut at the third character then it will appear in two views. In one it shall be a 3 character field, and in the other a 7 character field.

Note that the first character of field displayed on the screen will always be the first in the field. For example, if the field above held the value Budgerigar then in the first view (as a 3 character field) it would appear as Bud, and in the second (7 character one) it would appear as Budgeri. Because we have cut the field in this way the 8th, 9th and 10th characters will not be accessible.

In conclusion, do not use the MARGINS option to cut field at a midpoint. If you do, the field displayed will be truncated. To save the burden of trying to figure out which fields will and will not be bisected you may use the field marks in lines containing input fields.

It is not possible to display an input field wider than the maximum window or fullscreen width (72 or 80 characters). That is, you could not define an input field say, 200 characters long and expect to split it into 3 views. If you were to do this each view would only show the first 72 or 80 characters of the whole field.

10.4 Overlapping Margins.

The columns you specify as the MARGINS may overlap, or even include one view within another. The margin pairs you specify don't even have to increase either.

For example,

```
'EXEC NEATWIND (MESSAGE' ,
               'MARGINS (1-10 20-60 30-40 5-29 1-10 60-90 6-8)'
```

is perfectly valid. Do remember though, that lines containing field marks are not cut according to the MARGINS option.

10.5 Displaced Margins.

You can also include relative displacements in the list of column ranges. +n will shift the next view forward n characters, and -n shifts it backwards.

For example,

```
'EXEC NEATWIND (MESSAGE MARGINS (1-25 +10 +10 -8 30-50 +10)'
```

The first view shows columns 1-25. The second adds a displacement of 10, thus showing columns 10-35. The next adds yet another 10, hence 20-45. The -8 moves back 8 columns, so the next view is of columns 12-37. The range after that is an absolute one of 30-50, and finally we end with 40-60.

You should specify a starting column range to work against. If you don't Neatwind will apply your changes against columns 1 to 40. A negative displacement will never take a column range past the first column in the line.

10.6 Presentation.

Each column appearing in a horizontally scrollable window should be labelled. When the user scrolls left and right, the labels should move with the columns. Other information however, such as the window title and any descriptive text should remain fixed.

The top area of the window is defined with the `KEEPTOP=n,m` option. The first value `n,` defines the number of fixed lines in this top area. These will not scroll horizontally. The optional second value `m,` defines the horizontally scrollable section. When the user presses the left or right keys, this section will scroll. Note that the corresponding `KEEPBOTT` option only has a fixed area.

For example,

```
'MAKEBUF'
 $stk = Queued()
 Queue "List of Employees &MORE."
 Queue " "
 Queue "Name          Phone   Job Title           Location"
 Queue "------------  ------  ------------------- --------"
 Queue "Greer, J.     x9431   Deputy Director (I) LG04"
 Queue "Moore, A.     x8544   Director            LG03"
 Queue "Ritter, P.    x9021   Deputy Director (O) LG06"
 Queue "Ryan, J.      x9431   Act Deputy Director LG04"
 Queue " "
 Queue "Press F3 to exit."
'EXEC NEATWIND' Queued() - $stk '(' ,
     'MESSAGE SAA87 MARGINS (1-20 23-50) KEEPTOP=2,2 KEEPBOTT=2'
'DROPBUF'
```

Here `KEEPTOP=2,2` says that there are 4 lines in the top reserved area. The first two are fixed, the second two scroll left and right. The `KEEPTOP=2` keeps the bottom two lines in the window from scrolling horizontally. This code fragment would display the following window:

```
 List of Employees     More:  >

 Name          Phone
 ------------  ------
 Greer, J.     x9431
 Moore, A.     x8544
 Ritter, P.    x9021
 Ryan, J.      x9431

 Press F3 to Exit.

 F3=Exit F9=Left F11=Right
```

By pressing the `Right` key, they would see the other view.

```
 List of Employees     More:  <

 Job Title           Location
 ------------------- --------
 Deputy Director (I) LG04
 Director            LG03
 Deputy Director (O) LG06
 Act Deputy Director LG04

 Press F3 to Exit.

 F3=Exit F9=Left F11=Right
```

To abide by CUA we recommend you include the &MORE. icon in the `KEEPTOP` area of the screen. This displays a left and right arrow, letting the user know that there is more information offscreen. If the window is vertically scrollable it will also display an up and down arrow.

10.7 The PREFIX Option.

You may have noticed in the above example that it is easy to lose the visual continuity when scrolling horizontally. For example, say we are going to look up Jack Ryan's job title. Looking at the first view our eyes lock onto the line with his name, then we press the `right` PF key. We are now looking at a list of job titles, and we think Jack's is the last one, or was it the one before?

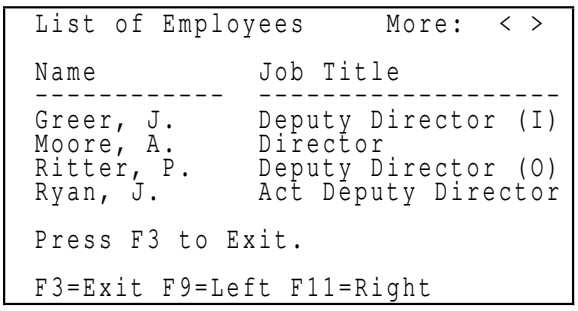
It'ld be easier if the name prefixed each field. For example, if we had three views. The first with name and phone, the second with name and job title and the third with name and location. That way we can always see what line refers to whom. We could replicate the name three times (once in each view), but this would be wasteful.

Thus the `PREFIX` option comes to the rescue. Simply set this option to the number of characters at the start of the line we want to prefix each view. Since the name field is 13 characters long, we would set `PREFIX=13`. We then divide the additional fields (phone, job title and location) into three new views. Thus we would set the following new margins `MARGINS (15-20 23-41 42-50)`.

Here is the revised code fragment:

```
'MAKEBUF'
 $stk = Queued()
 Queue "List of Employees &MORE."
 Queue " "
 Queue "Name          Phone   Job Title           Location"
 Queue "------------  ------  ------------------- --------"
 Queue "Greer, J.     x9431   Deputy Director (I) LG04"
 Queue "Moore, A.     x8544   Director            LG03"
 Queue "Ritter, P.    x9021   Deputy Director (O) LG06"
 Queue "Ryan, J.      x9431   Act Deputy Director LG04"
 Queue " "
 Queue "Press F3 to exit."
'EXEC NEATWIND' Queued() - $stk '(MESSAGE PF4=LEFT PF5=right' ,
     'MARGINS (15-20 23-41 42-50) KEEPTOP=2,2 KEEPBOTT=2 PREFIX=13'
'DROPBUF'
```

The first view (name and phone) would remain unchanged, but the by scrolling right we would see the second view now looks like this:



Note how much easier it is to match the name with the job title when they are presented alongside?

You may include one or more input or readonly fields in the prefixed area of the line. A word of caution though; if the last character in the prefix area is an field specifier (eg. "@" for input), and the first character in the view is also the same character Neatwind will insert a space. This will prevent the two input fields running together to create one (false) large one. A simple way to avoid this (it can cause some misalignment) is to always have the prefix area end with a space.

10.7.1 Use with Field Marks.

If a line contains field marks then the prefix segment will be the text before the very first mark. After that mark should follow the views delimited by more field marks. Thus:

```
prefix <FieldMark> 1st view <FieldMark> 2nd view <FieldMark> 3rd view
```

For these lines the actual value you specify in the `PREFIX` option will not apply. You must however still specify the `PREFIX` option so Neatwind knows these lines have prefix areas.

10.8 Restrictions.

Horizontal scrolling may not be used with the `TEMPLATE` option. The ability to do this may be added at a later date.

11.0 SAA 90.

The SAA87 and I/S styles are fully interchangeable. The SAA90 style is not, so we dedicate this chapter to describing its differences.

To the user the biggest differences are its colours (it uses a dull palette of dark blues and greens with the old splash of turquoise). This scheme has problems. For example, turquoise is used both for highlighted text and input fields, making the two hard to tell apart. Dark blue is very hard to read on most terminals. Red has been abandoned as an error indicator, and yellow takes its place. To most people, "Red" means stop, danger or error! Yellow means, well.... yellow.[12] The result is that people using SAA90 will need prior education, where as SAA87 or even I/S styles they could probably hop right in.

The next biggest difference is that the PF=Exit key takes on a new meaning. It no longer means "I have finished with this window and wish to continue," as it does in the other styles. It now means "I want to quit right out of the application." In effect, it has become an alternate cancel key. The difference is that cancel cancels the current window, where as Exit quits right out of the application.

Enter is the only key that is used to end a window "peacefully." To differentiate between the two we have to use two different return codes. PF3=Exit now gives a rc=9999. The Enter key gives a rc=0.

On full screen panels Enter does not leave the application. It loops back through, perhaps telling the user to pick the PF3=Exit key to leave the application.

SAA90 is the only style that gives a rc=9999 when the PF3=Exit key is pressed. It is probably a good idea to expect this code from any window, even if you are not coding with the SAA90 option. This will make it easier to upgrade in the future.

The third major difference is that the Enter key is never shown, and the PF3=Exit key isn't shown in windows, *even though these keys are valid at all times.* Users who have not been trained to use these windows will have trouble remembering a key is active when they aren't actually displayed! The justification given for this in the "Common User Access Basic Interface Design Guide" is that it "saves valuable screen space."

Agree with SAA90 or not, I have implemented it faithfully for those that want to stick by the standard. I try and avoid tainting Neatwind with my own preferences (but I didn't say anything about the document you are reading!!! :-)

In summary, if you use the SAA90 option expect rc=9999 from the PF3=Exit key. When you get this give the user a pop-up window asking them if they are sure, and if they are, exit the application.

You will also have to train your users about the new colour scheme (how for example, yellow means "bad input."), and that the PF3=Exit and Enter key are always there, and about PF3=Exit's new meaning.

---

[12] The CUA guidelines do however give you license to replace these colours by those you and your users might find more appropriate. You should however have a good reason for doing this, rather than recolouring the panel simply because you as the application designer prefer a "pink and green" colour scheme.

11.1 The PF3 key.

In `SAA90` the PF3 key always means “Exit the application.” The other styles are slightly different so we cover them here.

Under the `SAA87` and `I/S` styles the PF3 key means “I have finished this window and wish to continue.” On a message window this would return to the previous level. On a window asking for search arguments this would start the database search, for example.

This is the default action. Specifically, you cannot tell the difference on these styles between the Enter key and the PF3=END or PF3=Exit key.

If you *do* want to treat the PF3 and Enter keys differently then specify the `PF3=END=NOCHECK` or `PF3=Exit=NOCHECK` option depending on what style you are using.

This defines a customised PF key. With this defined, Neatwind will give a return code of 3 when the PF3 key is pressed, and exit without enforcing any datatype checking. The Enter key will continue to give a return code of 0. This be done on all windows except `MENU` or `QUERY` query windows, as they already use the return code to communicate their results.

12.0 Goodies.

There are a few options that we haven't discussed so far that are worth covering here.

12.1 Titling your windows.

The `TITLE` option is used to specify a title on your windows. This will be displayed at the top of your window or panel, under any action bar.

For example:

```
'EXEC NEATWIND (MESSAGE TITLE (Grummit Inventory System)'
```

If you wish to use brackets within the title you may use a single quote or double quote as alternate delimiters. You can also use highlighting or the bold or italic fonts in your title, but you must remember to reset them using the "%F" sequence. As another example:

```
'EXEC NEATWIND (MESSAGE TITLE (%eGrummit Inventory System%f)'
```

The "%e" turns on the bold font, and the "%f" sequence turns it off.

Every window or panel should have a title, and preferably a panel id. You can specify these using this option. Clearly labeling everything makes it easier for a user to find their way around the application, and for you to trouble shoot over the 'phone.

You may also use the `BTITLE` option which has the same syntax, but places the option on the top border. This option was added for people converting applications from the WINDMGR windowing system. The `TITLE` option should be used in all other cases.

12.2 The ENTEROK Option.

Under the `SAA87` and `I/S` standards the Enter key only ends non-scrollable windows. For scrollable windows the user must use the PF=END or PF=Exit key to end the window. This prevents the user prematurely quitting the window without realising there is more information.

If you're using the &MORE. indicator on your window (and thus the user realises there is indeed more text), or just want to be able to leave using Enter anyway then you can enable it by specifying the `ENTEROK` option on the command line.

`NOENTEROK` is the opposite of `ENTEROK`.

12.3 The VANISH Option.

Normally old windows drawn with Neatwind disappear when you return to that part of your application that redraws the whole screen (eg. IOS3270, ISPF, XEDIT or NEATWIND in `FULLSCREEN` mode).

If you are drawing several windows before returning the old windows will remain on the screen. You may force these old windows to disappear after they have finished by specifying the VANISH option.

However, don't fall into the trap of specifying VANISH on a NOWAIT window. If Neatwind did this then it would draw the window, which would finish immediately due to the NOWAIT option, and on the way out make the window it has just drawn VANISH straight away. All the user would see would be a brief flicker. Because of this Neatwind won't let you use these two options together.

12.4 The CONFIRM Option.

Ever typed in a screen full of data and hit the PF=Cancel key accidentally? A well-behaved application should throw up a window asking you to confirm you do want to PF=Cancel from that window, losing the changes you have made.

In Neatwind you can code this window yourself, or ask Neatwind to do it for you by specifying the CONFIRM Option. (This only applies to DATAENTRY and PICK windows). The confirm window will only appear when the user makes changes to the window and tries to cancel out.

Here is an example:

```
 f.0 = 3
 f.1 = 'Part Number: @@@@@@@'
 f.2 = 'Description: @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@'
 f.3 = 'Cost ea.:    1-10 @@@@@@@ 11-20 @@@@@@@@ 21+ @@@@@@@@@@'
'EXEC NEATWIND (SHOWSTEM F. DATAENTRY CONFIRM'
```

12.5 The XCONFIRM Option.

CONFIRM is fine for windows that are only drawn once. Take the above example. The user fills in the fields and presses Enter or Cancel, and that's it.

Now say we were to add a Help key via the F1=Help=NOCHECK option? To let the user go back and enter data after reading the help we would place the above code in a loop.

The problem with this is that CONFIRM only looks about data changed in the last call. Suppose the user types in some data and presses PF1 on the first call. They read the help, and when they come back decide to press Cancel. Because nothing was changed on the second screen call, the confirmation window will not be displayed.

This is where the XCONFIRM option comes to the rescue. It checks the neat$changed variable to see if the screen was changed in any previous call, as well as this one. Before calling the window you should set neat$changed = $.false, where $.false = 0. This will stop Neatwind from confusing the value of that variable if it were set by any previous windows.

Here is the example code. Without the XCONFIRM option you would have to check the fields yourself.

```
 $.true  = 1
 $.false = 0

 f.0 = 3
 f.1 = 'Part Number: @@@@@@@'
 f.2 = 'Description: @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@'
 f.3 = 'Cost ea.:    1-10 @@@@@@@ 11-20 @@@@@@@@ 21+ @@@@@@@@@@'

 neat$changed = $.false
 do forever
  'EXEC NEATWIND (SHOWSTEM F. DATAENTRY XCONFIRM'
   select
```

```
     when rc = 0 then leave
     when rc = 1 then call help
     otherwise call abend 'Unexpected Neatwind rc='rc
   end
 end
```

12.6 The WAIT Option.

This option tells Neatwind to display the window for a set number of seconds. For example;

```
 w.0 = 3
 w.1 = '  Welcome to the Payroll Application'
 w.2 = ''
 w.3 = ' Copyright (C) IBM Corporation, 1991.'
'EXEC NEATWIND (MESSAGE WAIT 15'
```

The `WAIT 15` will cause the window to be displayed for a maximum of 15 seconds. If the user presses a PF key or enter before that time expires then Neatwind will return normally. If however nothing has happened by the 15th second Neatwind will return, just as if the user had pressed Enter on the 14th second.

The `WAIT` option may only be used on `MESSAGE` windows.

12.7 The BEEP Option.

This option will sound the audible alarm in the user's terminal once. You can use this feature to get the user's attention. Use it sparingly though; user's whose terminals beep every 30 seconds tend to have very few friends in the workplace. ;-)

```
 f.0 = 5
 f.1 = "The best way to find out where something is"
 f.2 = "is to find out where it's not."
 f.3 = " "
 f.4 = "                                The Cat in the Hat,'
 f.5 = "                                Dr. Seuss."
'EXEC NEATWIND (MESSAGE SHOWSTEM F. BEEP'
```

Note that the beeping sound is by default disabled. The user must turn it on as described in “Warning Beep.” on page 86.

12.8 The CTLCHAR Option.

Recall that when you define fields in window text you specify an at-sign “@” for normal input, a not-sign “¬” for invisible, or a backslash “\” for read only. The field mark character, '05'x, is used to delimit views in horizontaly scrollable windows.

The `CTLCHAR` option lets you redefine these characters. Perhaps because one or more of them is appearing in your data, or because they aren't available on your country's keyboard. Specify the new characters in parenthesis after the `CTLCHAR` option. The first character defines the input fields, the second invisible and the third read-only. The fourth character redefines the field mark character. If any of these characters are omitted or left blank then they will assume the above defaults.

For example,

```
'EXEC NEATWIND (CTLCHARS (A C)'
```

defines A as the input field character. The blank that comes next means the invisible field character assumes its default, which is the not-sign "¬." Finally, the C becomes the read-only field character. Aside from the fact the characters have been changed, all the other rules of usage remain the same. Since we have ommited the fourth character it will remain unchanged, keeping its default value.

12.9 Using your own fonts.

When you specify PSLOAD you are actually instructing Neatwind to download two fonts; a bold one and an italic one. The graphical icons are also included in these fonts. Saying PSLOAD by itself is actually the same as saying PSLOAD=NEATBOLD,NEATITAL. This refers to the two PSET files included in the Neatwind package.

You may use the PSLOAD option to download your own fonts. These should contained in files in PSET format. The filename before the comma is that of the bold font. The one after is that of the italic font. You may omit either.

For example, say you have a file called ADMITALC PSET you wish to download and use as the bold font. You would specify

```
'MAKEBUF'
 Queue '%eHi there...%f'
'EXEC NEATWIND 1 (MESSAGE PSLOAD=ADMITALC'
'DROPBUF'
```

Be aware that while you are using your own font the bold and italic fonts will not be available. Also be aware that for this duration textual icons will be used instead of graphic ones. People who have terminals that don't support programmable symbol sets will not see any difference.

13.0 National Language Support.

You may skip this chapter if you are doing an English-only application, which will not need to be converted for international use at a later date.

The trouble with writing a good application is people start lining up at your door asking you to port it to their system. This is easy enough when you speak the same language, but when you speak different tongues things start getting complicated. With that in mind, Neatwind is designed to make this transition as painless as reasonably possible. The `NLS` option tells Neatwind to start speaking in the specified language; error messages, PF keys and most action bar functions.

Converting your actual application is somewhat more difficult. Attempts to use computers for automatic language translation started in the 1950's, and have continued ever since. The results to date have been disappointing. Language which comes naturally to humans is too complex and ambiguous for our silicon friends. The translation they produce for all but the simplest text is more often that not complete nonsense.

Because of this, it will be up to you and your fellow humans to translate the application windows, panels and messages. Neatwind does however provide a way for you to conveniently lump all these into a single panel file. When it comes time to convert to another language, you simply run through this file and convert it into the new language. Using the `PANEL` option you can direct Neatwind to simply switch to the appropriate panel file depending on which language your customer wishes to use.

Finally, you must get your keyboard to start talking in the desired language too. It may come as a surprise to most people, but even a U.S. terminal can display the French alphabet (for example), with a bit of coaxing. The `CODEPAGE` and `NOCODEPAGE` option handle this.

This chapter discusses these activities; asking both Neatwind, your terminal and application to talk in another tongue.

13.1 Changing your terminal's tongue.

Most European languages are based on the Latin alphabet, as used by the ancient Romans. This alphabet holds 26 letters, ranging from A through to Z. Each letter has an capital and lower case form.

English is the only language that has stuck with this original alphabet. Most other European languages have included variations of these characters, and in some cases totally new ones. Example of diacritic variations are the “ö” character used in German and Nordic languages, or the “é” used in French. The German “ß” is an example of a totally new character.

Keyboards are indvidually designed for different languages. What may surprise most people is that all screens hold the Latin characters. These globally available characters are known as the “syntactic character set.” They include the letters A-Z, a-z, digits 0-9 and various punctuation characters.

Many screens also include the Latin diacritic and new characters. The latter group are known as having a common character set known as “Latin 1.” U.S. and U.K. English, Spanish, French and German are among some of the screens with a “Latin 1” character set.[13]

There are also “non-Latin 1” character sets used by Middle Eastern and Asian countries. These still include the syntactic character set (basic Latin alphabet), but replace the Latin diacritics with their own local characters. There isn't room for both.

Thus, you can display any Latin based tongue on any other “Latin 1” terminal in the world. Aside from the obvious (such as displaying French on a French terminal), you can display French or a German, English, Spanish and even a Japanese Kanji terminal! On non-Latin 1 terminals (such as the Hebrew one) you can approximate the Latin diacritics should the need arise. Of course, as Latin 1 terminals have no characters approximating Hebrew the reverse is not true. At least, not easily.

If you want your application to run in or exchange data between different countries then you may use this to your advantage.

Firstly, we shall address the concept of a “codepage.”

13.1.1 Codepages.

Even though a common set of characters is available in all “Latin 1” codepages, they do not share the same position within, or “codepoint.” Take the “ö” character for example. In the Swedish “codepage” this character occupies codepoint '6A'x (hex). In the U.S. codepage the split vertical bar occupies '6A'x, and “ö” has been moved to codepoint 'CC'x.[14]

Now say Sven from Sweden decides to send a note to his friend Ingemar who is on assignment in the United States. If Sven types the word “popupfönster” in his note, it will read on Ingemar's U.S. terminal as “popupf|nster.” This is because Sven's terminal is using the Swedish/Finnish codepage while Ingemar's uses the U.S. one.

Each codepage has a unique 5 digit number which identifies it. The U.S. English codepage is number 00037. The Swedish/Finnish codepage is number 00278. A list of codepages is given in “Appendix C. Codepages” on page 102.

You can see the number of your terminal's codepage and a table of all characters therein by running the program `NEATCHAR`, which is included in the Neatwind Package. Refer to “Appendix B. Terminology of Codepages.” on page 100 if you are having trouble understanding the terms we have used here.

13.1.2 NLS Character Representation.

Neatwind advocates a codepage independent NLS character representation. These escapes can be placed anywhere within the Neatwind message file (dicussed later), the window text or various options. Using these escapes allows you to exchange files between different countries without fear of garbling the results.

Neatwind will look for and translate these escapes into their local equivalents. This is only done when a non-English language is used, or you explicitly specify the `CODEPAGE` option. For example, to display “popupfönster” in a window we would code it as follows:

---

[13] In practice there are minor differences in the implementation of the character set. The currency symbol is a good example of this.

[14] It is accepted this is a rather unwise arrangement. It would have been far simpler to have the same characters at the same codepoints in all “Latin 1” character sets. This problem has however been recognised and an international ISO codepage is in the works. For the moment however, we must make the most out of what we've got.

```
 Queue "popupf#donster"
'EXEC NEATWIND (NLS SVE'
```

The `#do` is the escape sequence. The hash `#` is the NLS escape character. The `do` refer to the character we want to produce. Specifically, the `d` means "double dots", as in umlaut or dieresis. The `o` refers to the letter "o," thus we have the umlaut o; "ö."

Here is a table of all the available NLS escape sequences:

ca	â	da	ä	ba	à	fa	á	ta	ã	pa	å
tc	ç	tn	ñ	fE	é	cE	ê	dE	ë	bE	è
fe	é	ce	ê	de	ë	be	è	fI	í	cI	î
dI	ï	bI	ì	fi	í	ci	î	di	ï	bi	ì
cA	Â	dA	Ä	bA	À	fA	Á	tA	Ã	pA	Å
tC	Ç	tN	Ñ	co	ô	do	ö	bo	ò	fo	ó
to	õ	cu	û	du	ü	bu	ù	fu	ú	dy	ÿ
cO	Ô	dO	Ö	bO	Ò	fO	Ó	tO	Õ	cU	Û
dU	Ü	bU	Ù	fU	Ú	so	ø	sO	Ø	_a	a
_o	o	_ex	¡	_?	¿	fY	Y	fy	y	'	'
ex	!	ae	æ	AE	Æ	oe	oe	OE	OE	B	ß
iT	P	it	p	E	Ð	e	⊕				

Some of characters shown here are approximations. For example, `#fE` will show a capital accented "E" where available. The last line includes approximations of the Icelandic thorn and E characters.

As a guideline, in the two and three letter sequences the first character defines an effect on those following. These first letters are:

Prefix Character	**Meaning**
c	circumflex, or carat.
d	double dots, umlaut or dieresis.
b	back accent, or accent grave.
f	forward accent, or accent acute.
t	tilde, twidle or cycle. Also used for cedilla or n caron.
p	point, period or overcircle.
_	underbar, underscore, or "upside down" for "!" and "?."
s	slash for o and O (as used in Nordic languages).

There are exceptions to this rule, such as the sequence `#ae` meaning the dipthong "æ," or `#ex` meaning exclamation mark "!."

13.1.3 The NEATCODE file.

When Neatwind is asked to speak in a non-English language it will find out what the codepage of the terminal is, and look for a `NEATCODE` file bearing that codepage as its filename. For example the standard U.S. terminal has codepage number 00037. Neatwind would thus look for a file named `00037 NEATCODE.`

Some older terminals do not know what codepage they are running, and are thus not able to tell Neatwind. Whenever Neatwind encounters one of these terminals it will will try and guess the codepage from the language specified in the `NLS` option. There is a chance Neatwind will guess incorrectly. Say you are running Swedish on a U.S. terminal that doesn't know its codepage. Neatwind would look at the language, see it is Swedish, and figure (incorrectly) you are probably running a Swedish terminal.

You can specify your own default in this case via the `CODEPAGE ASSUME` option. Thus,

```
'EXEC NEATWIND (MESSAGE CODEPAGE ASSUME 000278'
```

tells Neatwind to assume codepage 000278 instead of trying to guess. If you omit the `ASSUME` keyword, then Neatwind will forcibly use the codepage you specify regardless of whether the terminal knows its own codepage or not.

The `NEATCODE` file selected will contain a list off all of the above NLS escape sequences alongside their local equivalents. When Neatwind is asked to print one of these sequences it will use the value specified in this file.

13.1.3.1 Simulating diacritic characters.

By specifying `CODEPAGE ASSUME SIMULATE` you are telling Neatwind that if the terminal doesn't know its own codepage to only assume it has *no* diacritic or national characters.

Neatwind will simulate them using the Latin alphabet. For example, the umlaut o character “ö” would simply appear as “o.” Cedilia “Ç” would appear as “C.” Be warned; some languages are intolerant to this. In French for example, dropping accents can drastically change the meaning of a word.

However, some languages can take this sort of abuse. If your user population do speak such a language, are used to Latin for diactric subsitution, and there are may terminals around that don't have any diacritic characters anyway, then this is a convenient and practical compromise.

Note that we say `CODEPAGE ASSUME SIMULATE` rather than `CODEPAGE SIMULATE.` If we were to use the latter, then Neatwind would not even attempt to find out codepage information from the terminal. This could result in unnecessary Latin for diacrtric substiution. You are better off using the former, as the substiution may not be necessary anyway.

13.1.3.2 Do I need to create a NEATCODE file?

Hopefully someone has already done that for you. Type the following command from CMS:

```
NEATCHAR
```

This is a program included with the Neatwind Package. It will find out the codepage of your terminal (most terminals in your country will share the same codepage), and display that number along with a table of that codepage's characters in a window. If your terminal lacks codepage information then we recommend you run it on a more advanced terminal or a PS/2.

`NEATCHAR` will also look for a `NEATCODE` file for this codepage. If it gives the message that a `NEATCODE` file is found, then all is ok. You may skip the rest of this section and go straight to “Field Specification Characters.” on page 73. On the other hand, if this file is not found then you must create it. Don't panic; it isn't that difficult.

13.1.3.3 Creating the NEATCODE file.

Say we are creating a file to support the Swedish/Finnish codepage. If we ran Neatchar on a Swedish terminal it would have told us we are using codepage number 00278.

We take the U.S. codepage file `00037 NEATCODE`, which is included supplied with the Neatwind package, and make a copy of it with the new codepage number. Thus `00278 NEATCODE`. Once done, we shall work our way through this new file making the necessary changes. If you look through the it you'll see the layout is pretty simple. Ignore anything to the right of a star "*" or double-dash symbol "--." These are comments.

On each data line you'll find the escape sequence (eg. `ca`), followed by a colon and the local character code. The latter may be specified simply by typing it as it appears on your keyboard (if available), or in the standard REXX format for hexadecimal numbers (eg. '4C'x). The escape sequence may translate into more than one character, if necessary.

Work your way through the file. Let's take the "ö" character for example. Since we copied the U.S. codepage file, the line for that character would read:

```
do:  'CC'x
```

Remember the `d` means double dots. That is, dieresis or umlaut. The value currently reads 'CC'x, as that is the hex code for the "ö" character on the English keyboard. Since we are configuring Neatwind for a Swedish Keyboard, we use the Swedish character code for the "ö" which is '6A'x. Thus we would replace the line with:

```
do:  '6A'x
```

Alternately, if you if the "ö" character is available on your keyboard (your keyboard having the same codepage as the `NEATCODE` file you are creating), then you may simply enter it as is. Thus:

```
do:  ö
```

You must translate all characters, even if they are not directly accessible from your keyboard. For these you should enter the hex code of target character within your codepage. Remember you can run the `NEATCHAR` program to see the table of characters in your codepage. You may obtain the hex values from there. If a character isn't available, you may construct an approximate alternative. For example, if the ae dipthong "æ" isn't available you could just approximate it by typing an "a" followed by an "e." Thus:

```
ae:    ae
```

13.1.4 Going it alone...

Asian languages are notorious for their large alphabets. Your average 3270 terminal has room for 192 characters. In that 192 characters it is able to fit the Roman alphabet and its derivatives. There is simply no room to include letters from these more exotic languages. For that reason, we do not provide NLS escape characters for the letters of those alphabets.

Middle Eastern tongues by comparison have small alphabets not much larger than the Latin one. Unfortunately there are too many languages and characters to fit into the same codepage, so they each have their own. Since they are so varied it isn't possible to emulate them on non-Middle Eastern terminals. Thus we do not provide NLS escapes for these alphabets either. Middle Eastern languages also write right-to-left across the page. Neatwind currently makes no special concessions for this either.

If you are using one of these exotic languages then we suggest you type everything straight from the keyboard. Do not bother to make any use of NLS escape sequences. At the expense of portability (which is lost due to the different character set anyway), we will turn this feature off.

To disable the NLS escape sequence conversion specify the `NOCODEPAGE` option.

For example, suppose you want to run a Kanjii program on a Kanjii terminal. Assuming that JPN is the Japanese (Kanjii) National Language Code, you would specify:

```
'EXEC NEATWIND ... (NLS JPN NOCODEPAGE'
```

In spite of that, you still should create a NEATCODE file, because it will allow Latin based languages to be run off the Kanjii terminal. For example,

```
'EXEC NEATWIND (NLS FRA'
```

tells Neatwind to display its internal messages in French, but would work just as well on a Kanjii terminal as a French one!

13.1.5 Field Specification Characters.

Neatwind uses the characters "@," "\" and "¬" to specify normal input, read only and invisible input fields. Due to the difference in codepages, these may not be the same on non-English terminals.

These characters occupy codepoints '7C'x, 'E0'x and '5F'x in your codepage respectively. You can see what they are under your codepage by printing off these three values in a REXX Say statement.

Alternately, run the NEATCHAR program included with the Neatwind package, and press the F5=FieldSpec key. This will display a window showing how they appear locally.

To allow a high level of compatibility between countries and to make panel conversion easier, we suggest you use these characters as is. Even if they take on the value of a local multilingual character, there will be no clash. This is because you should be using the NLS escape sequences to display the multilinguals. Neatwind will resolve the field specifications before it converts the NLS escape sequences to their local equivalents, thus avoiding any conflict between the two.

Alternately, if these characters are not available on your keyboard you can override them with CTLCHAR option discussed in "The CTLCHAR Option." on page 66. If you choose to do this we recommend you substitute them for a local character. Steer clear of the Roman alphabet, the digits, or any other characters likely to be used by someone doing a conversion (such as a dash, colon or period).

13.1.5.1 The NLS Escape Character.

The NLS Escape character occupies codepoint '7B'x in your codepage. It appears in the U.S. codepage as a hash, "#." You can see what it is on your keyboard by running NEATCHAR and pressing the F5=FieldSpec key. This character cannot be changed.

13.2 Changing Neatwind's Tongue.

Each national language in the IBM World is identified by a three-letter code. For example, the code for U.K. English is ENG. You may ask Neatwind to speak in another language by passing it this code via the NLS option. For example,

```
'EXEC NEATWIND (NLS ENG ...'
```

Whenever this option is passed, Neatwind will print all its own messages, PF key definitions and most action bar functions in the target language. Internal error messages (such as running out of memory), will be in English regardless of this setting. You can see a list of all the valid language codes in "Appendix A. National Language Codes." on page 97.

What the `NLS ENG` option does is tell Neatwind to look for a file called `ENG NEATNLS`. This file will contain pretranslated phrases of all the messages and keywords used by Neatwind. Whenever Neatwind needs to know a translation, it looks it up in this file.

If you want Neatwind to talk in a certain language (say Swedish), then you should acquire the Swedish version of the `NEATNLS` file. As Sweden's Language Code is `SVE`, the file would be called `SVE NEATNLS`.

Some of these files are supplied with the Neatwind package. If your target language does not have a `NEATNLS` file available, then you must create it yourself. Don't get too worried though; This line is a couple of hundred lines long, so converting it won't require too much effort. Also, once you have finished send a copy to the Neatwind author. He'll include it with the next release of the package, thus saving others from having to recreate your file.

13.2.1 Creating the NEATNLS file.

Lets create the Swedish National Language file in this example. We take the `ENG NEATNLS` supplied with the Neatwind package, and simply copy it as `SVE NEATNLS`. Once done we work our way through the file, converting the phrases from English to Swedish.

13.2.1.1 Translating the Messages.

If you look at the layout of the `NEATNLS` file you'll see it is fairly straight forward. There are comments placed throughout the file. These are lines that have a star "*" or two dashes placed therein, "--." Ignore everything to the right of those symbols.

Each data line is made up of three parts. An attribute, followed by a colon, and value. For example:

```
60   :  Save As
```

All you have to do is translate the value (to the right of the colon) into the new language. Since "Save As" translated to Swedish is "lagra som", the new line would read:

```
60   :  lagra som
```

If the translated phrase contains any multilingual characters (such as å or ö), then insert those using the NLS escape sequences. For example,

```
82   :    bak#pat
```

Keep the messages short and to the point. Keywords (such as File) don't have a length restriction, though if you get too wordy there may not be enough space on the action bar or PF key line to fit them all.

Never ever change the attribute value to the left of the colon. Even if it is inappropriate to your language, leave it as is. Only change the value part of the line. Remember that this file only changes the displayed appearance of Neatwind. The programming interface remains in English.

Work your way through the file changing the messages and keywords as you go.

13.2.1.2 Message Width.

English is a reasonably terse language. Most other languages tend to use a lot more words and a lot longer ones at that. For example, the German translation of "message pop-up" is "Nachrichtenüberlagerungsfenster." As another example, the French translation of "save as..." is "sauvegarder en..."

Neatwind sets a minimum window width. This minimum width allows a window to hold the widest message that may appear, without truncation. The next section of the NEATNLS file is used to define these widths:

```
MINWIDTH.1:  38  * PICK, or DATAENTRY windows using eg. 1..20 TYPE ranges
MINWIDTH.2:  32  * All other sorts of windows, except nonscrollable MESSAGE
```

The first parameter sets the minimum window width for windows of type PICK, or DATAENTRY windows using the integer or real subrange types. The second width parameter applies to all other types of window. Non-scrollable message windows are exempt from these widths, as they do not display anything on the message line.

The widths you set here do not have to take into account the PF key names or action bar keywords. Neatwind will automatically increase the size of the windows for these.

One thing to remember; the widths you specify here will be the minimum width for most type of windows. It could look a little ugly if your minimum window is 70 characters across, so do try and keep the messages nice and terse. That'll in turn let you specify a smaller width here.

13.2.1.3 Translating the PF keys.

The last part of the file is used to name the PF keys under the new National Language. Here is that fragment of the file:

```
END:            Exit           END
BWD:            Bkwd           BWD
FWD:            Fwd            FWD
CANCEL:         Cancel         CANCEL
HELP:           Help           HELP
ACTION:         Action         ACTION
PROMPT:         Prompt         PROMPT
REFRESH:        Refresh        REFRESH
MARK:           Mark           MARK
LEFT:           Left           LEFT
RIGHT:          Right          RIGHT
UNDO:           Undo           UNDO
UNMARK:         Unmark         UNMARK
COMMAND:        Command        COMMAND
RETRIEVE:       Retreive       RETRIEVE
DISPLAY_KEYS:   Display_Keys   DISPLAY_KEYS
KEYS_HELP:      Keys_Help      KEYS_HELP
HELP_INDEX:     Help_Index     HELP_INDEX
TUTORIAL:       Tutorial       TUTORIAL
EXTENDED_HELP:  Extended_Help  EXTENDED_HELP
```

The attribute on each line (to the left of the colon) describes the function of the PF key. END ends a window and continues (under SAA90 it exits straight out of the application). BWD and FWD are used to scroll backwards and forwards through scrollable windows. CANCEL leaves the window, throwing away any changes that may have been made. HELP displays up the application help text. Finally, ACTION flips the cursor between the window's data and action bar areas. The remainder are SAA standard keys. They are not directly controlled by Neatwind, but we include them here so they may be automatically translated when converting between national languages.

To the right of the colon are the two names of this PF key. The first is for the SAA standard. The second is for the older I/S standard. In English, the latter just happens to be the same as the attribute. To take our Swedish example, we might translate the first line as follows:

```
END :     avsluta  st#dang
```

Do not include any spaces when you define these names. Rather, use the underscore character instead. Take the Norwegian example:

```
END:    g#pa_ut slutten
```

13.2.1.4 The Version Number.

At the very end of the file is a version number. For example:

```
NLSVERSION  :  1
```

This is checked by Neatwind to make sure the message numbers in the `NEATNLS` are correct for this release. If it is not Neatwind will abend with an error, asking that the file be updated.

Do not alter or change this number, unless you are specifically updating the `NEATNLS` file for a new release. If you change this number indiscriminately Neatwind will start talking nonsense.

13.2.1.5 Into action.

If you create a new `NEATNLS` or `NEATCODE` file, once you've satisified its been sufficiently tested then send a copy to the Neatwind author. In the future we'll try and bundle all the National Language and codepage support files into the Neatwind package. That'll save other people needlessly repeating the translation work you have just performed.

13.2.2 Action Bar Functions.

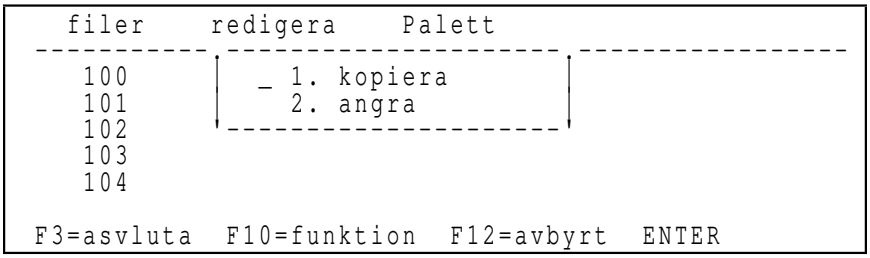
Neatwind will do its best to translate action bar function keywords and their pull down menus into the target national language. If Neatwind recognises any English keywords defined on the action bar or within a pull down menu it will automatically translate them. `File, Edit, Help, Print, Search, View` and `Options.` are among the words recognised. So are all those used on `STD` pull down menus. Any keywords not recognised is left “as is.”

The advantage of this is obvious. It means you can code these is English, regardless of the National Language you are working in. When it comes time to convert, you will not have to change any of these standard keywords appearing in the code.

Here is an example code fragment:

```
 do i = 100 to 104
   Queue i
 end i
'EXEC NEATWIND (SAA87 NLS SVE' ,
     'ACTION (File=STD Edit (1. copy=N@2. #pangra=N) Palett)' ...
```

This is what the window would look like:



13.3 Changing your application's tongue.

We now know how to ask Neatwind to speak in the target language. But what about our application? If it will only ever run under one language, that is simple enough. We simply write all the screens in that language and that language only.

But what if it will run under several languages? If we have to edit the actual program code to do the conversion it could be costly and error prone. This chapter discusses a simple mechanism to allow you to switch between target languages.

In all the examples we've seen previously, the application has passed Neatwind the screen definition using either REXX stem variables or the program stack. Neatwind provides an alternative for people writing applications that need to support two or more national languages. For these it is much more convenient to place your window text in a Neatwind Panel file. You can ask Neatwind to display panels or windows defined in that file.

When it comes time to convert to another language all you have to do is make a copy of that file, give it a new name and convert the text therein.

There is no reason why people doing non-NLS applications could not use Neatwind Panels either.

Panels should not be nested. Nor should they be generated *within* your application. That defeats the whole purpose of them. Should you have a variable length window you should use the `SHOWSTEM` or program stack to pass the window text, and rely on a traditional message file to provide any descriptive text.

13.3.1 Defining your Panels File.

Panel files are pretty rudimentary. The different panels and their components are divided up inside the file using tags. Each panel starts with a name tag. For example, a panel named “mainscreen” would begin with the tag `:MAINSCREEN.` The closing period is optional. Optionally this may be followed by one or more lines of Neatwind options (such as `FOREGROUND, FULLSCREEN` or `QUERY.`) The window text component starts with the `:BEGIN.` tag, and ends with the `:END.` tag.

Here is an example file called CUSP$ENU NEATPANL:

```
:SENT_FILE.
saa87 foreground=white message
:BEGIN.
&INFO.Your submission has been sent.

You'll receive a profs note from
the bids group after your submission
has been studied.
:END.
:DELFILE.
saa87 query
:BEGIN.
&WARN.Are you sure you want to
&WARNF.delete this submission?
:END.
```

You may include as many definitions in the Neatwind Panel file as you like. If the file gets too large it may be a good idea to split them should performance start to degrade.

You can give the file whatever name you like, though it would be advisable to incorporate your application name and three-character national language code into them. We have chosen ENU for our above example, which is U.S. “English.”

13.3.2 Displaying the panel.

To display the panel call Neatwind with the `PANEL` option. This option has one argument, which is the NEATPANL filename, followed by a period and the panel you wish to display. For example:

```
'EXEC NEATWIND (SAA87 PANEL CUSP$ENU.DELFILE FOREGROUND=BLUE'
```

The options in the NEATPANL file are substituted into the Neatwind options at that point. Note how it is possible to override any options in this file by specifiying that same option to the right.

13.3.3 Substitutions.

Panels lack the flexibility of windows created "on the fly." To comphensate for this, two new tokens are available for the exclusive use of panel definitions.

The first is the "`&PARMn.`" token. When you include this token within the panel's window text, Neatwind will look for a REXX variable of the same name in the calling program and substitute that variable's value in its place.

The second is the "`&STACK.`" token. This must be on a line by itself. This token will substitute itself for anything you had queued on the stack when you called Neatwind. Note the periods following the `PARM` and `STACK` tokens. If you omit these periods the panel will not work.

For example, say we create a panel file called `SAMPLE NEATPANL`.

```
:COLOURDEMO.
MESSAGE
:BEGIN.
&PARM1.This line is highlighted.%F
This isn't, but&PARM1.this is instead.%F

Here is a multiplication table:
&STACK.
End of multiplication table.
:END.
```

We can call this panel with the following code fragment:

```
/**/
 parm1 = '%H'    /* highlighting sequence */

'MAKEBUF'
 do i = 1 to 10  /* put something on the stack */
   queue i '* 5 =' (i * 5)
 end i

'EXEC NEATWIND (PANEL SAMPLE.COLOURDEMO'
'DROPBUF'
```

13.4 Integration.

In order to integrate National Language Support into our application we have to set two options. The `NLS` option defines what language Neatwind's own messages shall be in, and the `PANEL` option allows us to switch between different application panel files. Most likely, we would have one of these for each language.

Later on we shall discuss Neatwind's control panel. This allows the user to specify up to five national language preferences. For the moment, suffice to say there is a program included in the

Neatwind package that queries the control panel settings. The function call `NEATQRY('NLS')` will return a list of up to five national language codes, in order of preference.

We would scan this list from left to right and choose the first language we find that the application supports. If the list of preferences is empty, or we don't find a language the application supports then we would assume a default. In the latter case it would be a good idea to ask the user if if they are willing to use the default language.

```
/*
 * List of language codes the 'CUSP' application supports.
 * Eg. CUSP$ENU NEATPANL holds the U.S. English text, etc.
 *     CUSP$DEU NEATPANL the German and so on...
 */
supported = 'ENU ENG FRA DEU'

/* Get a list of languages in order of preference (eg. FRB FRA ENG) */
nlslist = neatqry('NLS')

if nlslist = ''                    /* Do they have any preferences?   */
then nls = 'ENG'                   /* No, so assume English           */
then do
  /* Find a supported language in the preference list */
  found = $.false                  /* Found a supported NLS code?     */
  do while nlslist ¬= ''
    Parse Var nlslist nls nlslist  /* Get the next code...            */
    if find(supported, nls) > 0    /* If application supports it...   */
    then do
      found = $.true               /* Yes! we got a match!            */
      leave                        /* No need to look any further...  */
    end
  end

  /* Handle a no match situation... */
  if ¬found
  then nls = 'ENG'  /* Choose a default, perhaps ask them to confirm? */
end

/*
 * Choose the selected NLS option for internal Neatwind messages.
 * Windows and panels have been isolated into eg. CUSP$xxx panel.
 */
$.langopt = 'NLS' nls 'PANEL CUSP$'nls'.'   /* append panel name here */

/* Bring up the opening logo window, and sound off a beep... */
'EXEC NEATWIND (' $.langopt||'LOGO BEEP'

/* Next the main menu, and so on... */
'EXEC NEATWIND (' $.langopt||'MAIN_MENU'
```

Once we have selected a language note how we set up the `NLS` and `PANEL` options? All we have to do is append the panel name to the `$.langopt` variable, along with any other options specific and we're in business!

13.5 Non-Latin Language Support.

13.5.1 DBCS Languages.

Neatwind has been tested on DBCS (Asian) terminals. We found when a window was overload on a screen already containing DBCS strings those (already present) strings would garble. It seems that either a fault in IOS3270 or in the 3270 architecture hardware wipes the SHIFT IN/OUT characters used to delimit DBCS strings. We got around this problem by forcing a screen clear before a window is drawn. Also, the `BACKGROUND` option has no effect on DBCS terminals. It is always set to green.

If later versions of IOS3270 fix these problems we shall amend Neatwind. Meanwhile, note that this is only an aesthic problem and in all other ways Neatwind is in perfect working order.

Neatwind works and is fully functional in the DBCS environment.

13.5.2 Middle Eastern Languages.

I do not have access to any bidirectional (Middle Eastern) terminals, nor do I know anyone who does. Thus I have not been able to test Neatwind in this environments.

In theory Neatwind should be capable of supporting Arabic and Hebrew languages, but may require some minor adjustments. If anyone does wish to use Neatwind in such an environment then please contact the Neatwind package author.

13.6 Finding out more.

If you want to find out more about National Language Support then check out the **IBMNLS** Tools Conference Disk. Among other things it contains a number of excellent introductory computer based tutorials. These are named `NLSCBT1` through to `NLSCBT6 PACKAGE`.

You can also refer to the following manuals:

- SE09-8001 NLS Volume 1. Enabled Products Rule Guide.
- SE09-8002 NLS Volume 2. Reference Manual.
- SE09-8003 NLS Volume 3. Arabic Script.
- SE09-8004 NLS Volume 4. Hebrew.

14.0 Making Neatwind faster.

There is a compiled version of the NEATWIND EXEC distributed with the package. The function of this compiled MODULE is identical to the EXEC version, but runs significantly faster.

In order to use the MODULE version you *must* have the runtime libraries distributed with the CMS REXX Compiler (program number 5664-390). These must be placed on a shared disk where all the users running Neatwind will have access to them. If you do not have these you cannot run the compiled version.

If you do have the runtime library then we strongly make use of the MODULE version of Neatwind. It is much faster than the EXEC.

14.1 The NUCXLOAD.

At the start of your application you *must* `NUCXLOAD` the NEATWIND MODULE. *This isn't as difficult as it sounds!* You do this simply by specifying the following command in the applications initialisation sequence.

```
'NUCXLOAD NEATWIND'
 if rc ¬= 0 & rc ¬= 1   /* if not loaded and not already there */
 then do
   Say 'Error: Unable to NUCXLOAD NEATWIND. rc='rc
   Exit 999
 end
```

This loads the NEATWIND MODULE out of harms way, so any other modules you call during your program won't stomp all over it. If you get a protection exception or something similar whilst running your application, you probably forgot to `NUCXLOAD NEATWIND.` You can test that hypothesis by changing your code to call the EXEC instead of the MODULE, and see if that gets rid of the protection exception.

14.2 Calling compiled Neatwind.

Everything from then on is the same as our earlier examples, *except* you must call it by saying

```
'NEATWIND ...'
```

instead of

```
'EXEC NEATWIND ...'
```

That is, omit the `EXEC` keyword. Everything else is identical.

14.3 The NUCXDROP.

If you're conscientious you'll do a `NUCXDROP NEATWIND` on leaving your application. This frees the memory Neatwind took up, which is useful if the user is running on a small virtual machine.

14.4 Detecting the runtime library.

You may not know in advance whether or not the node on which you plan to run your application has the REXXCOMP runtime libraries installed. There is however an easy way to check, and enable either the MODULE or EXEC accordingly.

`EAGRTLIB MODULE` is included in the REXXCOMP runtime library. The following code fragment issues a `STATE` command to see if that file is present. If it is then we know we can use the faster MODULE version of Neatwind. If we don't find it we shall use the EXEC version instead.

```
'STATE EAGRTLIB MODULE'          /* Is the runtime library there?     */
 if rc = 0
 then do
   $.neat = 'NEATWIND'           /* Yes, so use the MODULE            */
  'NUCXLOAD NEATWIND'            /* Load the MODULE as a nucleus ext. */
   if rc ¬= 0 & rc ¬= 1          /* If not loaded...                  */
   then do
     Say 'Unable to NUCXLOAD NEATWIND MODULE. rc='rc
     Exit 8
   end
 end
 else $.neat = 'EXEC NEATWIND'  /* No, so use the EXEC               */

 /* Call Neatwind like this: */
 f.0 = 2
 f.1 = 'Hi there,'
 f.2 = 'How are you?'
 $.neat '(MESSAGE SHOWSTEM F.'
```

15.0 Return Codes.

Neatwind gives a number of return codes. These depend on the options Neatwind is passed, so you'll only ever have to cater to a subset of these return codes. For example, a window without an Action Bar will never give the action bar return code.

The Neatwind return codes are as follows:

- rc = 0

  User pressed a key to end the window and continue. This Return code is only given by `MESSAGE, DATAENTRY` and `PICK` windows.

- rc = -10

  User pressed PF=Cancel (and `HIRETURN` option was not specified).

- rc = 10000

  User pressed PF=Cancel (and `HIRETURN` option *was* specified).

- rc = 9999

  User pressed PF=Exit key meaning quit right out of application. This return code is only given when the `SAA90` option is used.

- rc = 9998

  An action bar function was selected. The variables ACTION$TYPE, ACTION$OPT and ACTION$NAME describe what the user selected.

- 1 < = rc < = 12

  If customised PF keys were specified, then this is the number of the customised PF key the user has just pressed. If that key had the HELP attribute specified then the HELP$NUMBER, HELP$MAXFIELDS and HELP$NAME variables are set. You do not have to handle the action, forward or backward PF keys. Neatwind handles these internally.

- 0 < = rc < = 9000

  If a MENU window then this is the selected menu choice.

- rc = 0, rc = 1

  If a QUERY window then this means no and yes respectively.

- rc < = -30

  An Internal Neatwind Error. This is type of error results from invalid options passed to Neatwind, or an installation error.

- rc > = 20000

  A REXX Error occurred while running Neatwind. Report this error in the Neatwind Forum.

16.0 Customising Neatwind.

Colours are a very subjective thing. While you might consider pink panels with green and red highlighting to be visually appealing, others might consider your tastes rather bizarre. “Strange people” you think, adjusting your bright yellow shirt and puce slacks.

Most applications don't allow for personal taste, other than that of its authors. CUA recognised this, and stressed that users should be allowed to tailor their application colours. This is in line with the principle of putting the user in control of the application, and not the other way around.

Another personal preference is the beep generated by some applications to indicate an error, or which National Language a user prefers to use.

Neatwind provides a “control panel,” suitable for use by end-users. Indeed, you may choose to directly wire this control panel into your application to let your users express their own personal preferences.

This chapter discusses the Neatwind control panel, and how your application should accommodate it.

16.1 The Control Panel.

An end-user may invoke the Neatwind Control Panel by typing the CMS command

```
NEATCONT
```

This is the screen it produces:

16.1.1 The Control Panel Menu.

```
Neatwind Control Panel


_  1. Screen colours.
   2. Warning Beep.
   3. National Language.
   4. Country.


Type in the number of the option you wish to change, and Enter.


F3=Exit F12=Cancel
```

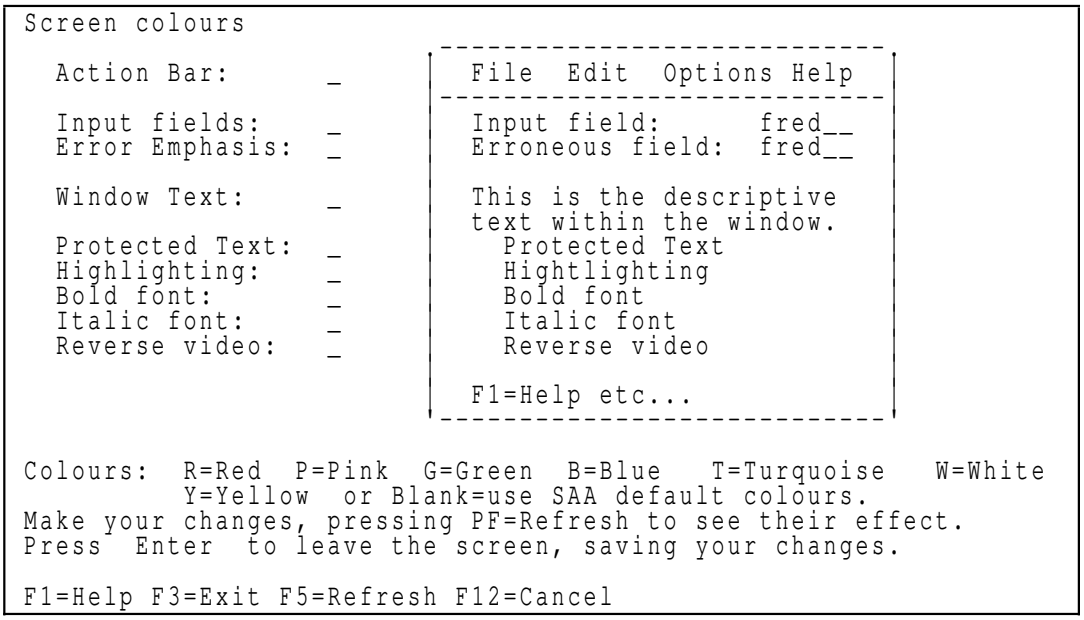
Selecting option `1` allows the user to tailor the colours of windows and fullscreen panels produced by Neatwind. Option `2` lets them turn the “warning beep” on or off, and option `3` lets them select their national language preferences. For example, English. Option `4` lets the user define what country they are working in.

The Neatwind Control Panel complies with the SAA90 entry-level standard. If your users are unfamiliar with this standard you may have to brief them on the differences between SAA90 and the mixed brew of dialogue managers they may be used to using. See "SAA 90." on page 62.

Basically though, the F3=Exit key exits straight out of the Control Panel (regardless of how many window levels you are down). The Enter key saves any changes you have made and ends a window. The F12=Cancel key ends the window, but doesn't save any changes.

Let's look at option 1.

16.1.2 Screen Colours.



This screen is divided into two halves. On the left are input fields where you may enter the one-letter abbreviation of the colours you wish to use. On the right is an example window, showing how those colours will look in use.

Say we want to make our window text turquoise, and our error emphasis (erroneous input fields and messages) red. We would move the cursor to the window text field, and type a T meaning turquoise. Without pressing Enter, we would Tab to the error emphasis field and type an R meaning Red.

We could press Enter now to save the changes and leave the window, but because we want to what the colours would look like we press F5=Refresh The example window will then show how those colours look in action. You may repeat the process, making other changes. When you are happy with the colour scheme press Enter and you will return to the main menu.

Note that the fields you leave blank assume their default values. The example window shows the defaults for SAA90, but these defaults differ for the SAA87 and I/S styles. To avoid unexpected defaults that throw your colour scheme out of gear, you may decide to enter a colour value in *all* the fields.

One final point. The application always has the last say in what colour a window shall be. This is to your advantage though. For example, the application should make warning messages a distinct colour (red say), to make it stand out from your normal window colours.

Press Enter if you have not already done so, and return to the main menu. Once there, pick option 2.

16.1.3 Warning Beep.

The CUA standard specifies that an audible beep should accompany warning and action messages. CUA also specifies that the user must have the ability to disable this sound if they so desire. By default the beep is silenced. The user must explicitly turn the beep by selecting this option.

Option `2` will display the window specifying whether or not to permit the beeping sound.

```
Warning Beep

  _    Audible Beep.

Place a slash beside the above option if you
want Neatwind to sound a beep when you make
an error.  Place a blank if you don't want
Neatwind to make any such beeping sounds.

F1=Help F12=Cancel
```

To permit the beeping sound, place a slash in the input field. If you want to silence the beeping sound place a blank there instead.

Press `Enter` to return to the main menu, then pick option `3`.

16.1.4 National Language Preferences.

```
National Language Preferences.

National Language Code:  FRB   +  (1st preference)
                         FRA   +  (2nd preference)
                         ENG   +  (3rd preference)
                         ___   +  (4th preference)
                         ___   +  (5th preference)

The languages you identify here will be used
whenever possible by Applications that use
Neatwind as their dialogue manager.
You may specify no preferences if you wish.
For a prompted list, point at the field with a
cursor and press the F4=Prompt key.

F1=Help F4=Prompt F12=Cancel
```

You may specify up to five languages, in order of preference. In the above example the user has specified Belgian French as their first preference, French (as spoke in France) as their second, and U.K. English as their third. You can find a list of all national languages and their three-letter codes in “Appendix A. National Language Codes.” on page 97.

If you want to see a list of selectable languages move the cursor onto the input field you want to fill in, then press `F4=Prompt`. Neatwind will display a further window. You can scroll through this window, and place a slash beside the language you want to use and press `Enter`.[15]

Unlike the other options in the Control Panel, this one is *not* automatically invoked. It would seem silly to have Neatwind give its messages in French if the panels are in English. In fact, it could be downright confusing. What if the English help text told the user to press `Cancel`, when the PF key is labelled `Annuller`?

---

[15] This window will only list languages that have had a `NEATNLS` file defined, as described in “Creating the NEATNLS file.” on page 74. This file must be created in order to select a language.

Thus Neatwind leaves it up to the application to find a language that both Neatwind and the application supports. This has already been discussed in “Integration.” on page 78.

Now, press `Enter` to return to the main menu, then take option `4` to set our country.

16.1.5 Country.

```
Country

Country Code: __ +

Enter your two-letter country code and
press Enter.  For a list of possible
country codes press F4=Prompt.

F1=Help F4=Prompt F12=Cancel
```

Each country is specified by a two-letter code. A list of country codes is in “Appendix D. Country Identifiers.” on page 103.

Type in the two-letter code of your country, and press `Enter`. If you do not know your country's two-letter code, press the `F4=Prompt` key and you will be presented with a list. Scroll through it and place a slash beside your country and press the `Enter` key.

The country code and name you select will be available to the application program. It will most likely use it to decide how to format numbers, what date format to select, and so on. This setting has no direct affect on Neatwind itself.

One you have finished press `F3=Exit` to exit out of the Control Panel. From now on, when you run a Neatwind application you will find the selections you have made will be in effect.

16.2 Application Considerations.

This section discusses what you should keep in mind when coding your application.

16.2.1 Respecting the user's choices.

As an application programmer you should consider that the user has the ability to define their own colour schemes. If you explicitly set the application colours using the `FOREGROUND` and other options you will override user's own selections.

Similarly, hardcoding colours into the window definitions you will have similar problems. For example, if you insist on hardcoding `%W` for white instead of `%H` for highlighting, the user would be unable to tailor that to their own taste.

Thus, you should only set colours if you really want to get the user's attention. For example, you might colour a warning window white to draw attention to it. On all other windows and screens, don't specify any colouring options and let the user apply their own.

16.2.2 Changing preferences from within.

In order to let the user change their preferences in the middle of your application, we recommend you make the control panel accessible off the action bar or a PF key.

You can invoke the Control Panel with the following code fragment:

```
'EXEC NEATCONT'
```

You can also pass `COLOURS, COLORS, BEEP, NLS` or `COUNTRY` as a parameter if you want to skip the control panel's main menu and access a function directly. You could do this if you want to integrate the control panel into a pull-down menu. Thus:

```
do forever
 'MAKEBUF'
  Queue 'Dear IBMer,'
  Queue 'By the time you read this I will be wrapped in plastic.'
  Queue '                                    signed, Laura Palmer.'
 'EXEC NEATWIND 3 (MESSAGE SAA90' ,
                  'ACTION (File=STD' ,
                      'Options (1. Colour=N@2. Sound=N@3. Language))'
  $rc = rc
 'DROPBUF'
  select
    when $rc = 9998 then  /* Action Bar */
      select
        when action$name = 'OPTIONS' then
          select
            when action$opt = 1 then 'EXEC NEATCONT COLOURS'
            when action$opt = 2 then 'EXEC NEATCONT BEEP'
            when action$opt = 3 then 'EXEC NEATCONT NLS'
          end
        when action$name = 'FILE' then
          ...                              /* File pulldown processing */
      end
    when $rc = 0 | $rc = 9999 then         /* F3=Exit or Enter leaves  */
      leave
  end
end
```

16.2.3 Disabling the Control Panel.

You may disable the colours and beep option the user specified in the control panel by specifying the `NOPROFILE` option when you call Neatwind. The colours will thus assume the defaults under their style (eg. `SAA90`), and the beep will be silenced.

17.0 The Programmer's Toolkit.

This chapter discusses programs which are also included in the Neatwind package. These are utility programs, aimed at making your life a little easier.

17.1 The NEATQRY Program.

This program serves as the interface between your application and the Neatwind Control Panel. You can also use it to find out some useful information about the user's terminal.

`NEATQRY` is called as a function. When you call it, pass one or more attributes from the below table. It will return the values of those attributes as its result.

Parameter	**Meaning**
NLS	NLS codes in order of preference.
COUNTRY	Two-letter Country code if supplied.
COUNTRY_NAME	Country Name if code was supplied.
COLOURS or COLORS	List of colours selected in the control panel in "areaname colour" pairs.
BEEP	Boolean; Is the BEEP enabled?
MAX_ROWS, MAX_COLS	Maximum screen size.
COLOURAVAIL	Boolean; Terminal supports colour?
EXTHI	Boolean; Extended highlighting available?
APL	Boolean; APL character set available?
PS2	Boolean; Is this terminal a PS/2 or PC?
MYTE	Boolean; Is PC or PS/2 running the MYTE emulator?
PSS	Boolean; Programmable Symbol Sets available?
PSSLIST	List of Programmable Symbol Set. eg "AB."
GESC	Boolean; Supports the graphic escape characters?
CODEPAGE_KNOWN	Boolean; Codepage and Charset GSCID known?
CODEPAGE	Codepage of base character set if known.
CHARSET	GSCID of base character set if known.
DELIMITER (str)	Use this string enclosed in parenthesis to delimit responses of attributes to the right of this one. The default is a single blank.

For example, to find out whether or not your user has enabled beeping from the Control Panel execute the following code fragment:

```
if neatqry('BEEP')
then Say 'Beeping enabled.'
else Say 'Beeping disabled.'
```

As another example, we could query the codepage of our terminal and whether or not it is a PS/2 by executing the following REXX statement:

```
Parse Value neatqry('PS2 CODEPAGE') With isaps2 codepagenr
Say 'Codepage:' codepagenr
if isaps2
then Say 'This terminal is a PS/2 or IBM PC.'
```

If NEATQRY encounters an error, it will simply return the keyword ERROR. You should check for this and abend your program if it is returned. It indicates either a failure within the terminal, or that the control panel values are incorrect.

17.2 The NEATTRAN Program.

Files containing multilingual characters (such as “ö”) can be in two forms:

- Codepage Independent.
  These files have the multilingual characters coded using NLS escape sequences, as described in “NLS Character Representation.” on page 69. For example, #do.
- Codepage Dependent.
  These files have the multilingual characters typed directly from the keyboard. For example, “ö.” This file is dependent on the codepage of the terminal that is being used. If a codepage dependent file is displayed on a terminal with a different codepage the multilingual characters will be garbled.

The NEATTRAN program provides an easy way to convert files between these two formats. Let's look at an example that converts a file from codepage dependent to codepage independent format.

17.2.1 Dependent to Independent Translation.

Here is an example of a file in codepage dependent format. We have named it POEM EXAMPLE.

Lorsque tout me ravit, j'ignore
Si quelque chose me séduit.
Elle éblouit comme l'Aurore
Et console comme la Nuit;

"Toute Entière"
Charles Baudelaire
20 Avril 1857

To convert this file into codepage independent format we would type the following as a CMS command:

```
NEATTRAN POEM EXAMPLE A (LOCAL TO NLS
```

If you now display the file on your terminal you will see the multilingual characters have been replaced by their NLS escape sequence equivalents:

Lorsque tout me ravit, j'ignore
Si quelque chose me s#feduit.
Elle #feblouit comme l'Aurore
Et console comme la Nuit;

"Toute Enti#bere"
Charles Baudelaire
20 Avril 1857

The option has the syntax `FROM format TO format.` You may optionally omit the `FROM` or `TO` keywords. The `format` can either be a specific codepage (eg. 00279 for France), `LOCAL` to default to the codepage of your terminal, or `NLS` meaning a file containing NLS escape sequences. If the file was originally entered on your terminal or another with the same codepage you may say `LOCAL.` However if someone with a U.S. terminal is sent a file typed on a French terminal they would specify the French codepage instead. As the French codepage is 00279 the command would be:

```
NEATTRAN POEM EXAMPLE A (00279 TO NLS
```

A word of warning; this translation process is semiautomatic. You will usually find it works, but you are advised to check the results. Strings that are ambiguous such as “ae” or “B” (which could be approximating the æ dipthong or German ß) will not be translated. After all, the person who types a “B” might mean either “Bear” or “groß.” To resolve differences like that `NEATTRAN` would need a German/English dictionary and a lot of smart logic it hasn't got, so do take a moment to check the output.

In practice you could type your `NEATPANL` files with native multilingual characters and use `NEATTRAN` convert it when you have finished. You could probably do this much quicker than if you typed in the NLS escape sequences manually.

17.2.2 Independent to Dependent Translation.

To translate a file from NLS format into codepage dependent is simply the converse of the example we gave above. Thus:

```
NEATTRAN POEM EXAMPLE A (FROM NLS TO LOCAL
```

If we were converting a file to send to someone in Brazil (codepage 00275) we could say:

```
NEATTRAN POEM EXAMPLE A (NLS TO 00275
```

We could place the translated text in a different file by specifying another file name:

```
NEATTRAN POEM EXAMPLE A POEM BRAZIL A (NLS TO 00275
```

As the NLS escape sequences are all unqiue there is no need to check for unprocessed amiguous strings.

You probably won't have much cause to use the `NLS to LOCAL` conversion discussed here, but it certainly has potential for exchanging files between countries where the text must not change. Files of stock or company names might be one example.

If you omit the arguments or options when you call `NEATTRAN` then a fullscreen panel will appear and prompt you supply the missing data.

18.0 Think of the User.

Your application should flow with the user. Ideally, the next thing you expect them to do from any screen should be only a few key strokes away.

Don't go overboard with windows. Throwing up a message window to report every simple action can get very tedious for the user. For example, if a user picks the option to save an object then they expect it to be saved, so why throw up a window that states the obvious? Even worse, it breaks their train of thought and they must answer that window to get rid of it.

A much better solution is to use a message line at the top or bottom of your screen for non-critical or obvious "everything is going well" messages. Only use a window to display messages if they are too long to explain in a single line, or are of a critical nature. In Neatwind the `MSG ()` option is used to display text on the message line.

Normal windows would have up to five components in their layout. These depend on its type:

1. An icon. This lets them identify the nature of the interaction immediately. For example, a yellow and black warning symbol if they are receiving a warning.

2. A brief message. A single line message next to the icon. This allows user familiar with the application to work out what is going wrong without having to read the detailed description which follows.

3. Explanation. This is a more detailed explanation of what the window is about. It elaborates on the brief message given above.

4. What next? If applicable, tell the user what they may do next. If your window is a message to tell them they can't do something, then tell them what they have to do so they can. Using your application shouldn't be a puzzle for the user.

5. Nastygram. Optionally some application designers include Nastygrams for specific dialogue. These are typically a catenation of the application name, a function code, a number and a severity indicator (ie. W for Warning, E for Error, Q for question). For example, CUGU047E. These run the risk of scaring a complete novice, but do allow you to easily locate an interaction within your application. The user can just supply the screen number and nastygram, rather than referring to "the window that comes up after I pick save from the third option on the second screen."

Keep the wording of the panels nice and simple. They should be easy to read and very clear. Don't use fifteen or fifty words when you can use five. Write for a ten-year old.

Windows and panels should be as uncluttered as possible. Crowded panels tend to turn people off. Ideally, each panel or window should have one distinct logical action. For example, select the next option, or edit the employee object.

Avoid technical terms in your application if you are writing for non-technical users. Talk in terms of the objects your application is transmitting. For example, in a report application you'd talk in terms of borders, table layouts and columns, rather than files, SCRIPT macros and server machines.

We've found users prefer applications when they talk in first person. For example, "I cannot print the report because you haven't selected the printer," rather than "Output Printer not defined." Of

course, some people may find this a little too cute to handle. Then again, these people probably sleep and shower in a three-piece suit. :-)

Try and lead the person through the application. If they make a mistake, if it is simple enough why not fix it with in the application? If they should know about it, then ask them via a window what they would like to do.

For example, if the user hasn't defined their printer and try to print off a document there are two things you could do. You could simply refuse, and tell them to go back and define their printer. A better choice would be to tell them they haven't defined their printer, and ask them if they'd like to do it now?

As another example, say the user is asked to enter a branch number in a panel, but gets it wrong. Rather than just telling them that, why not provide a window listing all branches and ask them to pick the correct one?

Context-sensitive help lets the user position the help key on a field of interest and press the Help key for an explanation. This makes it easier for the user to guide themselves through a new application. Neatwind provides this facility, so why not use it?

Prototyping is a good way to avoid having to make a large number of changes late in the project. You can use Neatwind to create a hard-coded demo version of the application, show it to your users and get feedback early on in the piece. Users tend to take a greater interest in the development stage of the application if they can actually see something. It also avoids misunderstandings which are more costly to correct later on.

Give the users feedback when you are doing something that may take some time. The “little clock” that appears on the bottom of peoples screens leaves people wondering if the system hasn't crashed again.

For relatively quick, atomic operations use the hour-glass icon provided by Neatwind. For longer operations use the “progress bar” icon. This shows an empty bar which slowly fills from left to right as the percentage of work done for this operation increases.

18.1 CUA Design Guidelines.

What follows is a terse summary of the “Common User Access Advanced Design Guide” SC26-4582 chapter on user interface design. This is well worth taking the time to read. This following summary overlaps with the authors own heuristics presented above.

The user should at all times be in control of the application, not the other way around.

Users should be able to develop a conceptual model of the application. For example, in a report generator the objects in the model may be headers and footers forming page boundaries, and columns forming tables, and page boundaries and tables together forming chapters of the report. This models how the user would *expect* the application to work, rather than some computer convenient model more convenient to the application programmer, such as SCRIPT macros and CMS flatfiles.

When you use a user-oriented model it's a lot easier for them to use the application, because they can intuitively figures out what to do by relating it back to the real world. For example, binding two chapters together is more intuitive than appending a flatfile for one to a flatfile of the other. The users' real world knowledge should be easily transferable.

The user interface must be consistent across the application.

The interface should be natural. The user should be able to think “I want to do this,” and do it easily and naturally. They shouldn't have to translate their original thoughts into some complicated or otherwise escape sequence.

Make the interface forgiving. Provide the “undo” feature. Users feel safer with an application that won't blindingly cause irreversible damage just because they pick the wrong option, not thinking before confirming an “are you sure” window.

Make the interface visual, rather than purely textual. People process visual cues faster than purely textual ones.

Provide immediate feedback. Eg. When they save an object, save “I have saved xyz.”

19.0 Support.

Neatwind is supported on a “best effort” basis by the author in his own time.

Please report problems, put forward requirements or make suggestions in the NEATWIND FORUM on the IBMVM conference disk. This is how support has been handled by Neatwind for some time, and it seems to be a satisfactory arrangement.

Please ask questions or make suggestions in the Forum, rather than sending personal correspondence to the author for the same. That keeps the author's reader free for “real work.” :-)

19.1 Prerequisites.

Neatwind is written entirely in REXX, and only uses the following standard program products and tools:

1. IOS3270 Display Input-Output Facility. 5785-HAX.

2. RXDISPIO from VMTOOLS.

   Neatwind uses this tool for operations IOS3270 is not capable of, such as saving and restoring the screen. RXDISPIO is supported actively, and for emergencies its source is available online.

3. RXVARS from CALLUP.

   This module is distributed with the official “Call Up” IBM Program Product. Although we could easily convert to use one of the alternate REXX variable passing tools on VMTOOLS, being part of a program product RXVARS is probably the safest.

4. PSLOAD from VMTOOLS.

   This is a fairly straight-forward REXX EXEC that downloads the NEATBOLD and NEATITAL PSET fonts using FSX commands.

5. FSX from VMTOOLS.

   FSX is being phased out of Neatwind, as it is no longer being actively supported. FSX is now only a prerequisite because PSLOAD needs it. Neatwind now uses RXDISPIO for low level screen I/O operations.

Because Neatwind uses only standard tools or programs, and is itself written in REXX it should be possible for your local I/S support person to find and fix any problem of a critical nature. If your problem can wait then use the forum.

19.2 The future of Neatwind.

With release 4.0 Neatwind had reached the level where is provides a complete CUA dialogue manager, though there will always be screens best written using alternate screen handlers. Neatwind will never be able to handle all possible cases. It aims to handle 90+% of likely application windows and panels.

It would be interesting to port Neatwind to run under OS/2 on PWS's. Presentation Manager and even the dialogue tag manager are a bit too complicated for someone not familiar with them who wants to get something done *fast.* A simple, easy to use dialogue manager might fit in nicely here for text-only applications. This would give Neatwind applications a high-level of portability, especially if written in conjunction with RXSQL. Do I hear any volunteers? :-)

Appendix A. National Language Codes.

For your convenience we have listed some of the national language codes in the below table. Should the code you desire not be listed here, refer to the "National Language Information-Design Guide, Volume 1 and 2." SE09-8001 and SE09-8002.

NLS Code	Language Name	Original Name	Principal Country
ARA	Arabic	Arabi	Arab Countries
AFR	Afrikaans	Afrikaans	South Africa
BGR	Bulgarian	Bulgarski	Bulgaria
CAT	Catalan	Catala	Spain
CHT	Traditional Chinese	Zhongwen	R.O.C.
CHS	Simplified Chinese		P.R.C.
CSY	Czech	Cesky	Czechoslovakia
DAN	Danish	Dansk	Denmark
DEU	German	Deutsch	Germany
DES	Swiss German	Schweizer-Deutsch	Switzerland
ELL	Greek	Ellinika	Greece
ENG	UK English	English	United Kingdom
ENU	US English		United States
ENP	English Upper Case		United States
ESP	Spanish	Espanol	Spain
FIN	Finnish	Suomi	Finland
FRA	French	Francais	France
FRB	Belgian French		Belgium
FRC	Canadian French		Canada
FRS	Swiss French	Suisse-francais	Switzerland
HEB	Hebrew	Ivrith	Israel
HUN	Hungarian	Magyar	Hungary
ISL	Icelandic	Islenska	Iceland
ITA	Italian	Italiano	Italy
ITS	Swiss Italian	Italiano svizzero	Switzerland
JPN	Japanese	Nihongo	Japan
KOR	Korean	Choson-o; Hanguk-o	Korea
NLD	Dutch	Nederlands	Netherlands
NLB	Belgian Dutch		Belgium
NOR	Norwegian - Bokmal	Norsk - Bokmal	Norway
NON	Norwegian - Nynorsk	Norsk - Nynorsk	Norway
PLK	Polish	Polski	Poland
PTG	Portuguese	Portugues	Portugal
PTB	Brazilian Portuguese		Brazil
RMS	Rhaeto-Romanic	Romontsch	Switzerland
ROM	Romanian	Romana	Romania
RUS	Russian	Russkij	USSR
SHC	Serbo-Croatian (Cyr)	Srpsko-hrvatski	Yugoslavia
SHL	Serbo-Croatian (Lat)		Yugoslavia
SKY	Slovakian	Slovensky	Czechoslovakia
SQI	Albanian	Shqip	Albania

SVE	Swedish	Svenska	Sweden
THA	Thai	Thai	Thailand
TRK	Turkish	Turkce	Turkey
URD	Urdu	Urdu	Pakistan

This table was derived from the **LANGUAGE CODES** file on the **IBMNLS** Tools Conference Disk.

Appendix B. Terminology of Codepages.

You might think of a *codepage* as a table with two columns; codepoints in one, graphic characters in the other. A codepage defines a mapping of codepoints to graphic characters.

A *codepoint* is simply a number, usually between 0 and 255 in most codepages. The codepoint is usually specified in hexidecimal (base 16) format. That is, between '00'x to 'FF'x.

The term *graphic character* is misleading. Such a character is either a numeric digit, a letter or a special characrer, such as a punctuation symbol. For example, “A” is a graphic character. They have nothing to do with graphics is the classic sense of lines, circles and squares.

Codepages are identified by a five-digit, right adjusted number. For example, the current U.S. codepage is number “00037.” Different terminals in other countries will have different codepages. On top of that, there may also be more than one codepage in use in a country at any one time. See “Appendix C. Codepages” on page 102 for a list of codepages.

As an example of the mapping codepages perform, the graphic character “A” occupies codepoint 'C1'x in the U.S. codepage.

All codepages share a common set of characters. These are known as the *syntactic character set.* These include the Latin alphabet (upper and lower case), digits and some punctuation symbols. One exception though (someone has to be different, don't they :-); the Japanese Katanka codepage does not have the lower case Latin letters.

Since the Latin based alphabets share so many common characters there is a character set known as *Latin 1.* Some examples are U.K. and U.S. English, French, German and Spanish. This lumps all latin alphabet and their derivatives (such as “ö”) into one common character set.

All codepages founded on the “Latin 1” character set are known as *CECP* codepages. That is, “Country Extended Codepage.”

A single byte character set has 256 characters (though some of these are reserved as special control characters). Single byte languages (such as those based on “Latin 1”) can fit their entire alphabet (both cases), digits and punctuation characters into the spare character slots.

Double Byte character sets are used by Asian languages. These languages have too many characters to fit into a single byte character set. These character sets use a two byte arrangement, and are known as DBCS character sets.

B.1 Syntactic Character Set.

By definition, the syntactic character set comprises a set of characters which are available on *all* terminals. Here are all the characters in this set:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z
0 1 2 3 4 5 6 7 8 9
, . : ; ? ( ) ' " / - _
& + % * = < >
```

Note that the explanation mark “!”, the tilde “~” and curly brackets “{ }” are excluded.

While the syntactic character set is available in all codepages they are not necessarily at the same codepoints. The split vertical bar, in particular, tends to move around.

There is always one exception to every rule, and that exception in this case is the Japanese Katanka keyboard. This keyboard does not carry the lower case letters from the syntactic character set. For this reason it is not possible to enter NLS escape codes (which use the lower case letters) directly off one of these keyboards.

Appendix C. Codepages

Here is a list of some of the current codepages.

There isn't a one-to-one correspondence between countries and codepages. For example, in Austria you will find most terminals running codepage 00273. You may however find some older ones running codepage 00007. There's also a good chance of finding some U.S. terminals (running codepage 00037), and perhaps a few running International codepages (00256, 00330 and 00500).

Codepage Number	Codepage Name
00037	United States. (also used in Australia)
00273	Germany and Austria.
00274	Belgium.
00275	Brazil.
00276	Canadian Franch.
00277	Denmark and Norway.
00278	Finland and Sweden.
00279	France.
00280	Italy.
00281	Japan.
00282	Portugal.
00283	Spain.
00284	Latin America (Spanish Speaking).
00285	United Kingdom.
00500	International.

A registry of all codepages is maintained on the **STDCODES** Tools Conference Disk via **EDATOOLS at RALVM11.** For a description of this disk request the file `00201003 TERS3820` from the **SISCONF** conference disk. Use `TERSE` to `UNPACK` the file which is in `LIST3820` format. The unpacked file can be printed on any 3820 or compatible printer.

Appendix D. Country Identifiers.

Countries are identified by a two-letter identification code. Here is a list of countries as derived from ISO standard 3166.

Country Code	Country Name	Country Code	Country Name
AD	Andorra	**KY**	Cayman Islands
AE	United Arab Emirates	**KW**	Kuwait
AF	Afghanistan	**LB**	Lebanon
AG	Antiqua	**LC**	Saint Lucia
AL	Albania	**LI**	Liechtenstein
AN	Netherlands Antilles	**LK**	Sri Lanka
AO	Angola	**LR**	Liberia
AR	Argentina		
AT	Austria	**LU**	Luxembourg
AU	Australia	**LY**	Libya
BB	Barbados	**MA**	Morocco
BD	Bangladesh	**MC**	Monaco
BE	Belgium	**MG**	Madagascar
BG	Bulgaria	**ML**	Mali
BH	Bahrain		
BI	Burundi	**MR**	Mauritania
BJ	Benin	**MT**	Malta
BK	Burkina Faso (Upper Volta)	**MU**	Mauritius
BM	Bermuda	**MW**	Malawi
BN	Brunei	**MX**	Mexico
BO	Bolivia	**MY**	Malaysia
BR	Brazil	**MZ**	Mozambique
BS	Bahamas	**NA**	Namibia
BU	Burma	**NC**	New Caledonia
BW	Botswana	**NE**	Niger
CA	Canada	**NG**	Nigeria
CF	Central African Republic	**NI**	Nicaragua
CG	Congo	**NL**	Netherlands
CH	Switzerland	**NO**	Norway
CI	Ivory Coast	**NZ**	New Zealand
CL	Chile	**OM**	Oman
CM	Cameroon	**PA**	Panama
CN	People's Republic of China	**PE**	Peru
CO	Colombia	**PG**	Papua New Guinea
CR	Costa Rica	**PH**	Phillippines
CS	Czechoslovakia	**PK**	Pakistan
CU	Cuba	**PL**	Poland
CY	Cyprus	**PR**	Puerto Rico
DD	(absorbed in DE, Oct 2, 90)	**PT**	Portugal
DE	Federal Republic of Germany	**PY**	Paraguay
DK	Denmark	**QA**	Qatar

DO	Dominican Republic	**RO**	Romania
DZ	Algeria	**SA**	Saudi Arabia
EC	Ecuador	**SC**	Seychelles
EG	Egypt	**SG**	Singapore
ES	Spain	**SD**	Sudan
ET	Ethiopia	**SE**	Sweden
FI	Finland	**SL**	Sierra Leone
FR	France	**SN**	Senegal
GA	Gabon	**SO**	Somalia
GB	United Kingdom	**SR**	Suriname
GH	Ghana	**SU**	Soviet Union
GM	Gambia	**SY**	Syria
GN	Guinea	**SV**	El Salvador
GR	Greece	**SZ**	Swaziland
GT	Guatemala	**TD**	Chad
GW	Guinea-Bissau	**TG**	Togo
GY	Guyana	**TH**	Thailand
HK	Hong Kong	**TN**	Tunisia
HN	Honduras	**TR**	Turkey
HT	Haiti	**TT**	Trinidad and Tobago
HU	Hungary	**TW**	Republic of China
ID	Indonesia	**TZ**	Tanzania
IE	Ireland	**UG**	Uganda
IL	Israel	**US**	United States
IN	India	**UY**	Uruguay
IQ	Iraq	**VE**	Venezuela
IR	Iran	**VU**	Vanuatu
IS	Iceland	**WS**	Western Samoa
IT	Italy	**YE**	Yemen
JM	Jamaica	**YU**	Yugoslavia
JO	Jordan	**ZA**	South Africa
JP	Japan	**ZM**	Zambia
KE	Kenya	**ZR**	Zaire
KR	Korea, Republic of	**ZW**	Zimbabwe

This table was derived from the **COUNTRY CODES** file on the **IBMNLS** Tools Conference Disk.

Index

A

Action Bar 15
  Cancel 15
  Colour 31
  Edit 39
  File 39
  Help 41
  Options 40
  Pull-down 37, 38
  Pull-downs 36
  Standard Pull-downs 41
  STD 41
  View 40
Application Design
  Clear wording 92
  First person 92
  Flow 92
  Leading them through 93
  Prototyping 93
  Simplicity 92
  Technical terms 92

B

Bit String 14
Bold
  See Emphasised Text, Bold

C

Characters
  Changing 66
  Field Mark 66
  Field Marks 57, 61
  Field Specification 73
  Input 12, 19, 66
  Invisible 66
  Multilingual 69
  Read Only 26, 28, 66
Codepages 69, 100, 102
Colors
  See Colours
Colours 3, 9, 28, 30, 62
Complaints
  See Grievances
Countries
  List of 103
CUA 3, 12, 15, 16, 17, 36, 38, 62, 93
Custom PF keys 15

D

Data Entry
  Changes to screen 29
  Cursor Positioning 33
  Dynamic Lists 54
  Getting Results Back
    GLOBALV 26
    OUTVARS 25
    REXX Variables 25
    Stack 26
  Highlighting errors 24
  Input field spacing 29
  Invisible fields 28
  Locking old input fields 14
  NEAT$CHANGED variable 29
  Read Only Fields
    See Read Only Fields
Date 21
  AFTER 22
  AM, PM 22
  BEFORE 22
  EUROPEAN 21
  FUTURE 23
  IBMEUROPEAN 21
  ISO 21
  JIS 21
  JULIAN 21
  Meridian 22
  NOW 23
  ONAFTER 22
  ONBEFORE 22
  PAST 23
  Restrictions 23
  Spaces 22
  TIME 21
  TODAY 23
  User defined format 22
Disappearing windows
  See Options, VANISH
Dynamic Lists

See Data Entry, Dynamic Lists

E

Emphasised Text 24
Bold 9
Control Characters
E - emphasised bold 9
F - foreground 9
H - hilite 9
I - italic 9
V - reverse video 9
% - escape 9
ESCAPE character 9
Flare 9
Highlighted 9
Inverse Video 9
Italics 9
Executive summary 1

F

Field Validation
Application checking 24
CHAR 20, 21
DATE 21
See also Date
EXEC 23
External function 23
LEN 20
LENGTH 20
LIST 20
MANDATORY 20
NUMERIC 21
OPTIONAL 20
Subranges 21
Character 21
Integer 21
Real 21
TIME 21
See also Date
WHOLE 20
Fullscreen panels 4

G

Grievances
See Complaints

H

Help
Context Sensitive 32
Defining a help key 15
Integrated 45
Pull-down 41
Question Mark 32
Types of 50
Usability 93

I

Icons 6
Filler Icons 7
List of Icons 7
More indicator 17
Progress Bar 8
Italics
See Emphasised Text, Italics
I/S 3, 15, 16

L

Languages
List of 97
Layout 92

M

Message line 16, 31
More indicator 17

N

National Language Support 68-80
Arabic 80
Asian 79
Bidirectional 79
CODEPAGE Option 71
Codepages 69, 102
DBCS Support 79
Escape Sequences 69
Table 70
Hebrew 80
Language Codes 97
Multilingual 69
NLS Option 73
NEATCODE File. 70
NEATHELP 52
NEATNLS File 73
NLS
See National Language Support

O

Objects 42
Options
  ACTCOLOUR 31
  ACTION 36
  ASSIST 42
  BACKROUND 3
  BEEP 66
  BLANK 24, 25
  BOLD 9
  BTITLE 64
  CHECKTYPE 24
  CODEPAGE 69, 71
  CODEPAGE ASSUME 71
  COLUMN 4, 14, 37
  COMPRESS 30
  CONFIRM 65
  CTLCHAR 66
  DATAENTRY 19
  DEFAULT 12, 14
  ENTEROK 64
  ERRFIELDS 24, 34
  ERRINPUT 31
  ESCAPE 9
  FLARE 9
  FOREGROUND 3
  FULLSCREEN 4, 17
  HELP 49
  HILITE 9
  HIRETURN 15, 83
  ICONCOLOUR 31
  INPUT 31
  ITALIC 9
  I/S 3, 15, 16, 62
  KEEPBOTT 17, 25
  KEEPTOP 17, 25, 59
  LENGTH 19
  LOCK 14
  MARGINS 56
  MENU 11, 16
  MESSAGE 2
  MSG 16
  NLS 73
  NOBLANK 14, 24, 25, 26, 27
  NOCHECK 16, 36, 37
  NOCODEPAGE 72
  NOENTEROK 64
  NOPROFILE 88
  NOUSEALL 4
  NOWAIT 6, 65
  OUTVARS 13, 24, 25
  PANEL 77
  PERCENT 8
  PERMISSION 27
  PF 15
  PICK 12
  PREFIX 60
  PROTECT 28
  PSLOAD 6, 67
  PSLOADBOLD 18
  QUERY 5, 16
  READONLY 27
  REVEALPOSN 33
  ROW 4, 37
  rrccc/s/z/l/c/v/w 33
  SAA87 3, 15, 16, 62
  SAA90 3, 15, 16, 62
  SCROLLSIZE 17
  SHOWSTEM 2
  SQUEEZE 38
  STD 41
  STRIP 26
  TEMPLATE 54
  TITLE 64
  TYPE 19
  UPPERCASE 26
  USEALL 4
  VANISH 64
  WAIT 66
  XCONFIRM 65

P

Panel files 77
Performance
  Compiled NEATWIND 81
  NUCXDROP 82
  NUCXLOAD 81
  REXX Compiler 81
PF keys 15, 62
  Action 15
  Bkwd 15, 16
  BWD 15, 16
  Custom 15
  DELETE 55
  effect of styles 16
  END 15, 16
  Exit 15, 16, 62
  FWD 15, 16
  HELP 15, 32
  INSERT 55
  PF13-24 15
  PF3 16, 63
  Synonyms 16
  3279 15
Positioning 4, 33
Prerequisites
  FSX 95
  IOS3270 95
  PSLOAD 95
  RXDISPIO 95
  RXVARS 95
Presentation Manager 95
Productivity 1
Programmable Function Keys 15
Programmable Symbol Sets 6, 18
Pull-downs
  See Action Bar, Pull-downs

R

Read Only Fields
  Backslash method 27
  PERMISSION 27
  PROTECT Option 28
  READ attribute 27
  READONLY 27
  WRITE attribute 27
Recursion
  See Recursion
Return Codes 83
REXX Variables
  ACTION$COL 37
  ACTION$OPT 37
  ACTION$OPTN 37
  ACTION$ROW 37
  ACTION$TYPE 37
  field values 24
  HELP$MAXFIELDS 32
  HELP$NAME 32
  HELP$NUMBER 32
  NEAT$CHANGED 29, 42
  NEAT$POSN 33
  Object stems 42
  OUTVARS 24
  Return codes 83

S

SAA
  See CUA, Options SAA87, SAA90
Screen size 4
Scrolling 15
  Horizontal 56
  Reserved areas 17
  Tailoring 17
  Vertical 17
Security 28
Styles 15, 16
Support 95

T

Time
  See Date
Time Delay 66
Titles 64
Trace
  negative return codes 15
Tutorials 47

W

Window text
  Stack passing 2
  Stem Variable passing 2
Window types
  DATAENTRY
    See Data Entry
  Long Delays 8
  MENU 11
  MESSAGE 2
  PICK lists 12
  QUERY 5
  Short Delays 6